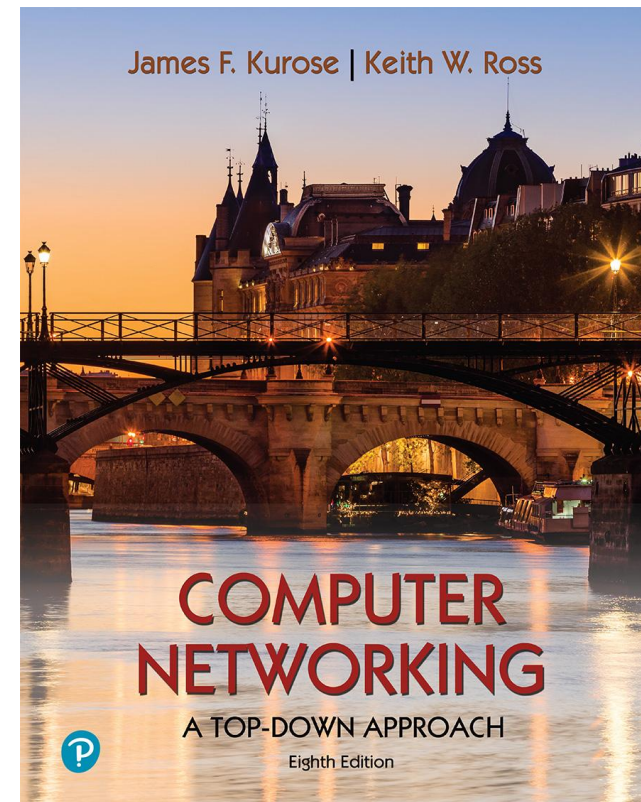


第3章 传输层



计算机网络：一种自上而下的方法

第八版

吉姆·库罗斯（Jim Kurose），基思·罗斯（Keith Ross）
皮尔逊，2020年

传输层：概述

我们的目标：

- 了解传输层服务背后的原则：
 - 多路复用与多路分解
 - 可靠的数据传输
 - 流控制
 - 拥塞控制
- 了解互联网传输层协议：
 - UDP：无连接传输
 - TCP：以连接为导向的可靠传输
 - TCP拥塞控制

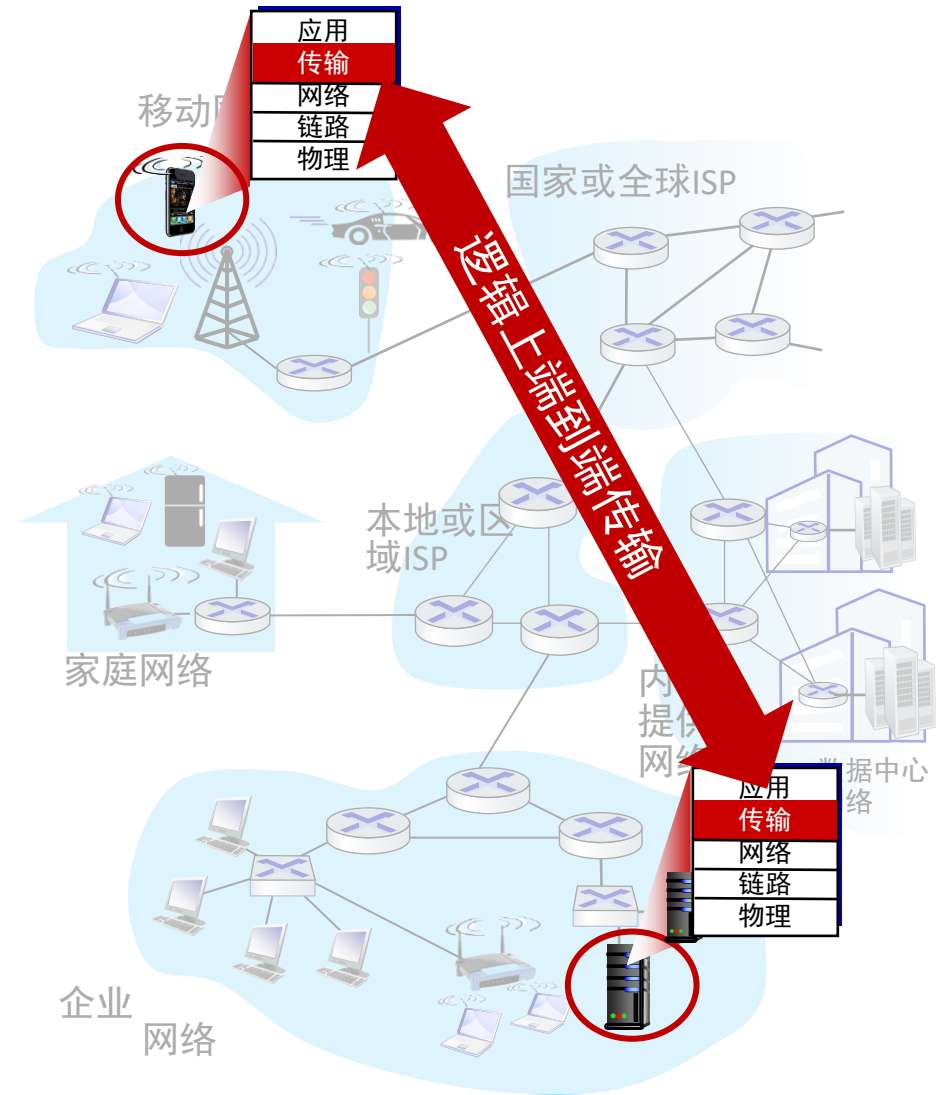
传输层： 目录

- 传输层服务
- 多路复用和多路分解
- 无连接传输： UDP
- 可靠数据传输原则
- 面向连接的传输： TCP
- 拥塞控制原则
- TCP拥塞控制
- 传输层功能的演变



传输服务和协议

- 提供在不同主机上运行的应用程序之间的**逻辑通信**
- 传输协议在端系统中的操作：
 - 发送者：将应用程序消息分为报文段，传递到网络层
 - 接收者：将报文段重新组合到消息中，传递到应用程序层
- 互联网应用程序可用的两个传输协议
 - TCP, UDP



传输与网络层服务和协议



一个类比：

安的家庭中的12个孩子向比尔家中的12个孩子发送信件：

- 主机=房屋
- 进程=孩子
- 应用消息=信封中的字母

传输与网络层服务和协议

- 网络层：主机之间的逻辑通信
- 传输层：进程之间的逻辑通信
 - 依靠，增强，网络层服务

一个类比：

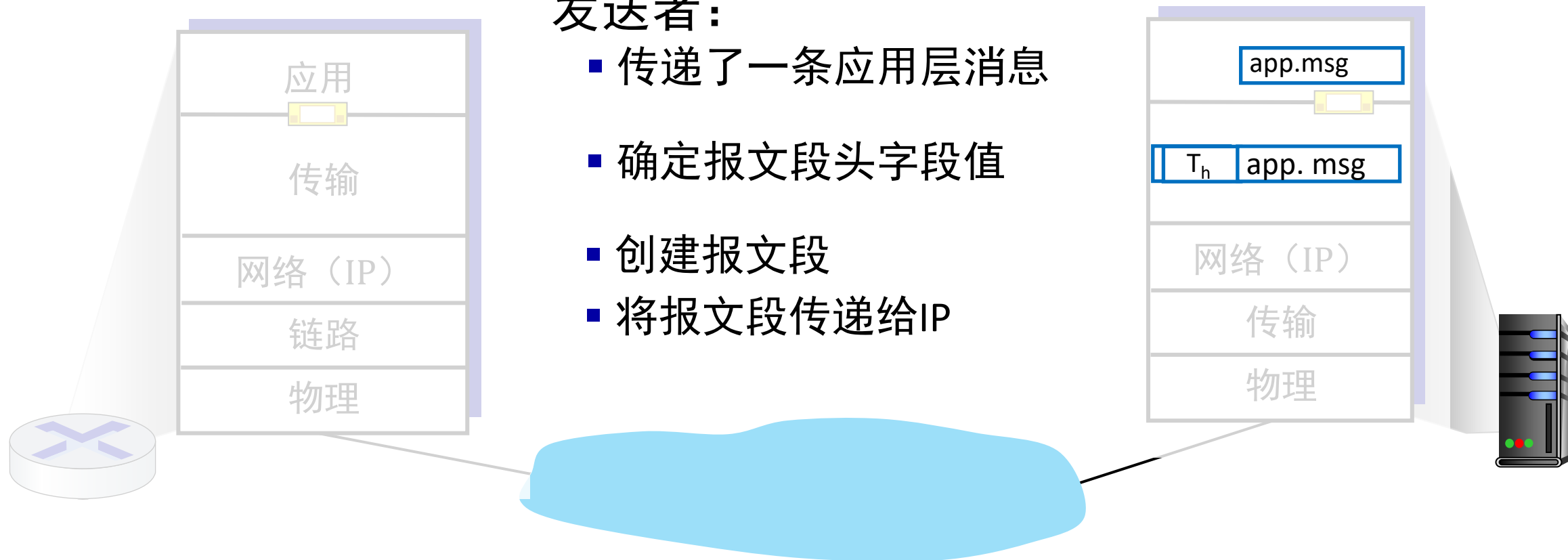
安的房子中有12个孩子向比尔家中的12个孩子发送信件：

- 主机=房屋
- 进程=孩子
- 应用消息=信封中的信件

传输层动作

发送者：

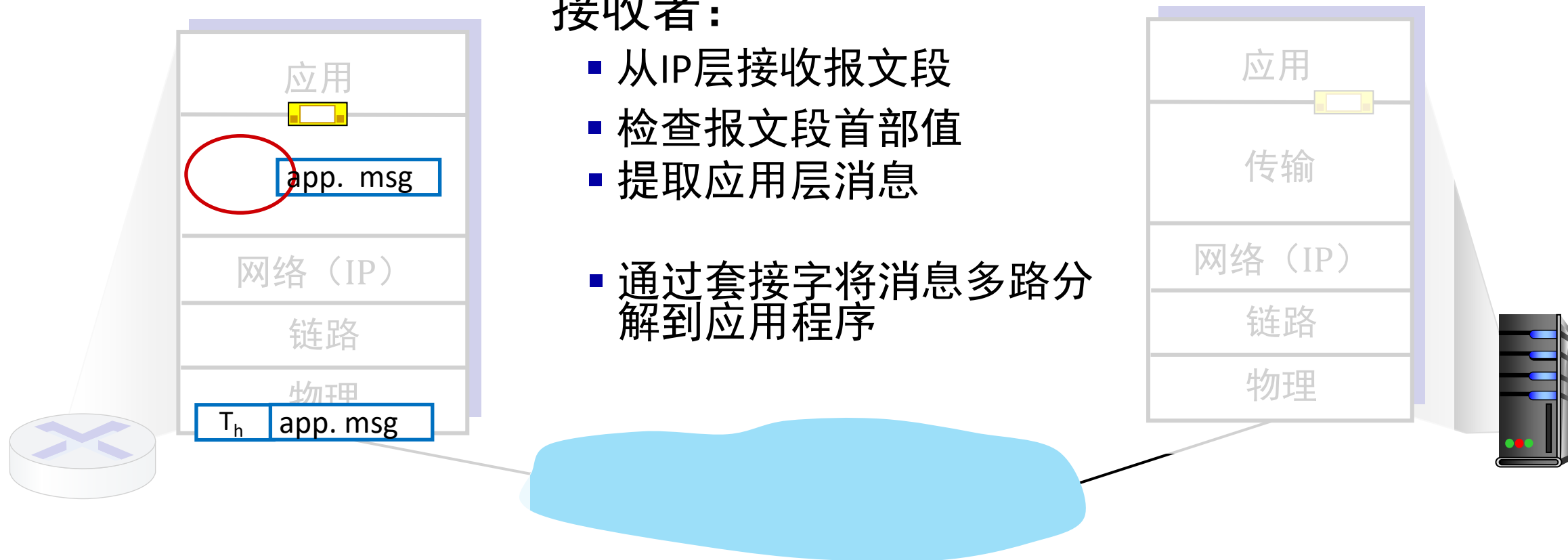
- 传递了一条应用层消息
- 确定报文段头字段值
- 创建报文段
- 将报文段传递给IP



传输层动作

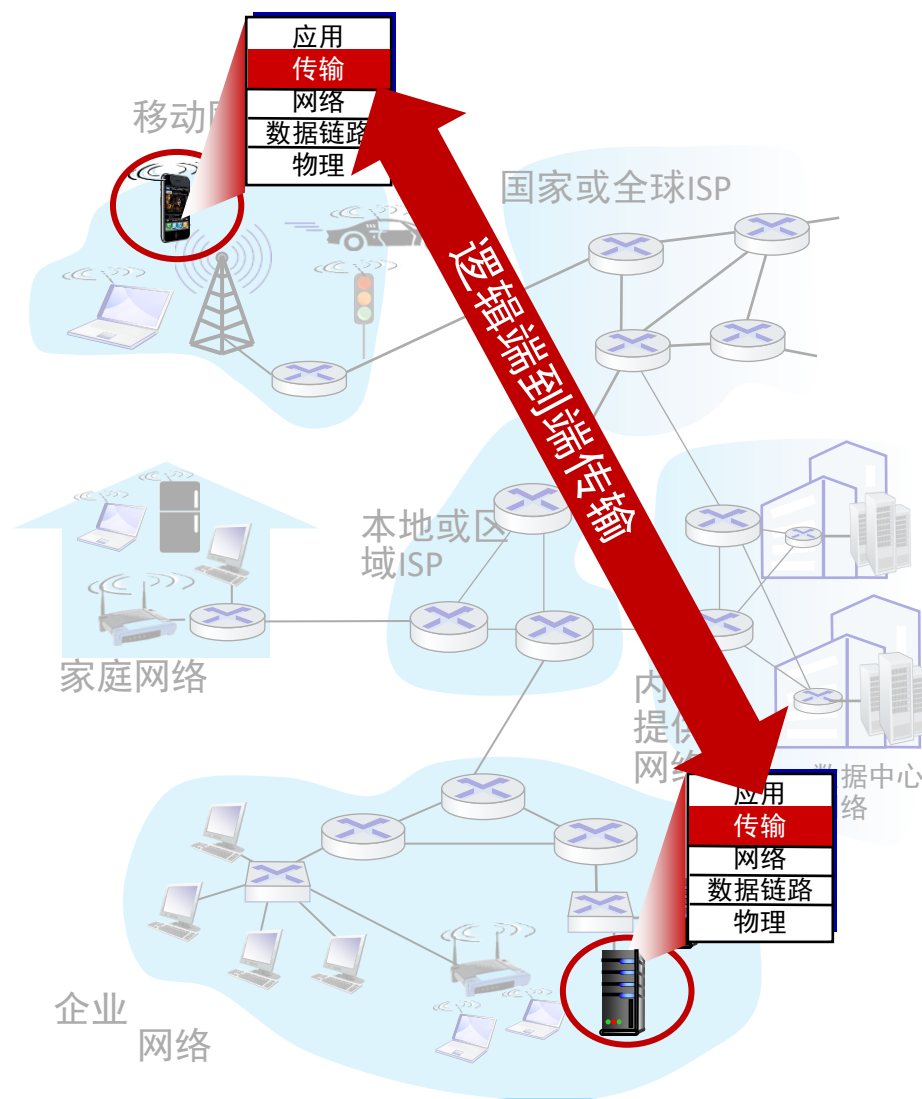
接收者：

- 从IP层接收报文段
- 检查报文段首部值
- 提取应用层消息
- 通过套接字将消息多路分解到应用程序



两个主要的互联网传输协议

- **传输控制协议**
- **(TCP: Transmission Control Protocol)**
 - 可靠，有序传输
 - 拥塞控制、流量控制、连接建立
- **用户数据报协议**
- **(UDP: User Datagram Protocol)**
 - 不可靠的，无序的交付
 - “Best-effort” IP的无约束扩展
- **不提供的服务：**
 - 延迟保证
 - 带宽保证



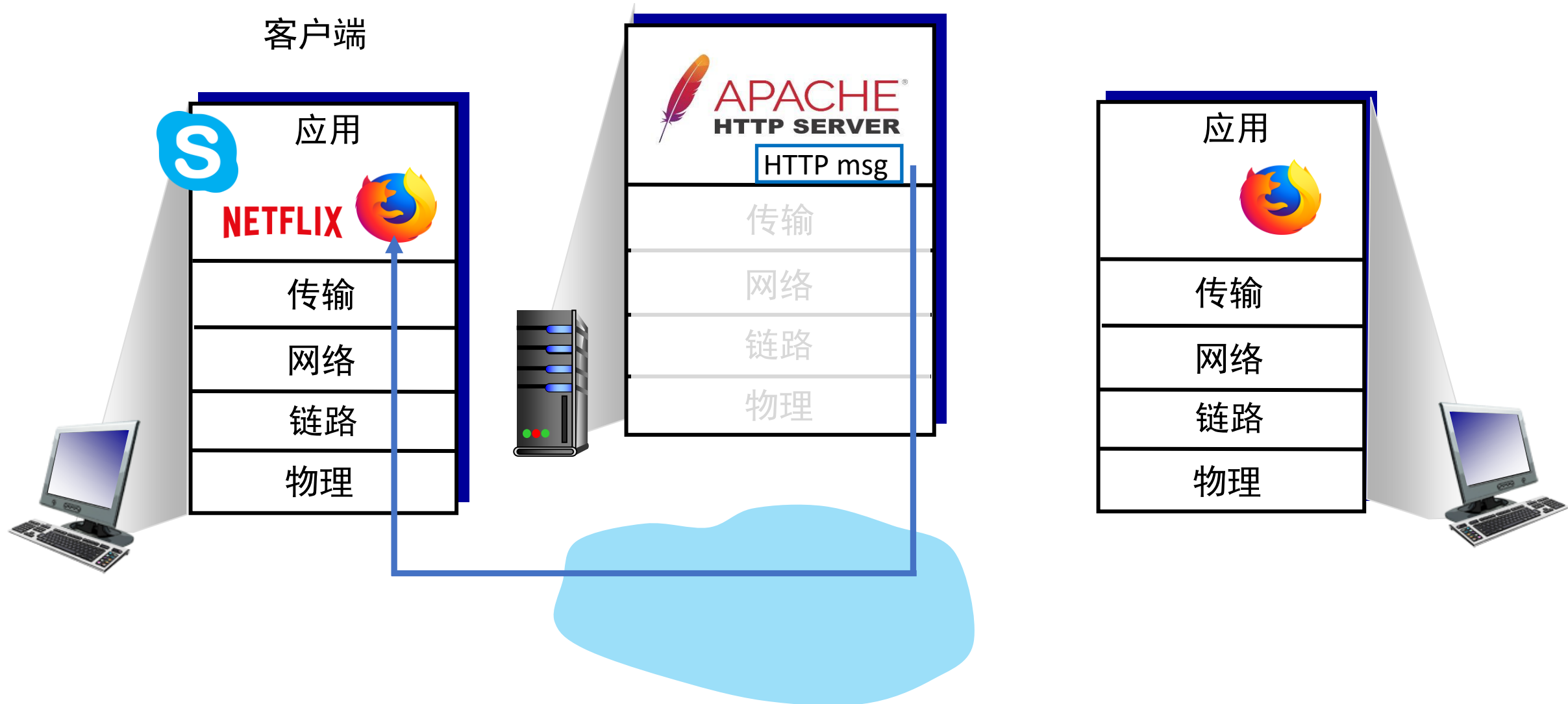
第3章： 目录

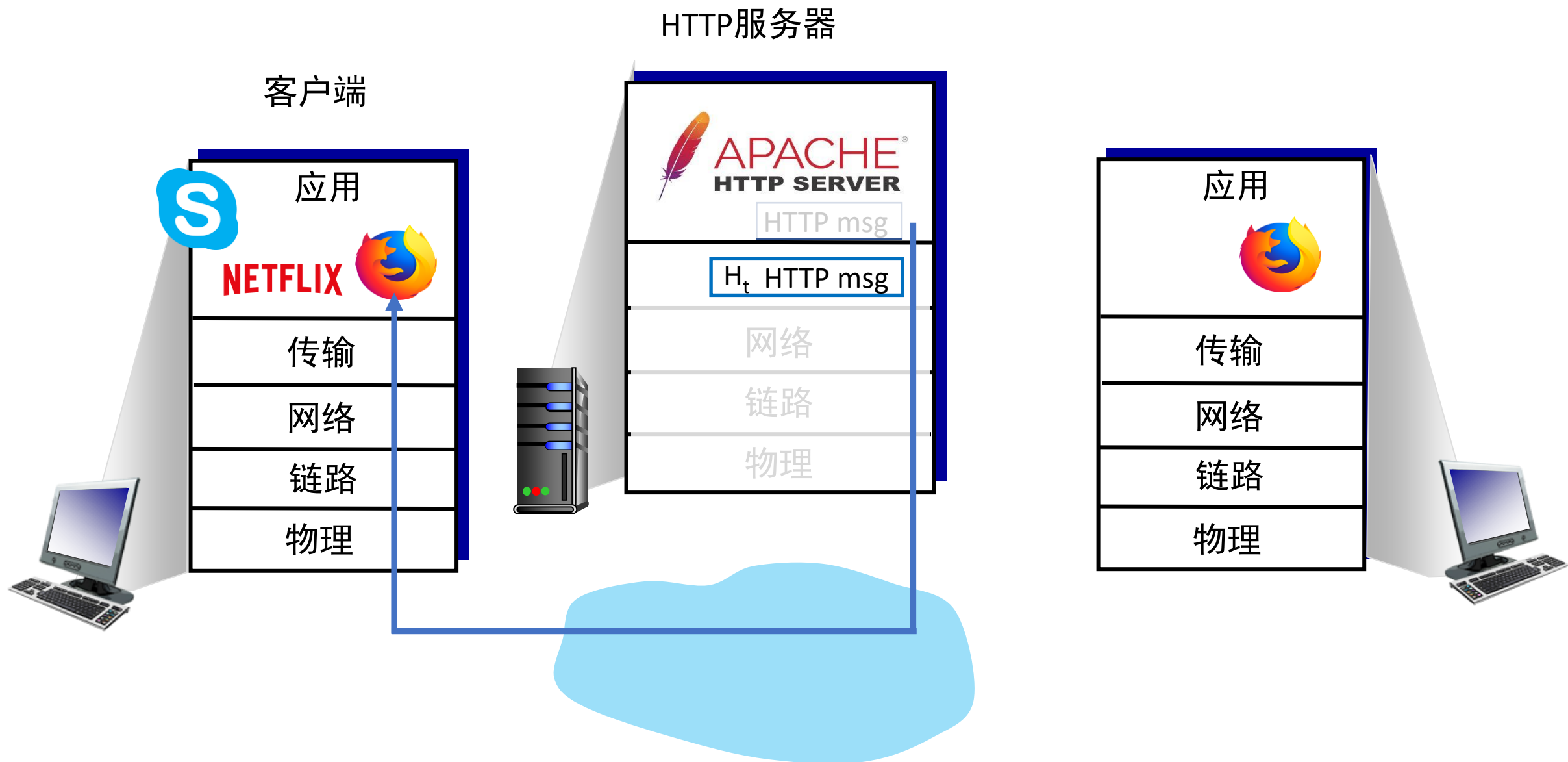
- 传输层服务
- 多路复用和多路分解
- 无连接传输： UDP
- 可靠数据传输原则
- 面向连接的传输： TCP
- 拥塞控制原则
- TCP拥塞控制
- 传输层功能的演变



HTTP服务器

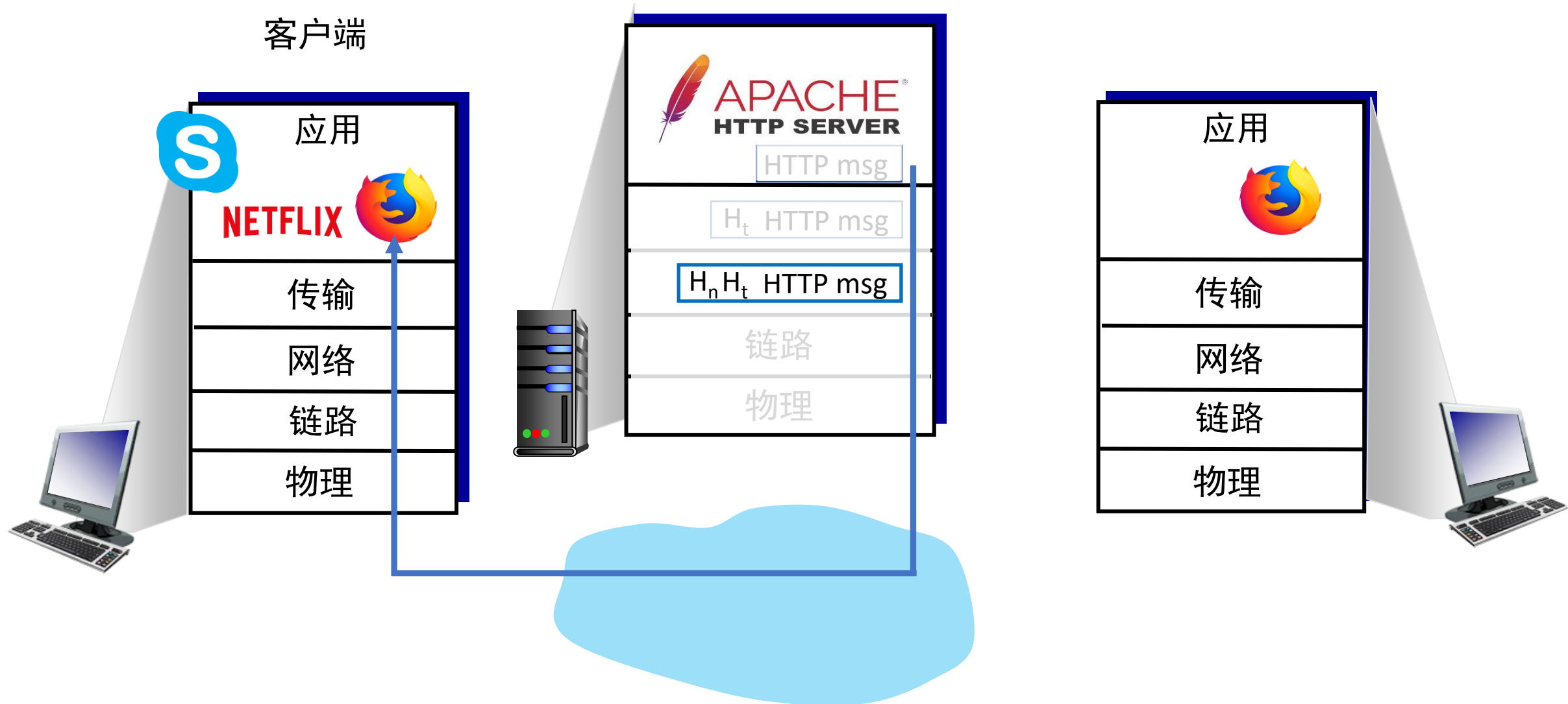
客户端

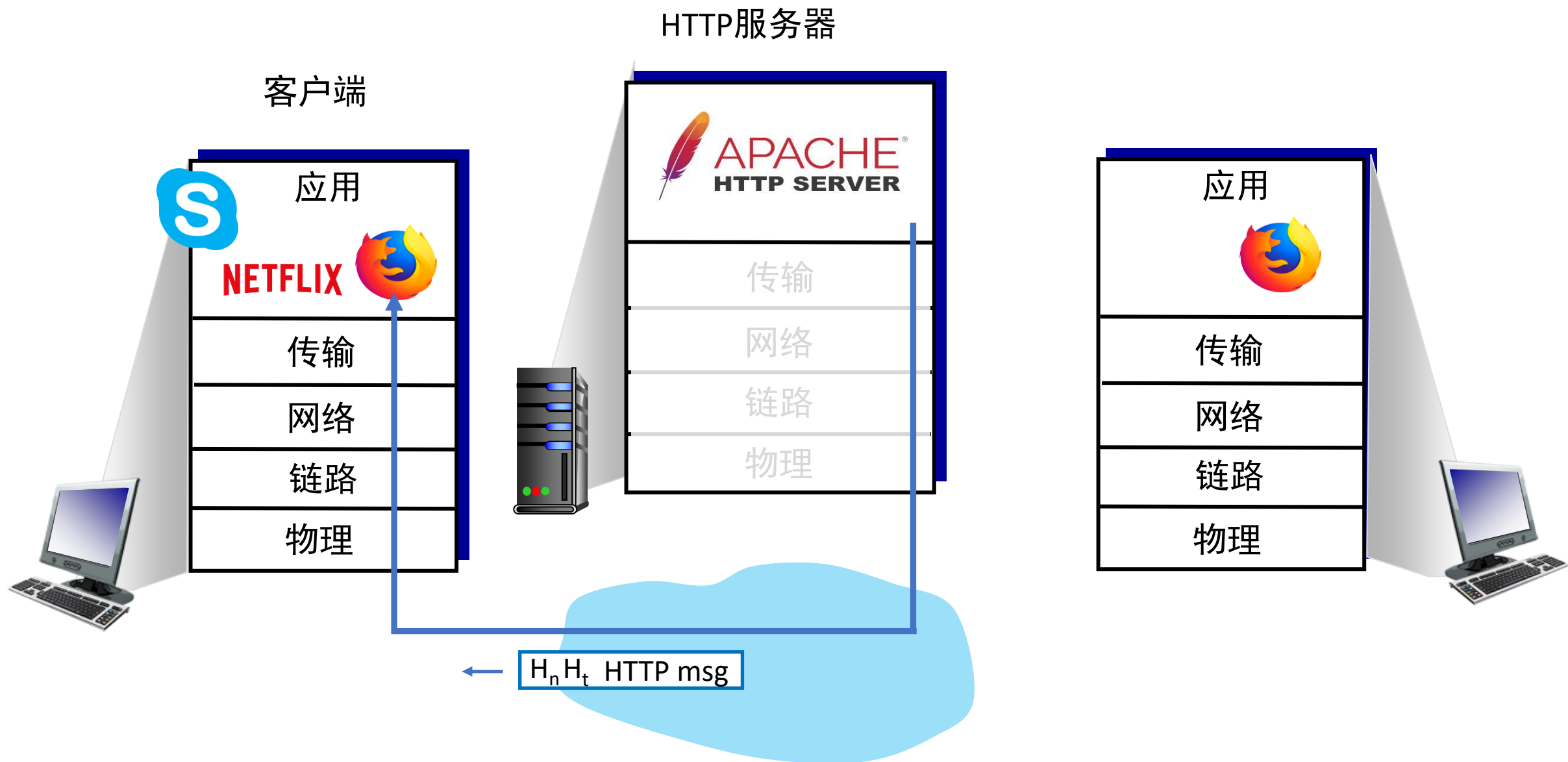


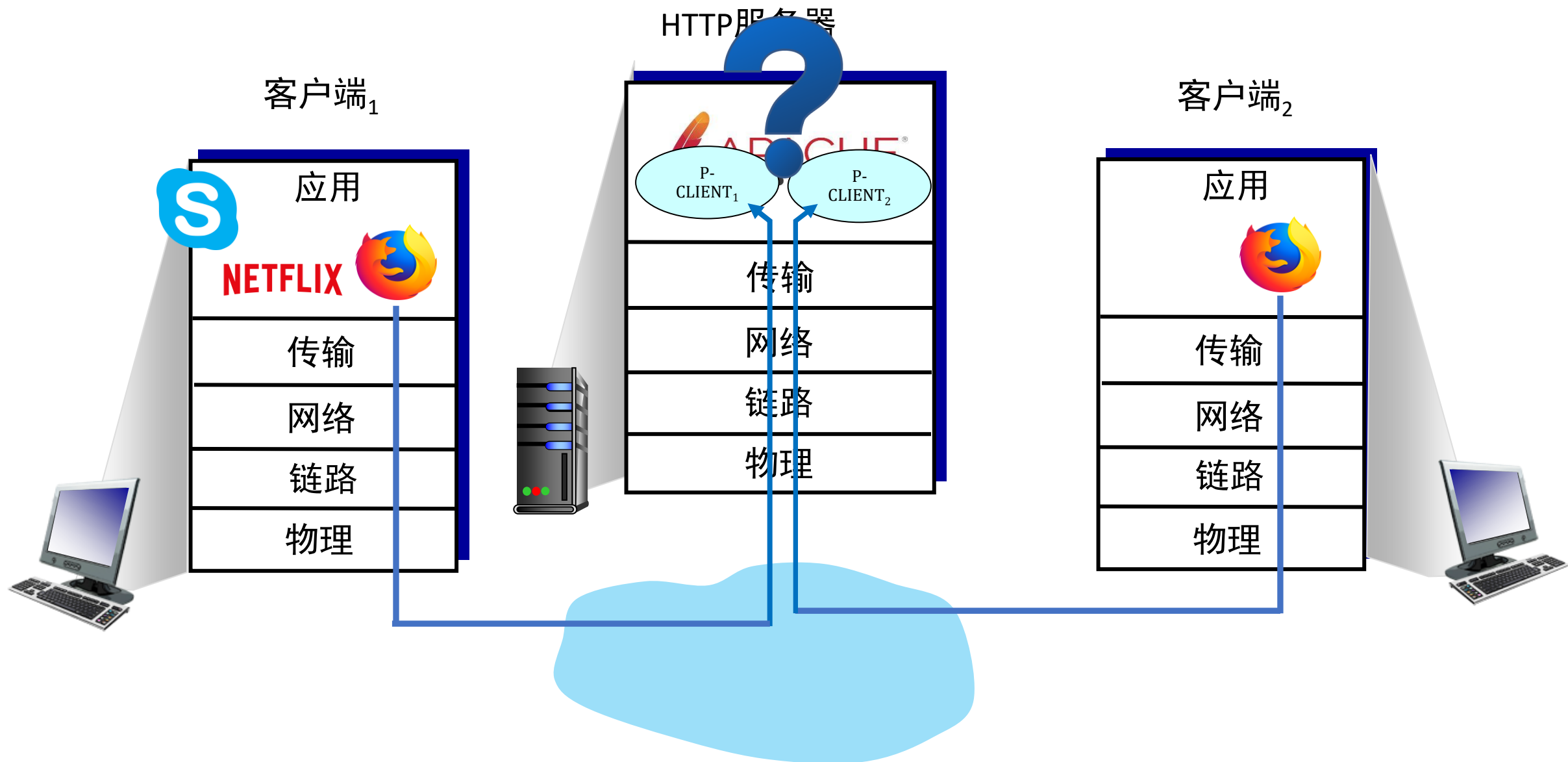


HTTP服务器

客户端







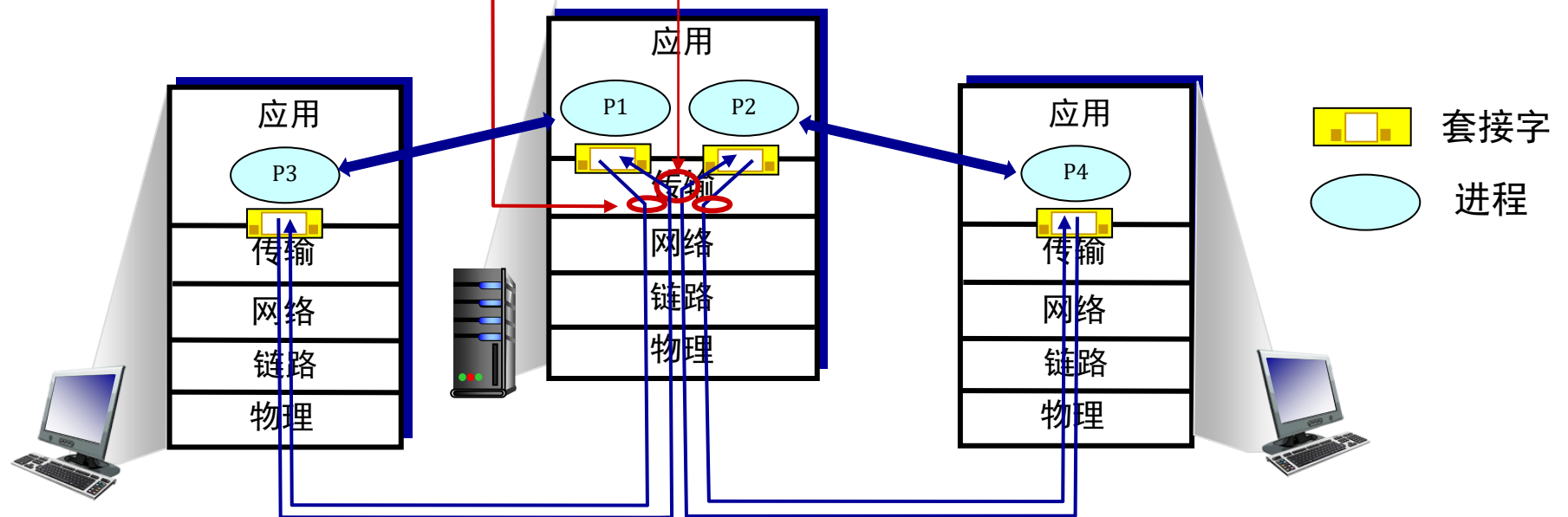
多路复用/多路分解

发送者的多路复用:

处理来自多个套接字的数据，添加传输头（稍后用于多路分解）

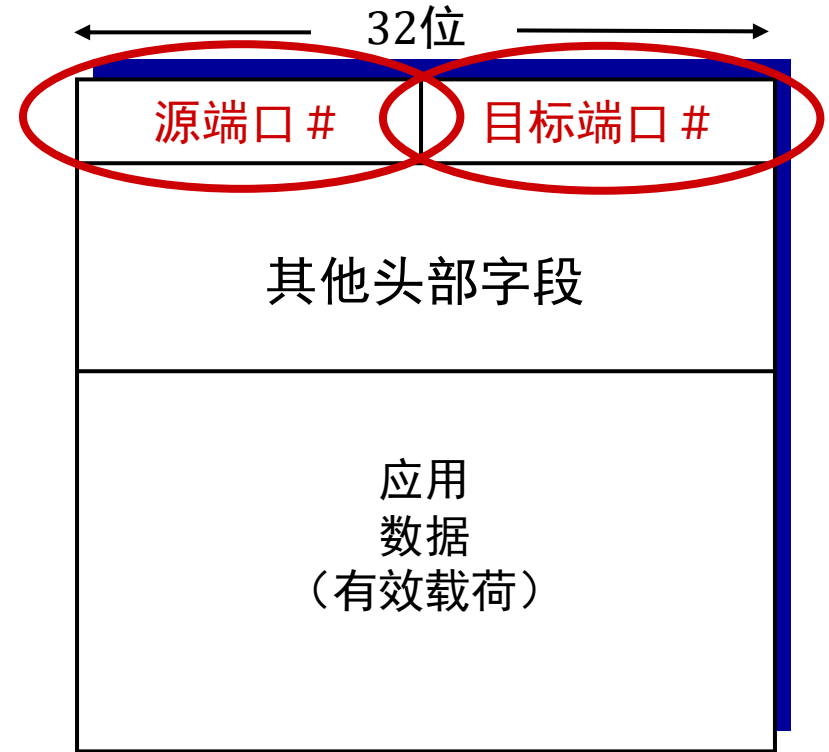
接收者的多路分解

使用传输层头部信息将收到的段发送到正确的套接字



多路分解的工作方式

- 主机接收IP数据报
 - 每个数据报都有源IP地址，目的IP地址
 - 每个数据报带有一个传输层段
 - 每个传输层段都有源，目标端口号
- 主机使用IP地址和端口号将报文段定向到适当的套接字



TCP/UDP段文格式

无连接多路分解

前文提要:

- 创建套接字时，必须指定主机本地端口 #：

```
DatagramSocket mySocket1  
= new  
DatagramSocket(12534);
```

- 当创建数据报发送到UDP套接字时，必须指定
 - 目标IP地址
 - 目标端口号 #

接收主机时接收UDP段：

- 在报文段中检查目标端口 #
- 将UDP报文段定向到端口号 #的套接字。



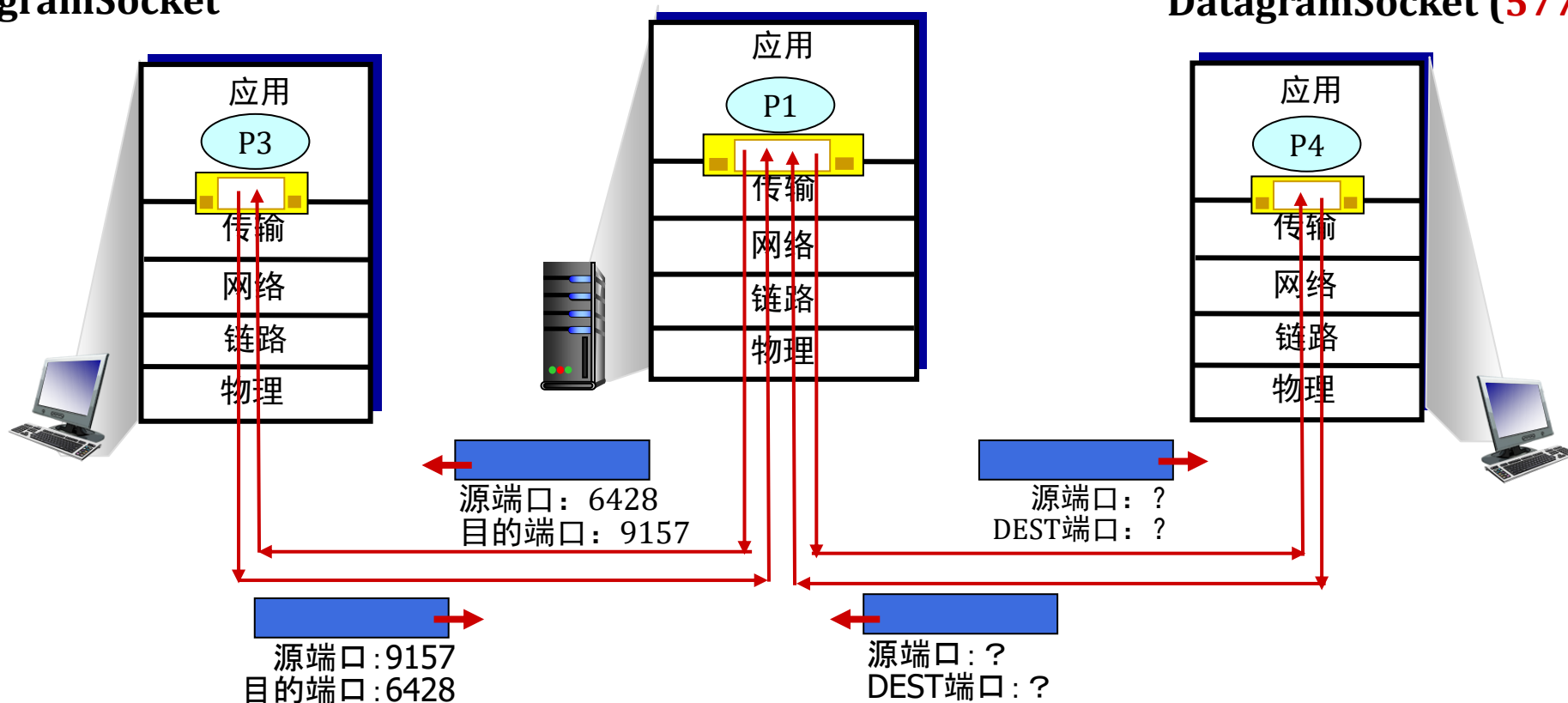
IP/UDP数据报具有相同的目的地端口号 #，但是不同的源IP地址和/或源端口号将被定向到接收主机的同一套接字

无连接的多路分解：一个例子

DatagramSocket mySocket2 =
new DatagramSocket
(9157);

DatagramSocket serverSocket =
new DatagramSocket
(6428);

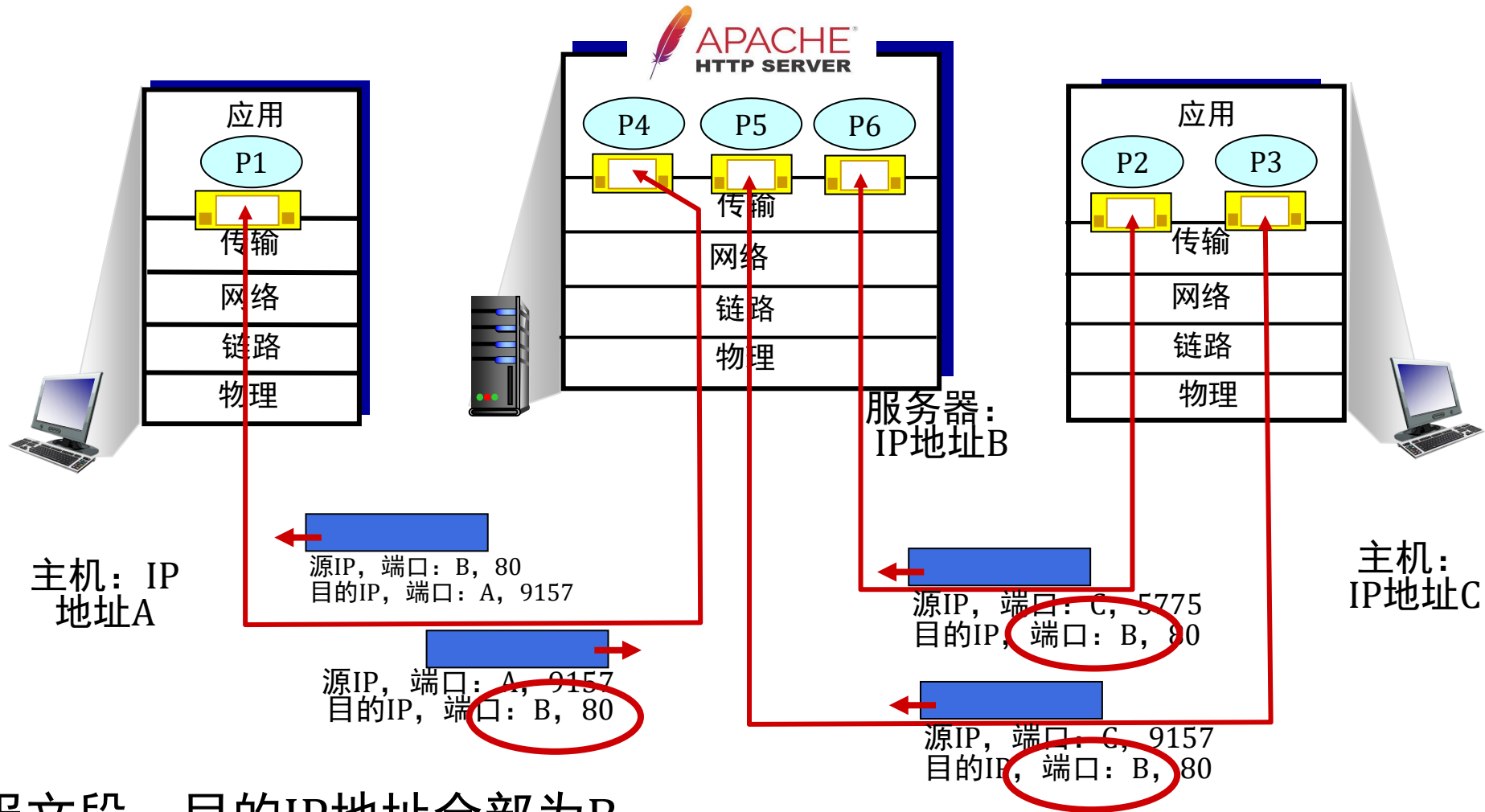
DatagramSocket mySocket1 = new
DatagramSocket (5775);



面向连接的多路分解

- TCP套接字由4元组标识：
 - 源IP地址
 - 源端口号
 - 目的IP地址
 - 目的端口号
- 多路分解：接收者使用所有四个值（4元组）将报文段定向到适当的套接字
- 服务器可以同时支持多个TCP套接字：
 - 每个套接字由其自己的4元组标识
 - 每个套接字与不同的已连接客户端相关联

面向连接的多路分解： 示例



三个报文段，目的IP地址全部为B，
目的端口：80，被多路分解去往不同的套接字

总结

- 多路复用，多路分解：基于段，数据报头部字段值
- **UDP**：仅使用目标端口号进行多路分解
- **TCP**：使用四元组进行多路分解：源和目标IP地址以及端口号
- 多路服用、多路分解发生在所有层

第3章： 目录

- 传输层服务
- 多路复用和多路分解
- 无连接传输： UDP
- 可靠数据传输原则
- 面向连接的传输： TCP
- 拥塞控制原则
- TCP拥塞控制
- 传输层功能的演变



UDP：用户数据报协议

- “简化版”，“基础版” 互联网传输协议
- “尽力而为(best effort)” 服务，UDP段文可能会：
 - 丢失
 - 无序交付给应用
- **无连接：**
 - UDP发送者，接收者之间没有握手
 - 每个UDP段独立于他人处理

为什么有UDP？

- 无需建立连接（这会增加RTT延迟）
- 简单：发送者，接收者之间没有连接状态
- 头部尺寸小
- 没有拥塞控制
 - UDP可以尽可能快地发送数据！
 - 在拥塞情况下仍然可以正常工作

UDP：用户数据报协议

- UDP使用场景：
 - 流媒体多媒体应用程序（耐损耗，对速率敏感）
 - DNS
 - SNMP
 - HTTP/3
- 如果需要在UDP上建立可靠的传输（例如，HTTP/3）：
 - 在应用层添加所需的可靠性
 - 在应用层添加拥塞控制

UDP: 用户数据报协议[RFC 768]

INTERNET STANDARD

RFC 768

J. Postel

ISI

28 August 1980

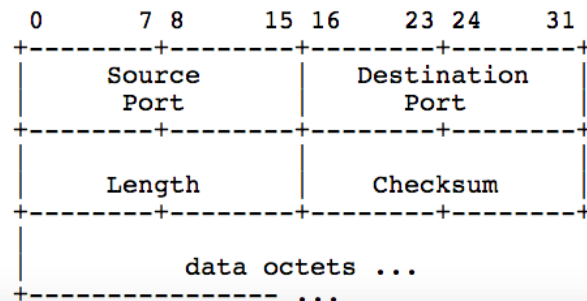
User Datagram Protocol

Introduction

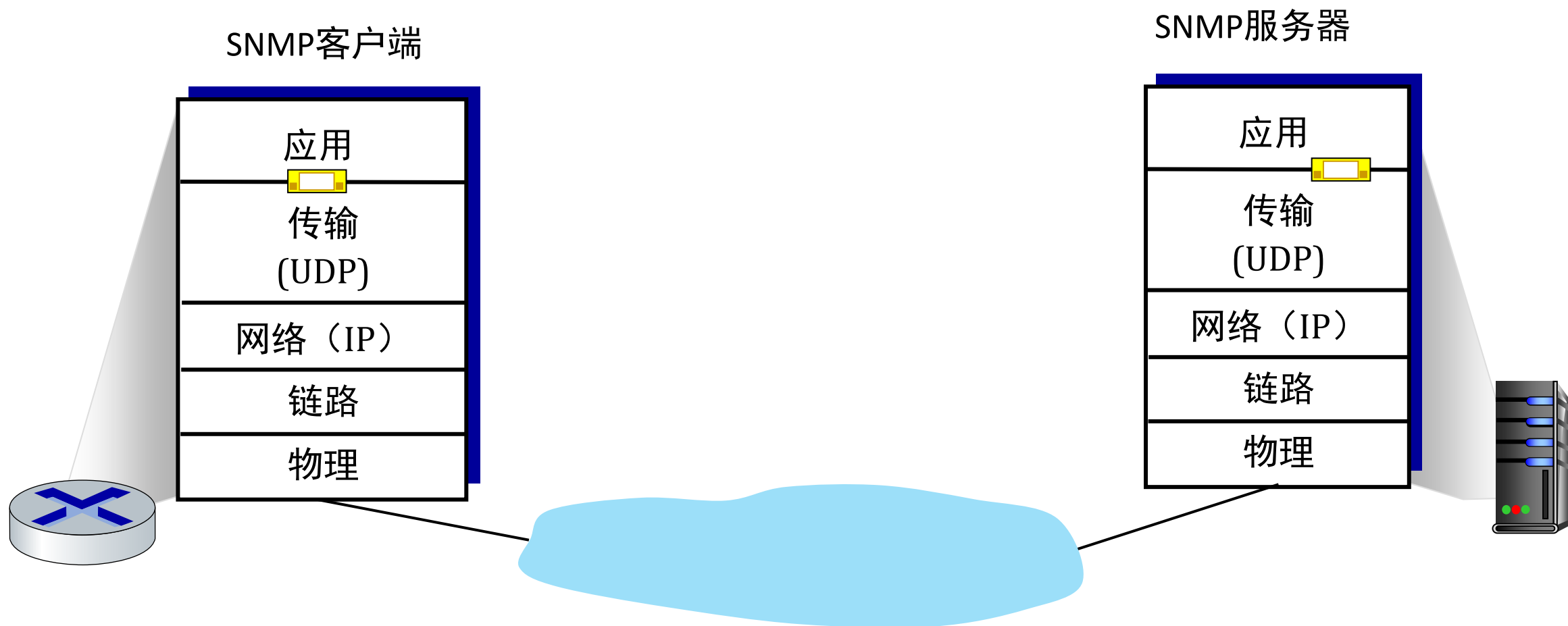
This User Datagram Protocol (UDP) is defined to make available a datagram mode of packet-switched computer communication in the environment of an interconnected set of computer networks. This protocol assumes that the Internet Protocol (IP) [1] is used as the underlying protocol.

This protocol provides a procedure for application programs to send messages to other programs with a minimum of protocol mechanism. The protocol is transaction oriented, and delivery and duplicate protection are not guaranteed. Applications requiring ordered reliable delivery of streams of data should use the Transmission Control Protocol (TCP) [2].

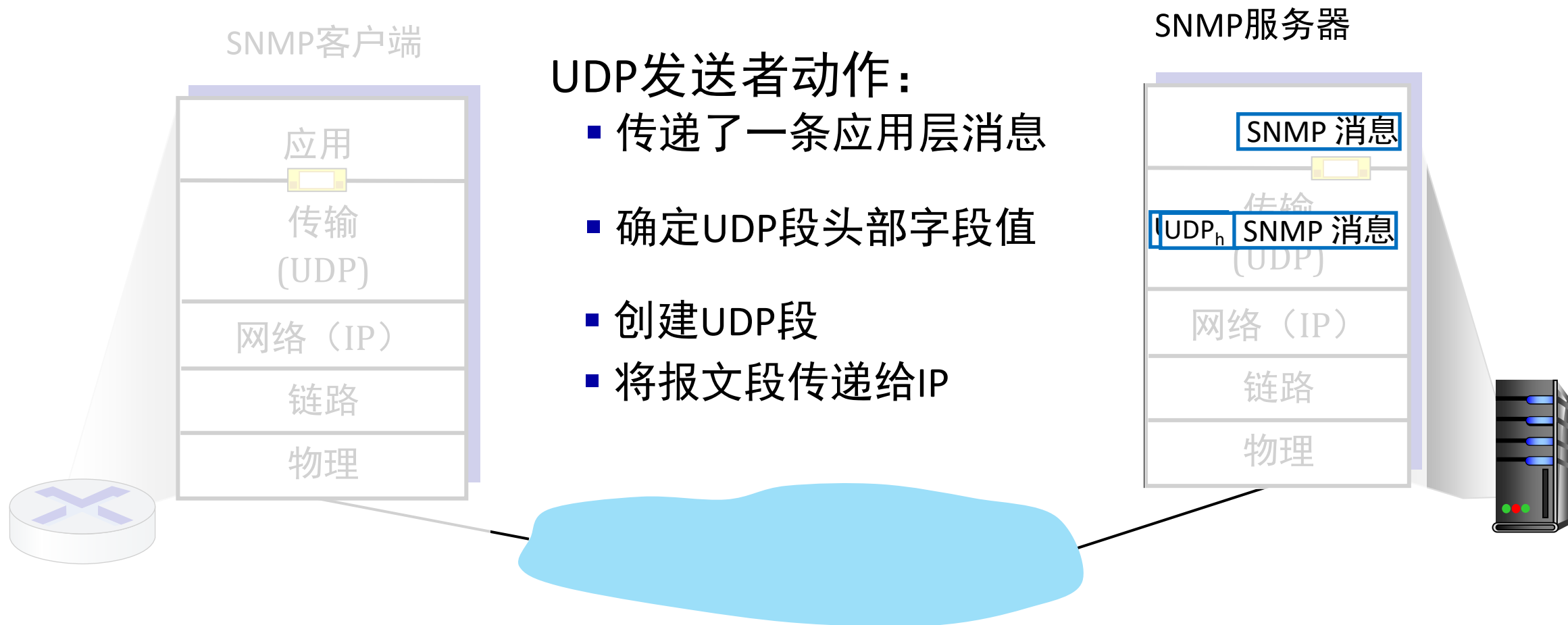
Format



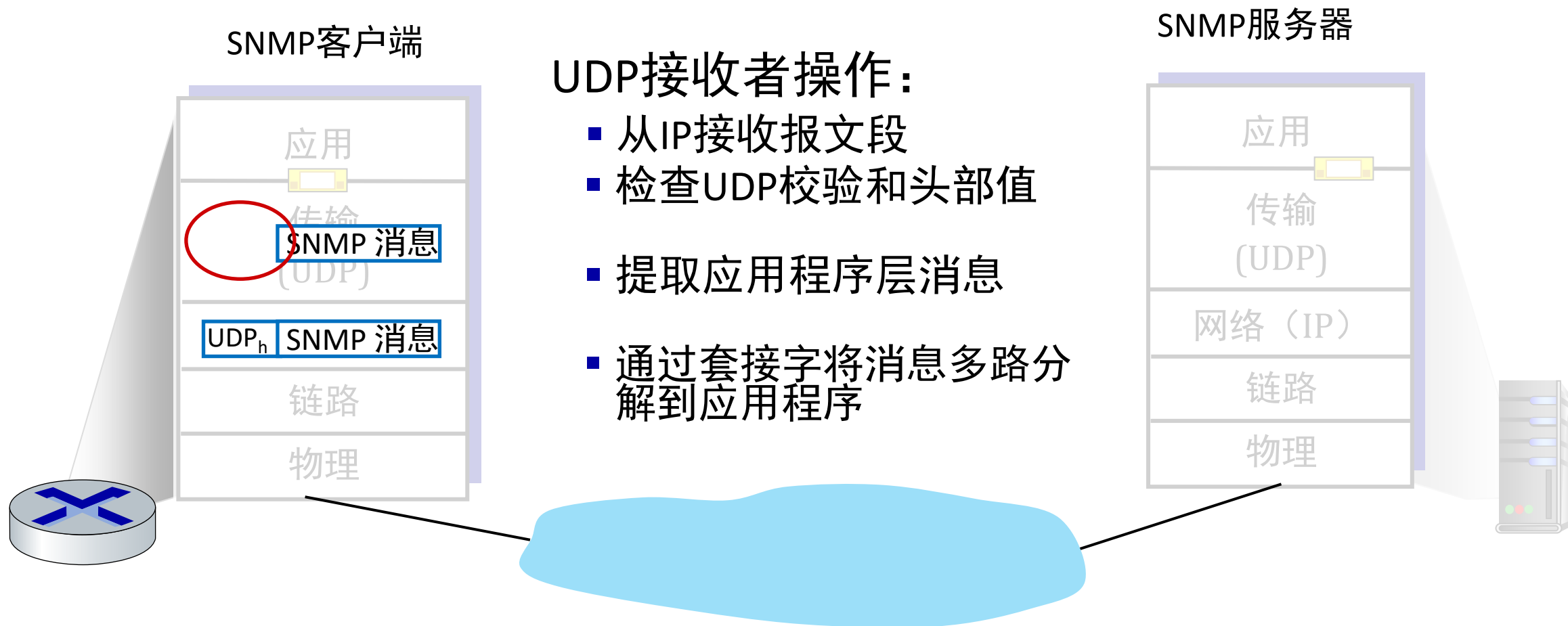
UDP: 传输层动作



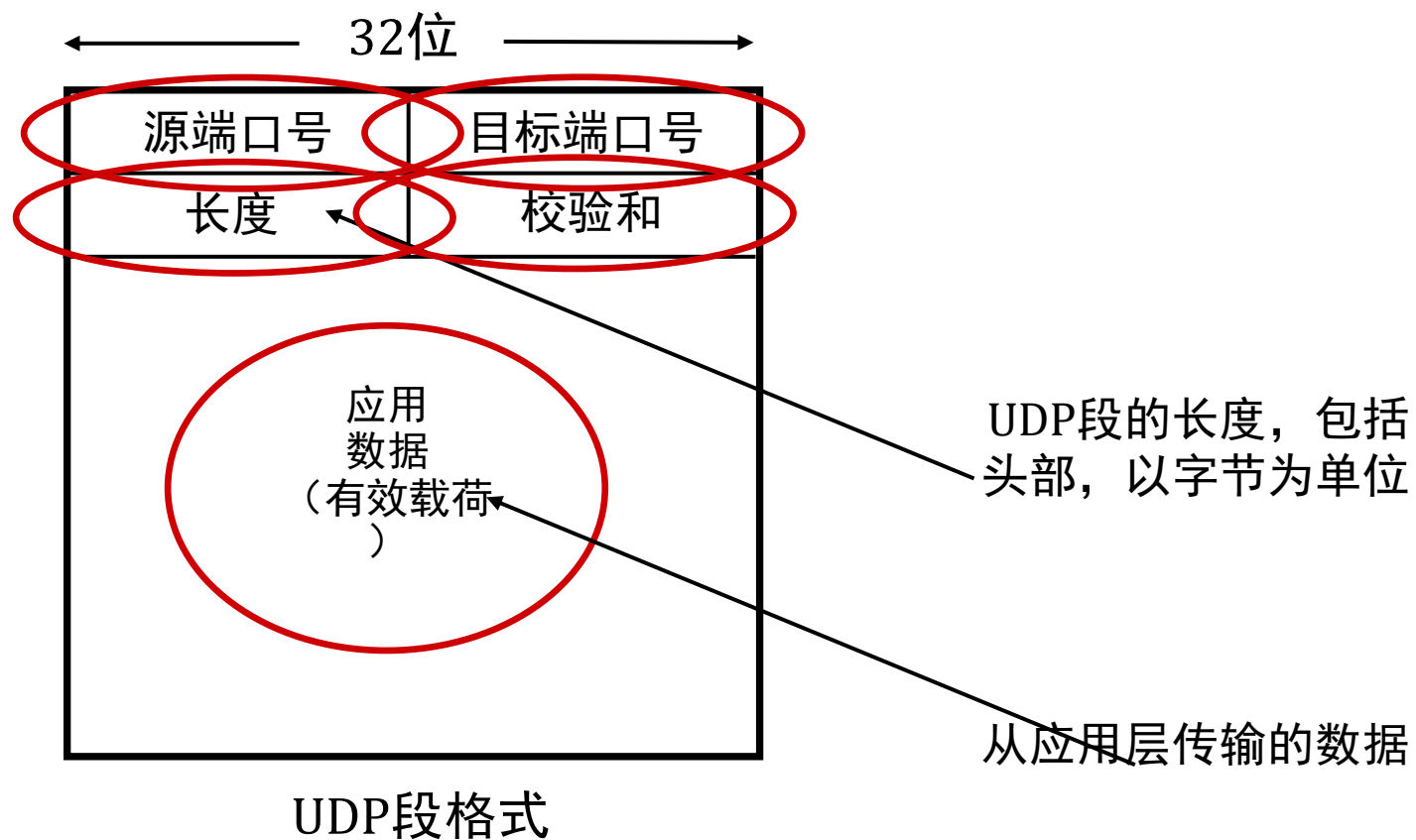
UDP：传输层动作



UDP: 传输层动作

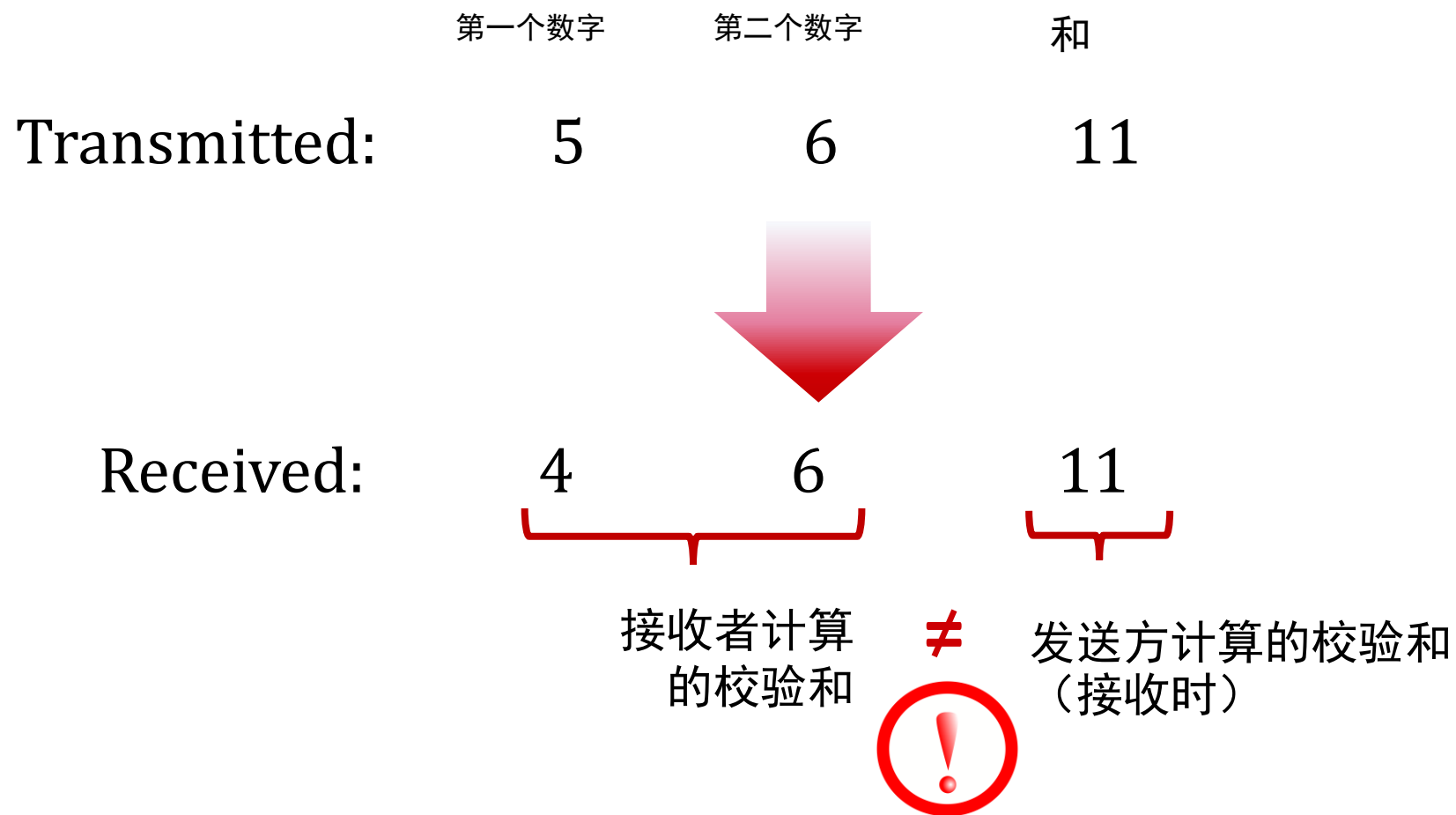


UDP段标头



UDP校验和

目标： 检测传输段中的错误（即翻转的位）



UDP校验和

目标： 检测传输段中的错误（即翻转的位）

发送者：

- 将UDP段的内容（包括UDP头部字段和IP地址）作为16位整数的序列
- **校验和**：段内容的加和（反码求和）
- 将校验和值放入UDP校验和字段中

接收者：

- 计算接收段的校验和
- 检查计算的校验和是否等于校验和字段值：
 - 不相等 - 检测到错误
 - 相等 - 未检测到错误。但是仍可能存在错误吗？

互联网校验和示例

示例：加和两个16位整数

	1	1	1	0	0	1	1	0	0	1	1	0	0	1	1	0
	1	1	0	1	0	1	0	1	0	1	0	1	0	1	0	1
回绕	1	1	0	1	1	1	0	1	1	1	0	1	1	1	0	1
和	1	0	1	1	1	0	1	1	1	0	1	1	1	1	0	0
校验和	0	1	0	0	0	1	0	0	0	1	0	0	0	0	1	1

注意：当数字相加时，需要在结果中加上最高有效位的进位

互联网校验和：弱保护！

示例：添加两个16位整数

1 1 1 0 0 1 1 0 0 1 1 0 0 1 1 0

1 1 0 1 0 1 0 1 0 1 0 1 0 1 0 1

1 0 1 1 1 0 1 1 1 0 1 1 1 0 1 1

1 0 1 1 1 0 1 1 1 0 1 1 1 1 0 0

0 1 0 0 0 1 0 0 0 1 0 0 0 0 1 1

0 1

1 0

即使数字发生了变化（位翻转），

但校验和没有变化！

总结：UDP

- “简化版”协议：
 - 报文段可能会丢失，或者乱序传送
 - 尽力而为的服务：“发送并希望最好的结果”
- UDP优点：
 - 无需建立连接/握手（不会产生往返时间延迟）
 - 当网络服务受到损害时可以运行
 - 提供可靠性（校验和）
- 在应用程序层基于UDP可以构建其他功能（例如，http/3）

第3章： 目录

- 传输层服务
- 多路复用和多路分解
- 无连接传输： UDP
- **可靠数据传输原则**
- 面向连接的传输： TCP
- 拥塞控制原则
- TCP拥塞控制
- 传输层功能的演变



可靠数据传输原则

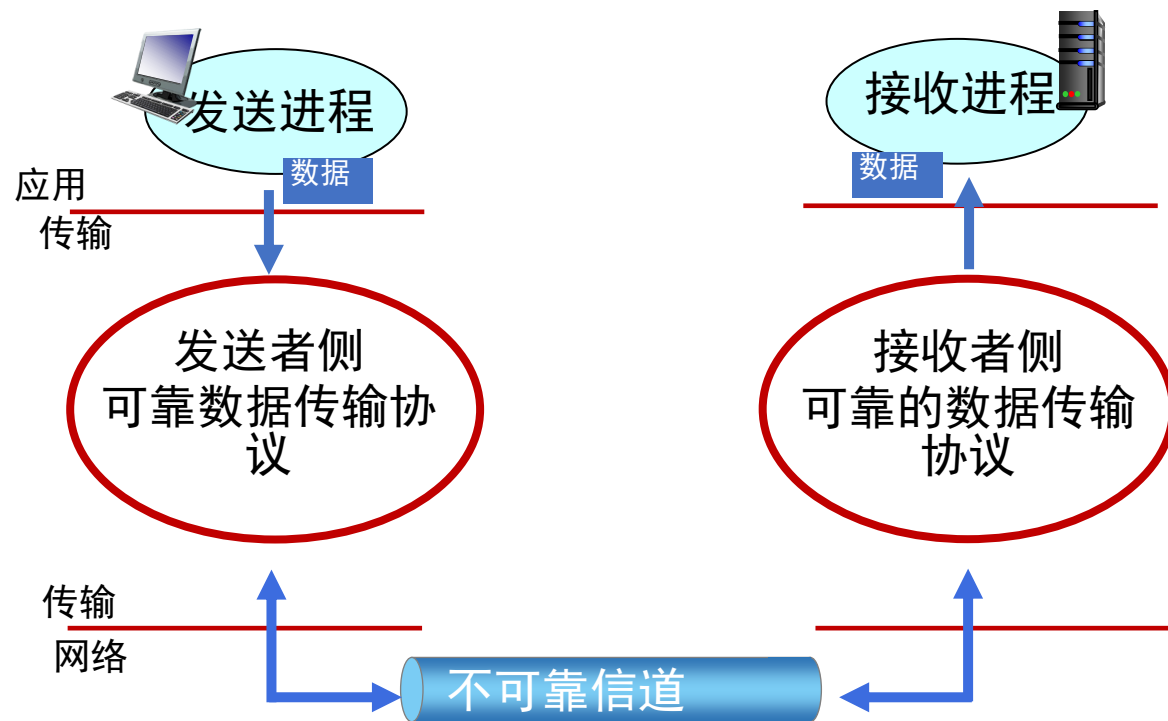
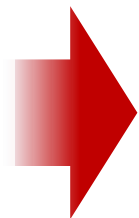


可靠服务的抽象

可靠数据传输原则



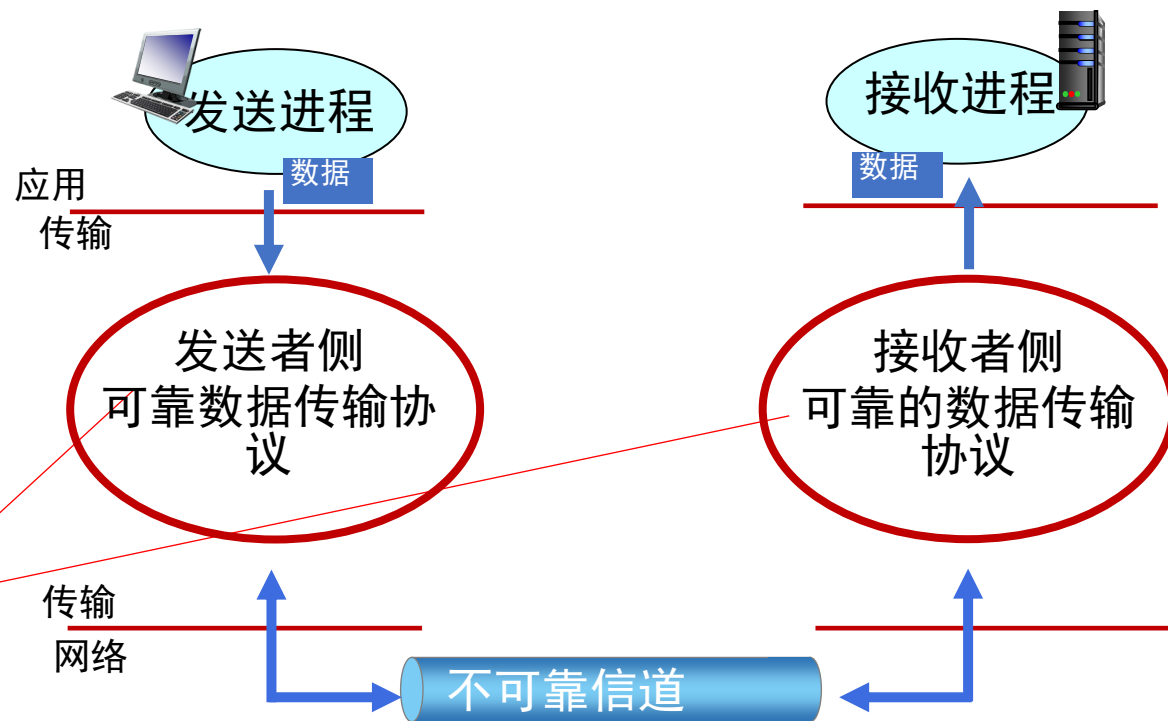
可靠服务的抽象



可靠服务的实现

可靠数据传输原则

可靠数据传输协议的复杂性将（强烈）取决于不可靠信道的特征（丢失，损坏，重新排序数据？）

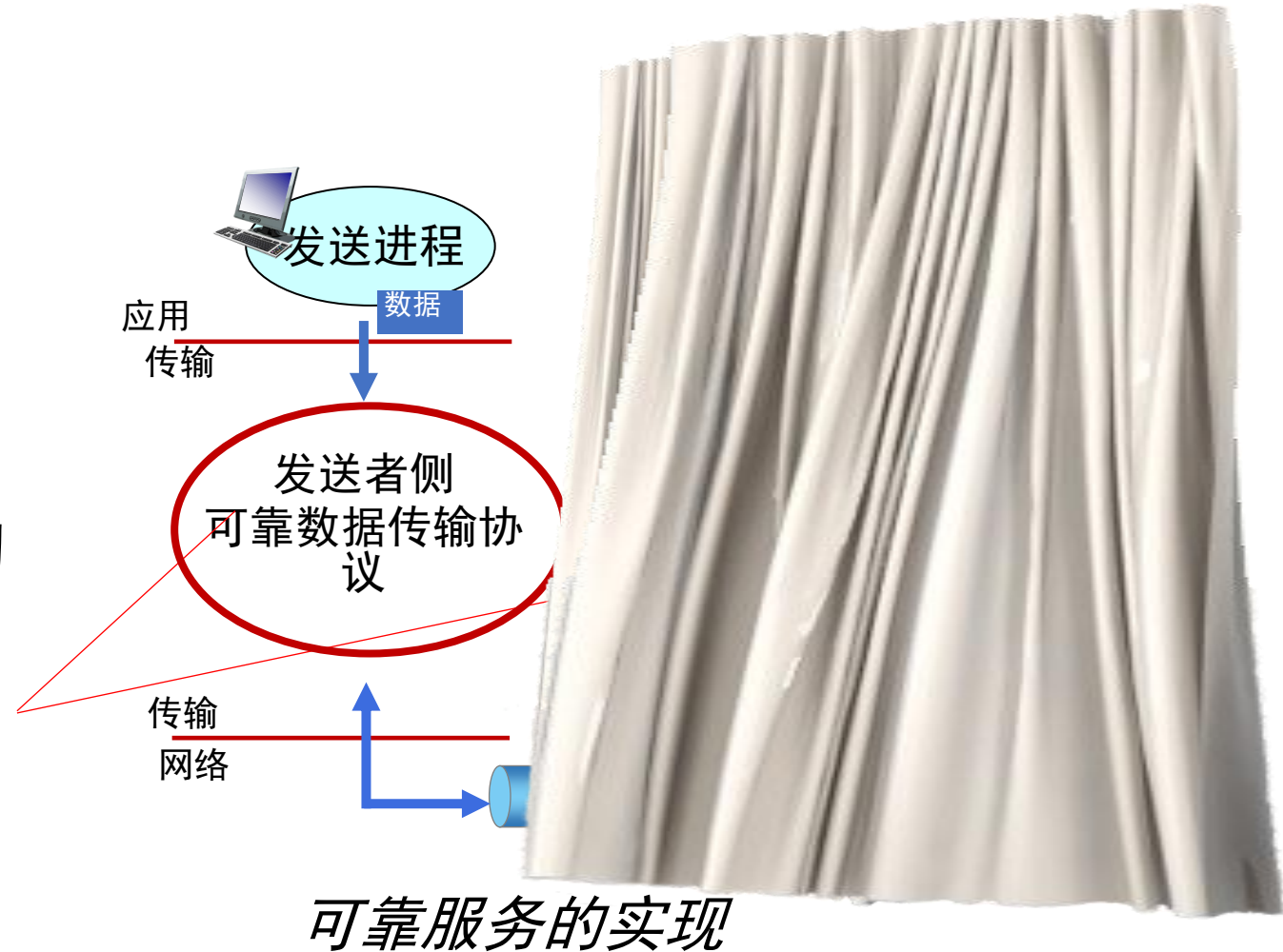


可靠服务的实现

可靠数据传输原则

发送方和接收方不知道彼此的“状态”，例如，消息是否已被接收？

- 除非通过消息进行通信。

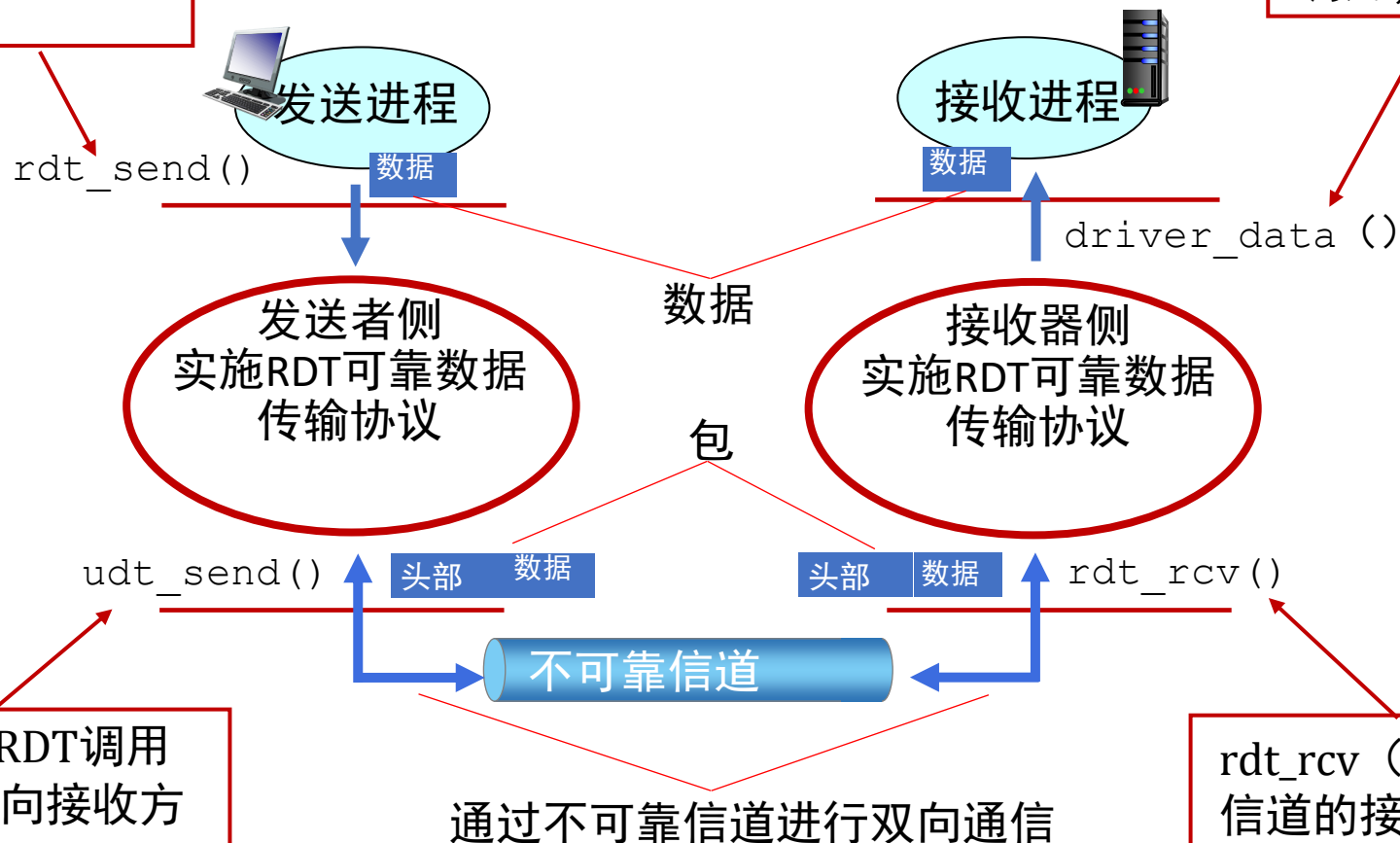


可靠的数据传输协议 (RDT) : 接口

data transfer protocol

rdt_send () : 从上方调用
(例如, App.)。将数据
传送给上层接受方

deliver_data () : 由 RDT
调用, 将数据投递到上层



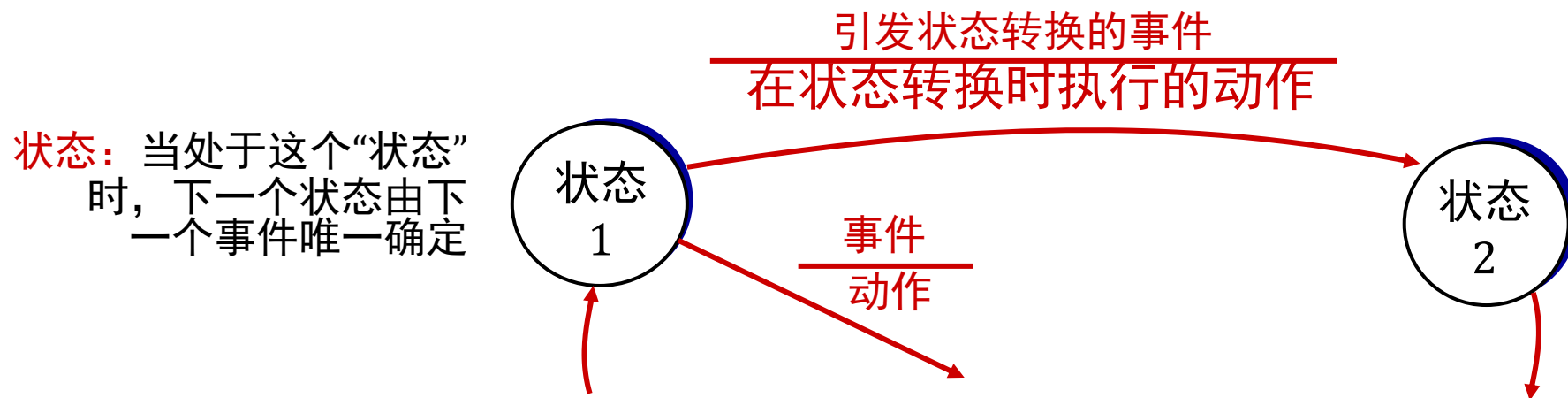
udt_send () : 由RDT调用
以通过不可靠信道向接收方
传输数据包

rdt_rcv () : 当数据包到达
信道的接收端时调用

可靠数据传输：入门

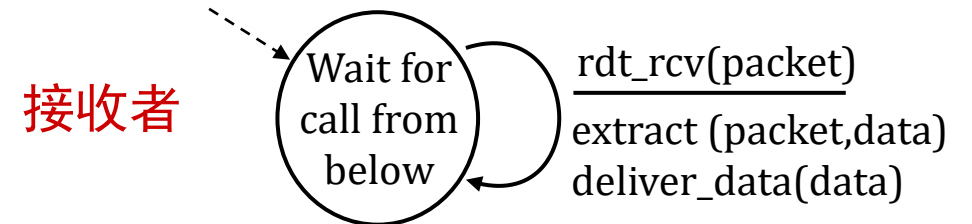
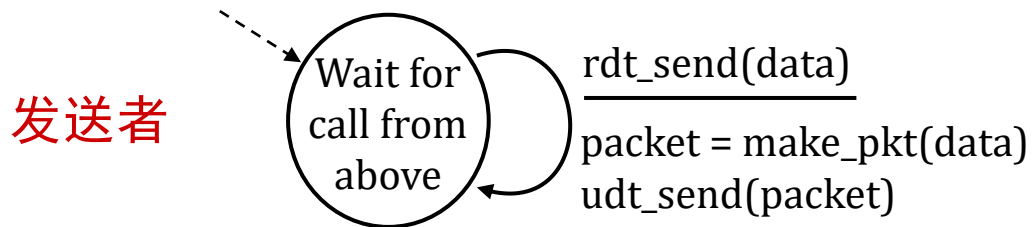
我们将：

- 逐步展开可靠数据传输协议（RDT）的发送者和接收者端
- 仅考虑单向数据传输
 - 但是控制信息将在两个方向上流动！
- 使用有限状态机（FSM， finite state machines）指定发送者，接收者



RDT1.0: 通过可靠信道进行可靠传输

- 底层信道完全可靠
 - 没有比特错误
 - 没有丢包
- 发送方和接收方各自独立的FSM:
 - 发送者将数据发送到底层信道
 - 接收者从底层信道读取数据



rdt2.0: 有位错误的信道

- 底层信道在传输数据包中可能会产生位翻转
 - 校验和可以检测位错误
- *问题*: 如何从错误中恢复?

人类如何从对话中的“错误”中恢复过来?

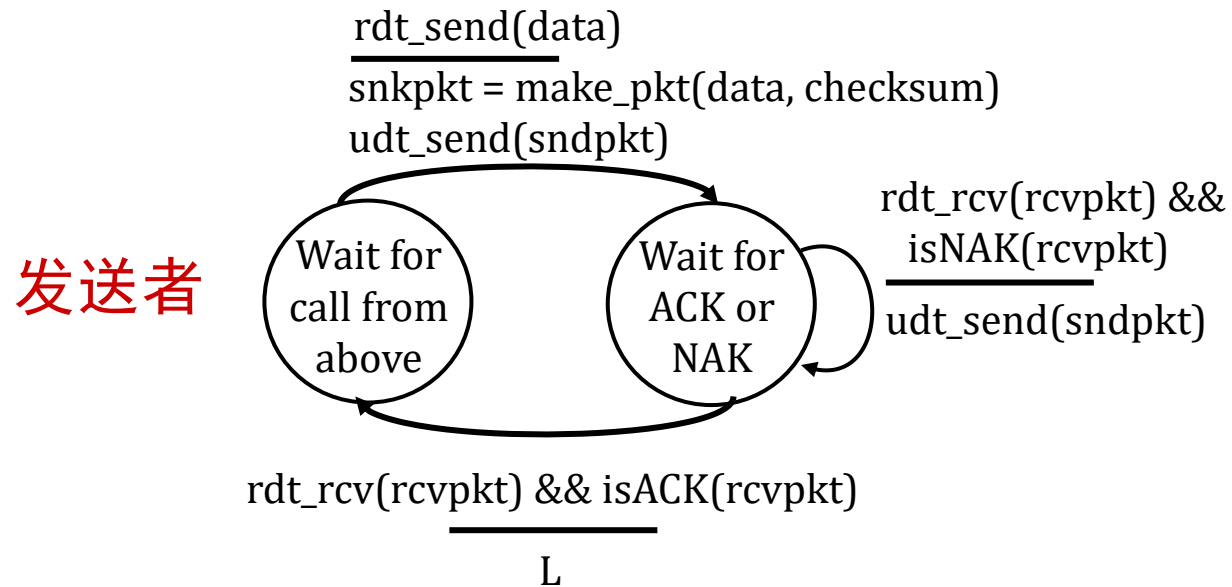
rdt2.0: 有位错误的信道

- 底层信道在传输数据包中可能会产生位翻转
 - 校验和可以检测位错误
- *问题*: 如何从错误中恢复?
 - **确认 (ACKS)**: 接收者明确告诉发送者, 收到的包是正常的
 - **反面确认 (NAKS)**: 接收者明确地告诉发送者, 数据包有错误
 - 发送者在收到NAK时重新传输数据包

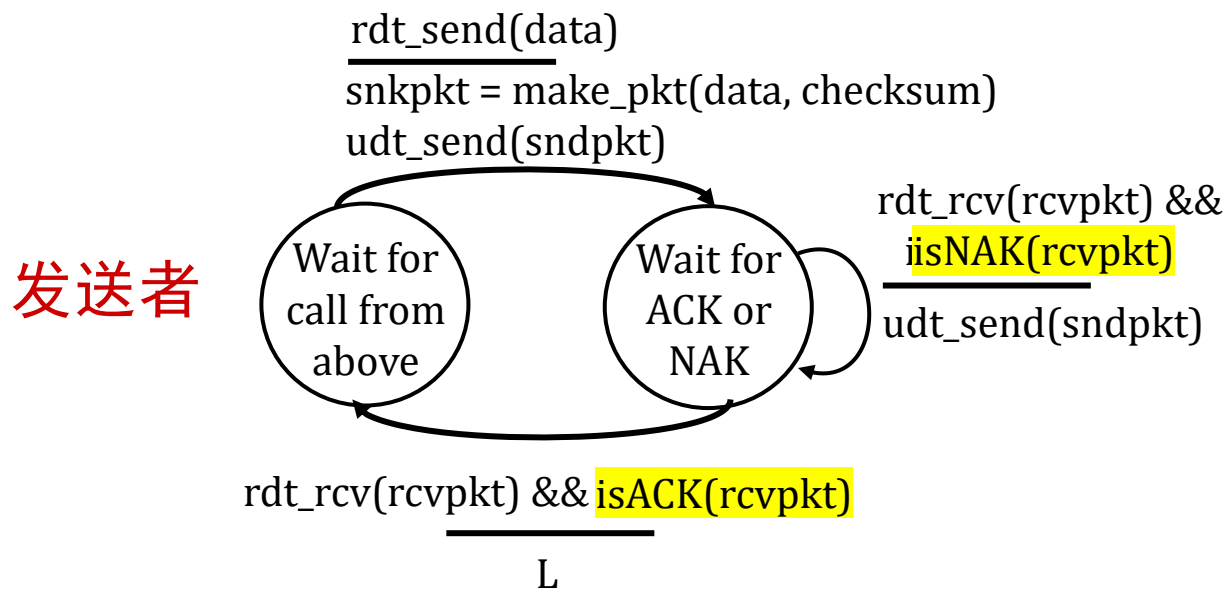
停止-等待

发送者发送一个数据包, 然后等待接收者响应

RDT2.0: FSM规格说明



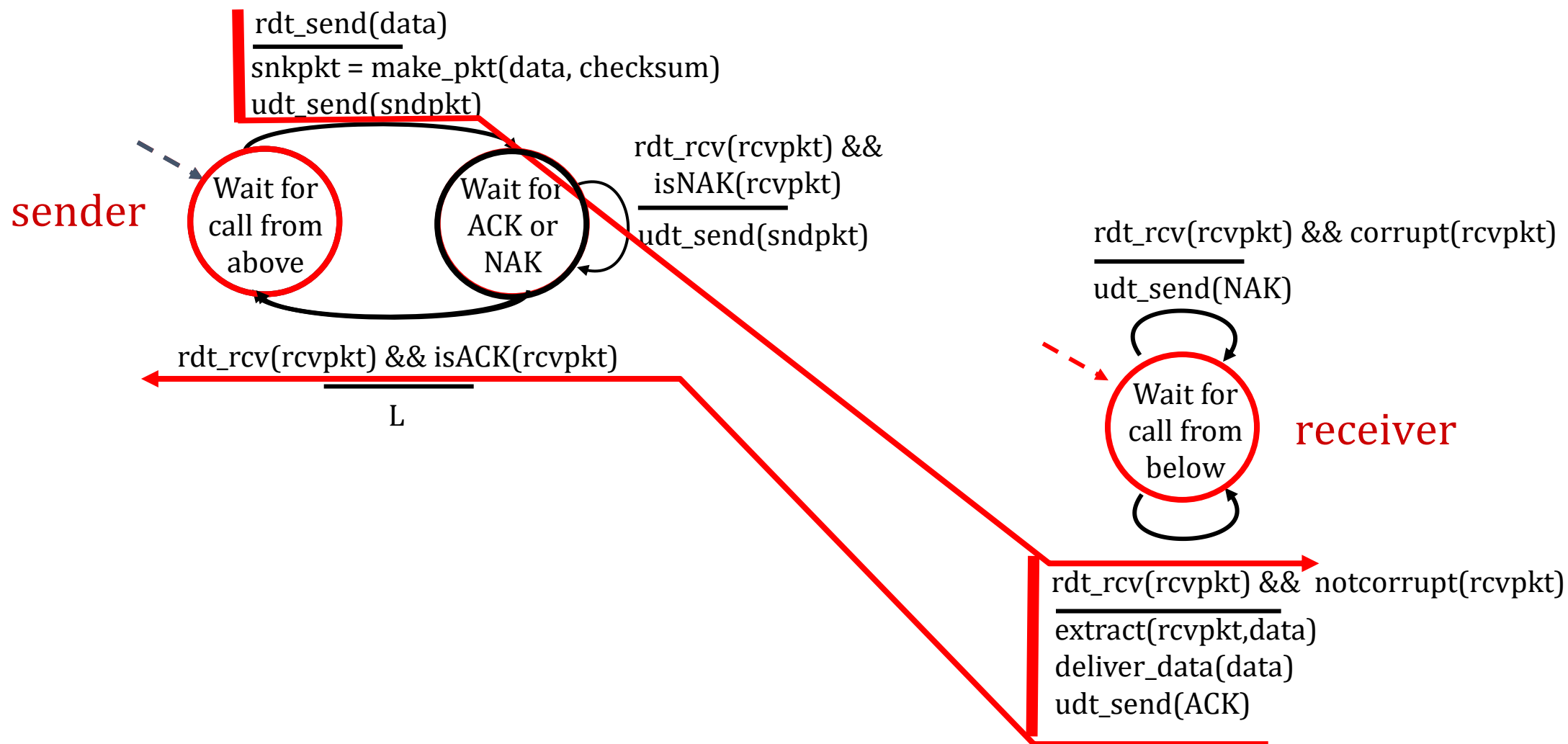
RDT2.0: FSM规格



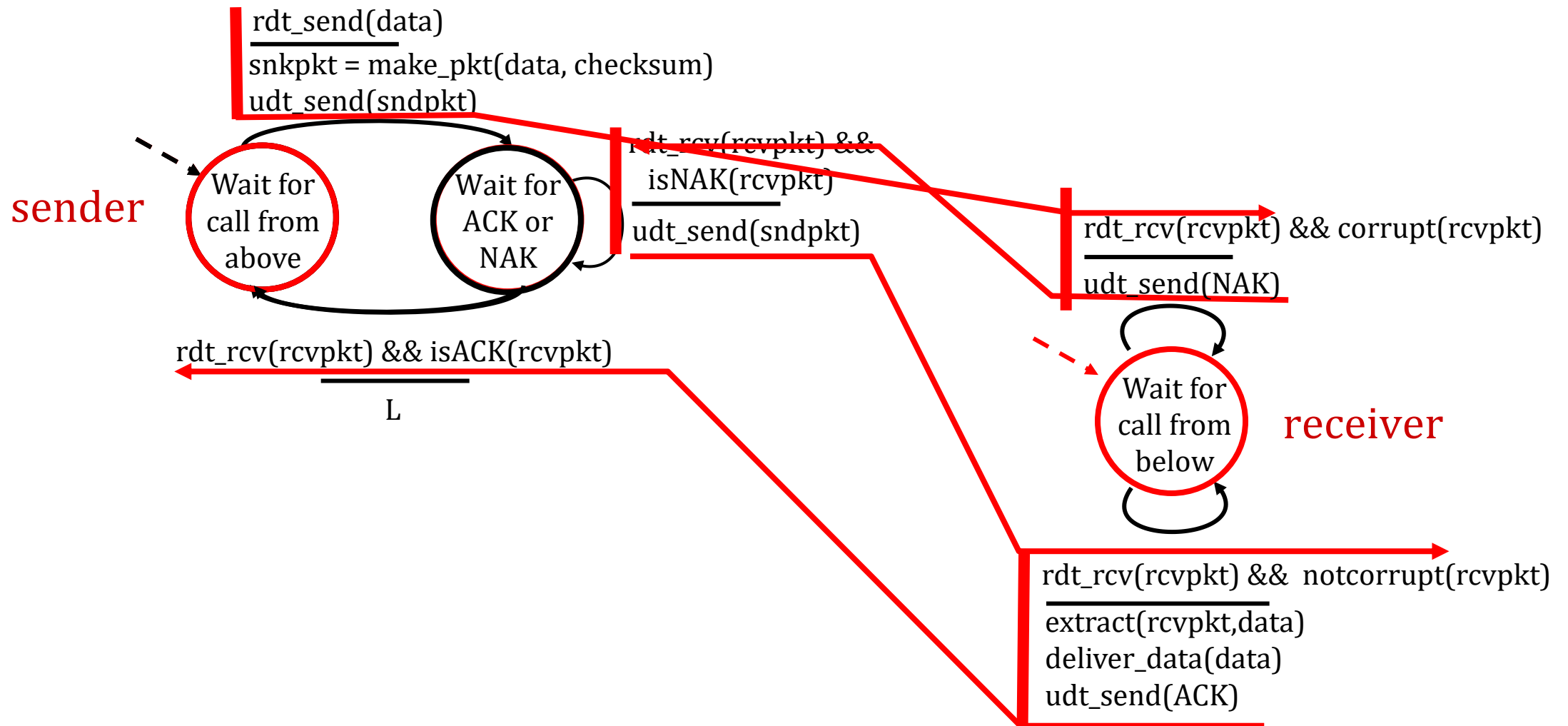
注意：接收者的“状态”（接收者是否正确地收到了我的信息？）对于发送者来说是不知道的，除非以某种方式从接收者传递到发送者

- 这就是为什么我们需要协议！

RDT2.0: 无错误操作



RDT2.0: 数据包损坏场景



RDT2.0有致命的缺陷！

如果ACK/NAK损坏怎么办？

- 发送方不知道接收方发生了什么！
- 不能仅重传：可能重复

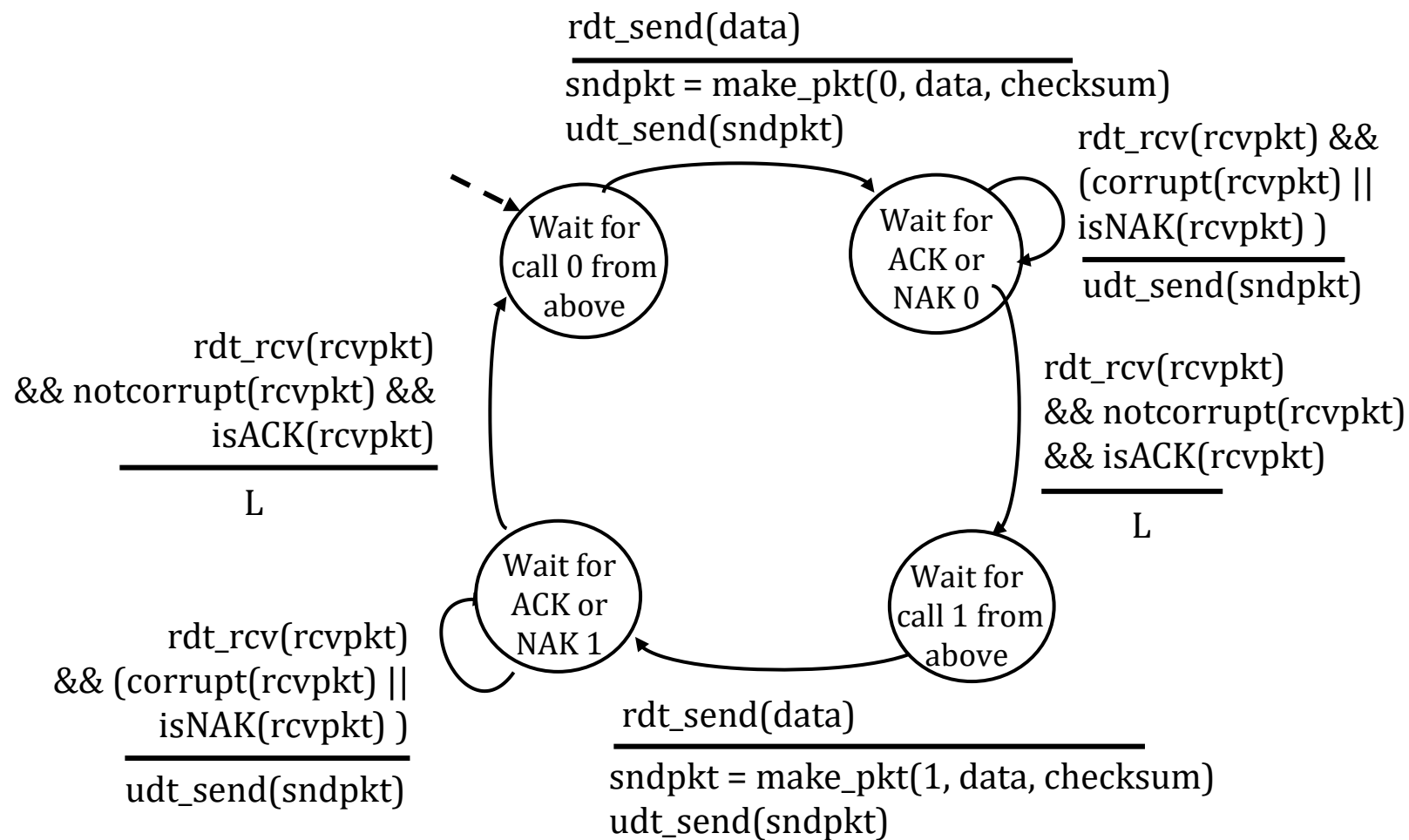
处理重复项：

- 如果ACK/NAK损坏，发送方重新传输当前pkt
- 发送方将序列号添加到每个PKT
- 接收方丢弃（不交付）重复的PKT

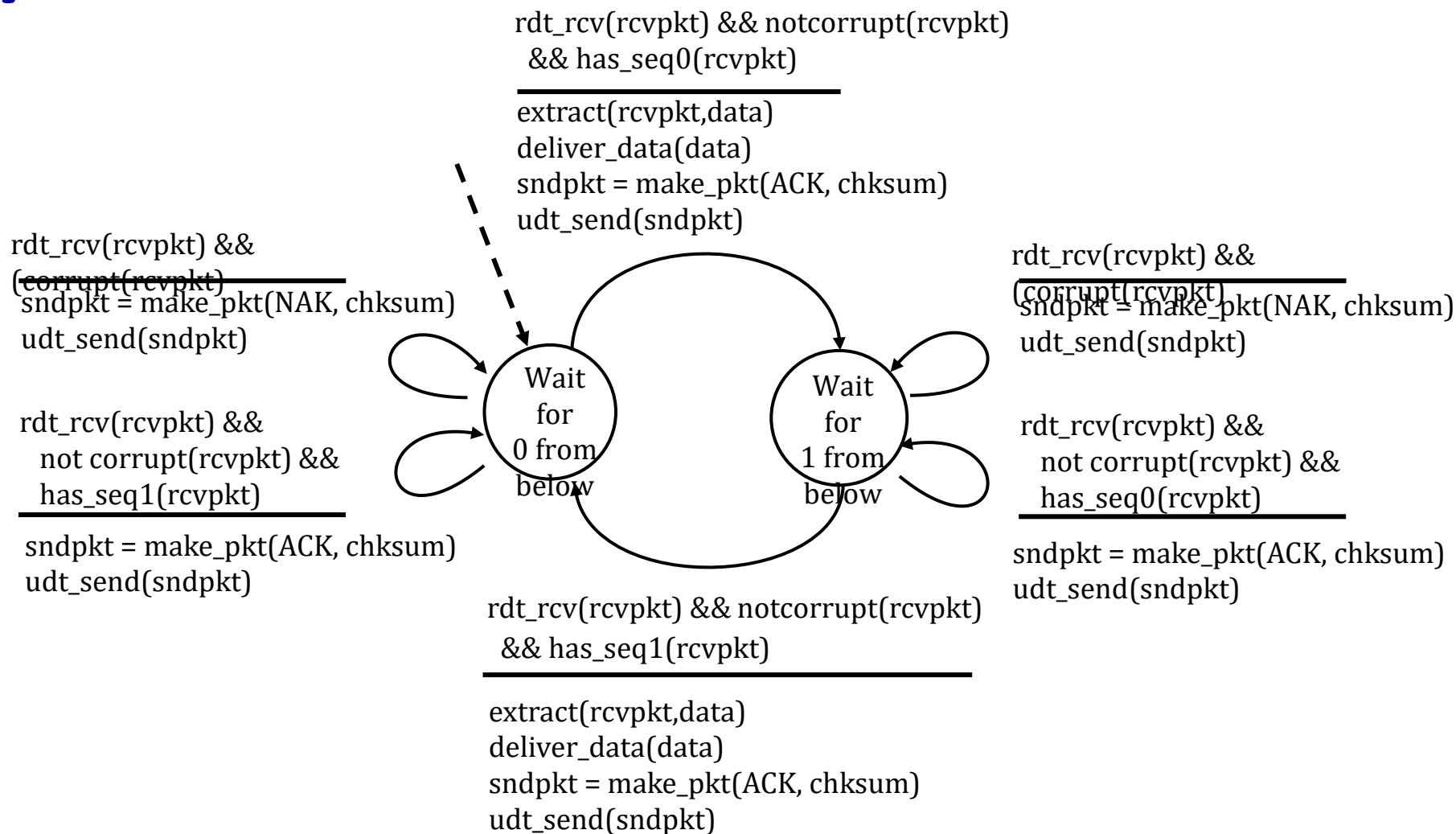
停止-等待

发送者发送一个数据包，然后等待接收者响应

rdt2.1: 发送者, 处理损坏的的ACK/NAKS



rdt2.1: 发送者，处理损坏的的 ACK/NAKS



RDT2.1: 讨论

发送者:

- 在数据包中添加序列号 (seq #)。
- 仅需两个序列号 (0 和 1)。为什么?
- 必须检查接收到的 ACK/NAK 是否损坏。
- 状态数量加倍。
- 状态必须“记住”当前期望的数据包序列号是 0 还是 1。

接收者:

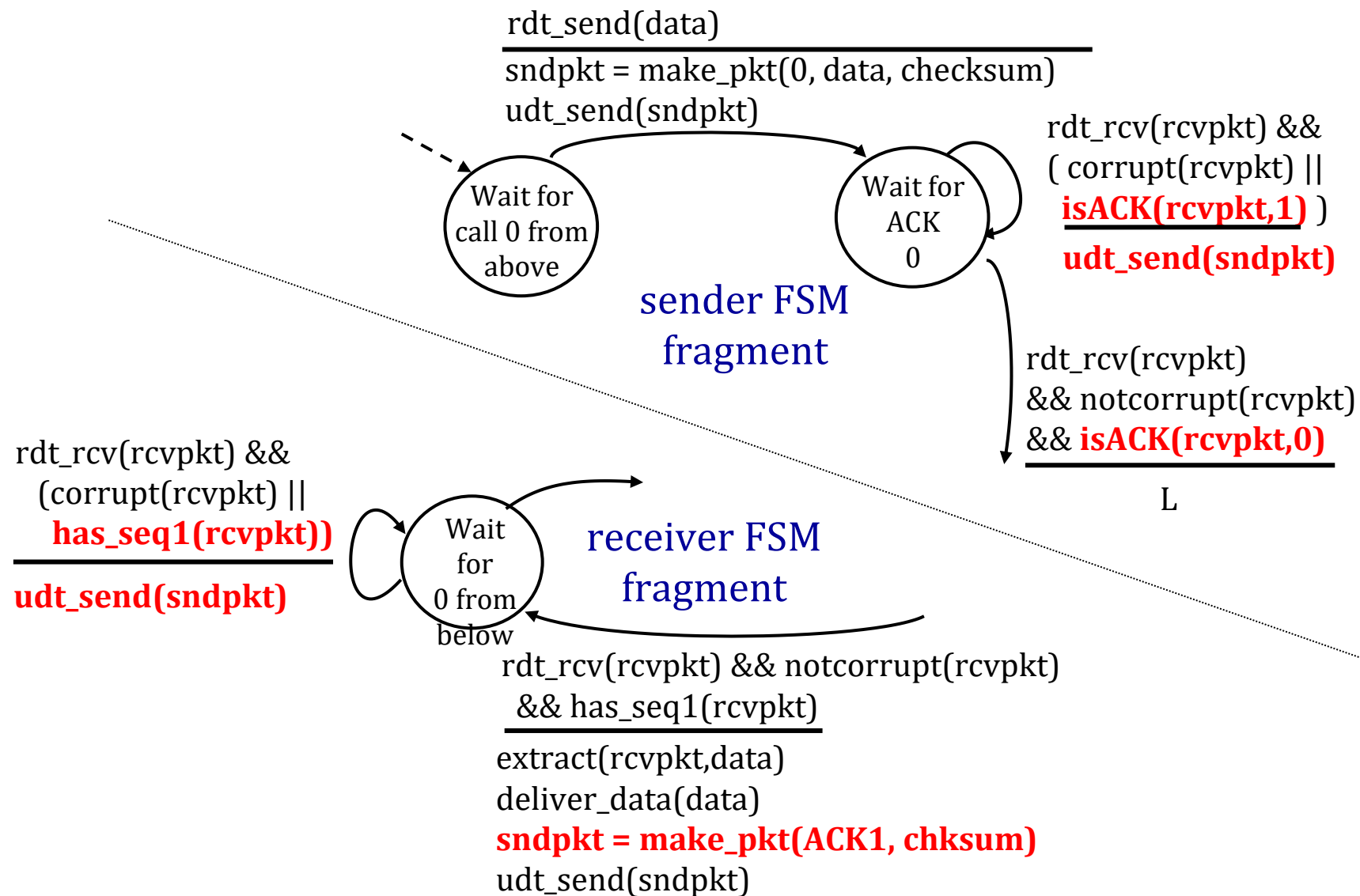
- 必须检查接收到的数据包是否是重复的。
 - 状态指示当前期望的数据包序列号是 0 还是 1。
- 注意: 接收方无法知道其最后接收到的 ACK/NAK 是否在发送方处正确接收。

RDT2.2: 无NAK协议

- 与 rdt2.1 相同的功能，只使用 ACK 而不使用 NAK。
- 接收方对最后一个成功接收的数据包发送 ACK，而不是 NAK。
- 接收方必须明确地包含正在确认的包的序列号。
- 发送方收到重复的 ACK 时，执行与 NAK 相同的操作：重传当前数据包。

正如我们将看到的那样，TCP是使用这种方法实现无NAK的

RDT2.2: 发送者, 接收器分片



RDT3.0：带错误和损失的信道

新信道假设：底层通道也可能丢失数据包（数据，ACKs）

- 校验和、序列号、ACK、重传会有所帮助.....但还不够

问：人类如何处理在对话中丢失的发送者
对接收者的词语？

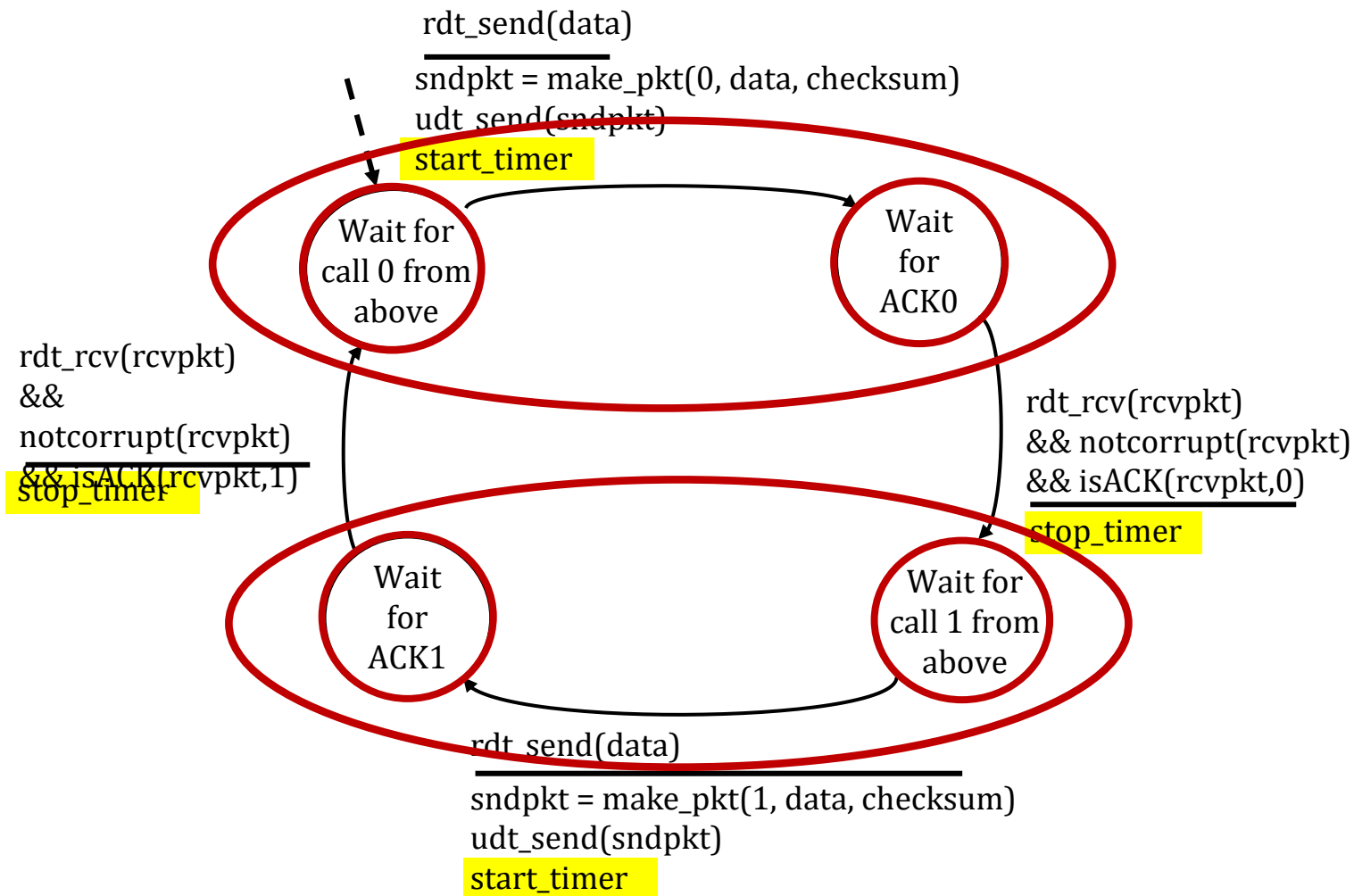
RDT3.0：带错误和损失的频道

方法：发送方等待“合理”的时间来接收 ACK

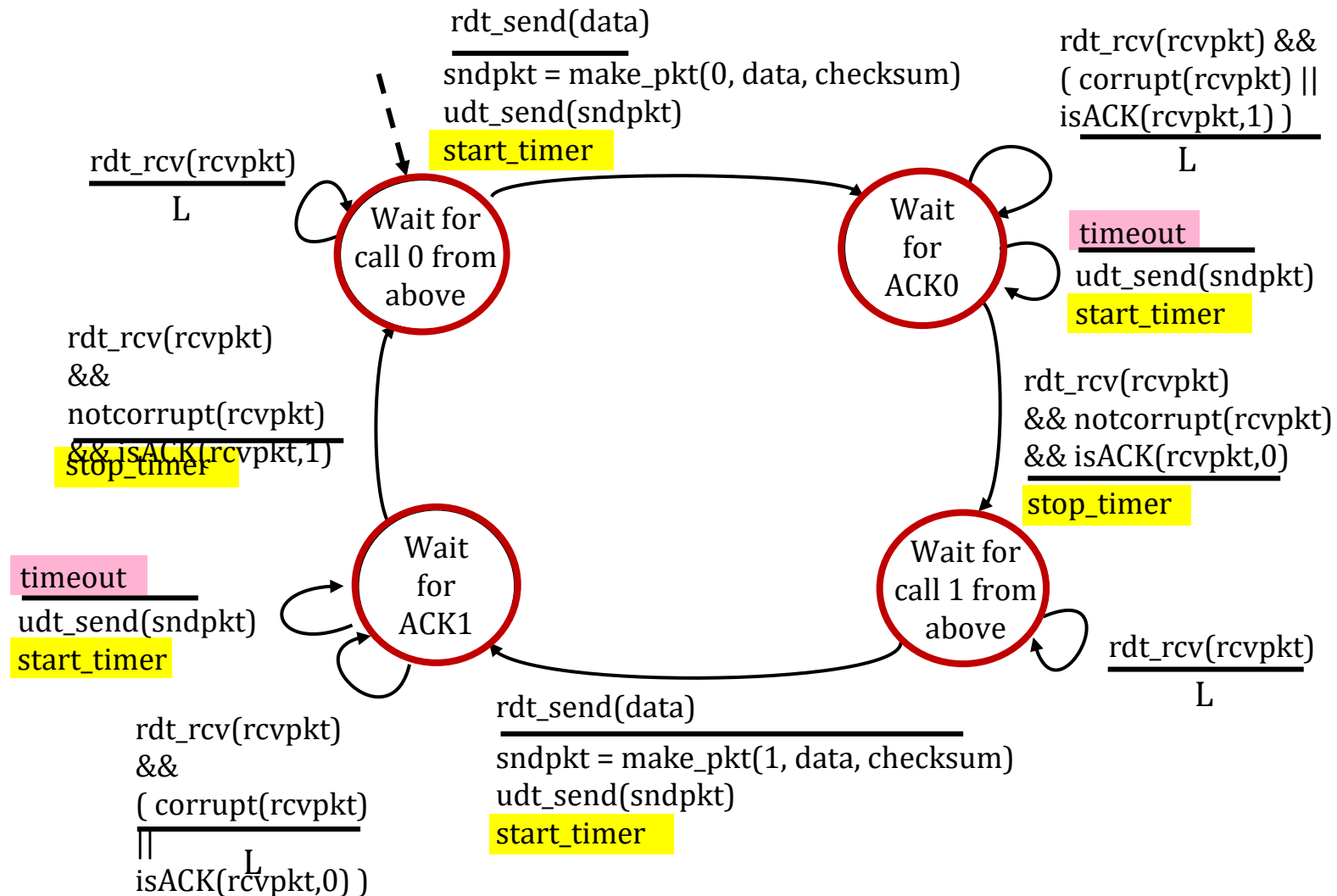
- 如果在此时间内没有收到 ACK，则重新发送数据包。
- 如果数据包（或 ACK）只是延迟（而不是丢失）：
 - 重传会是重复的，但序列号已经处理了这一点！
 - 接收方必须指定被确认的数据包的序列号。
- 在“合理”的时间之后，使用倒计时计时器中断



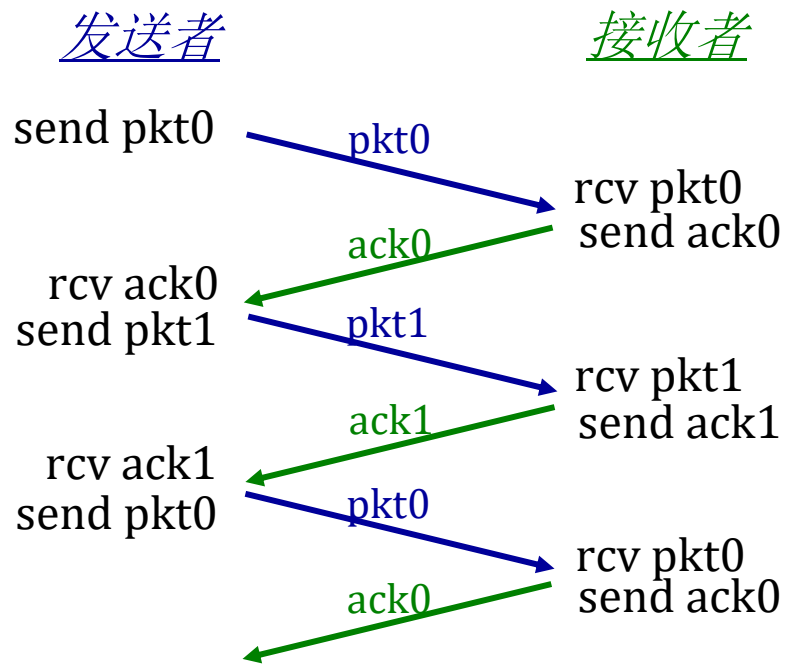
RDT3.0发送者



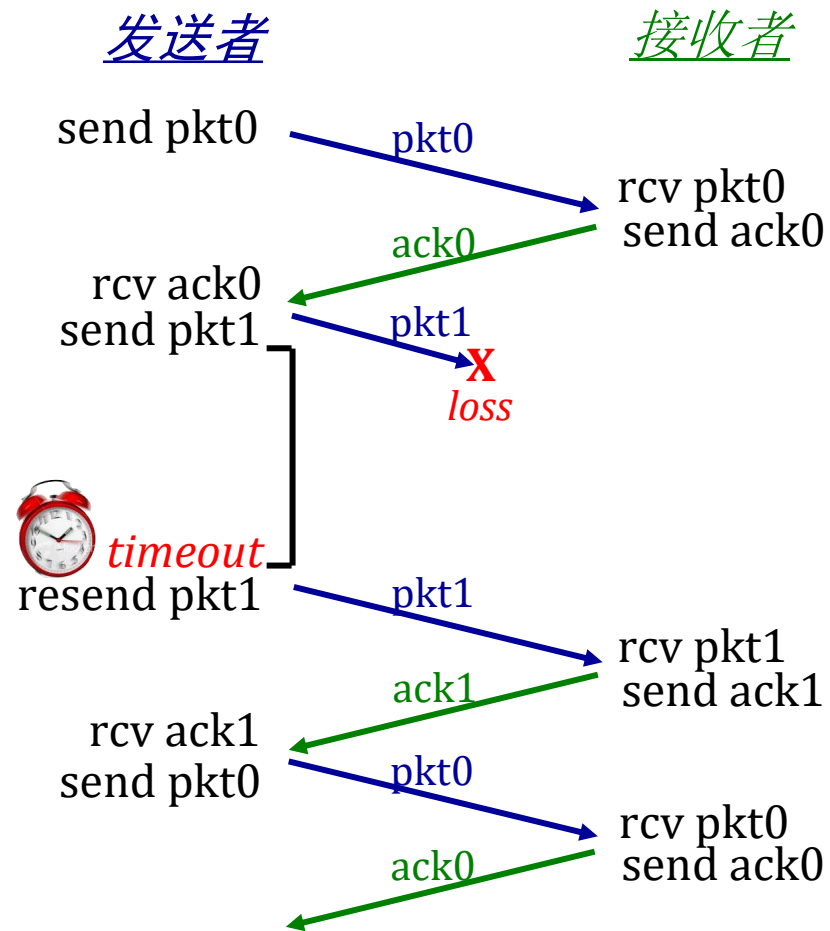
RDT3.0发送者



RDT3.0运作过程

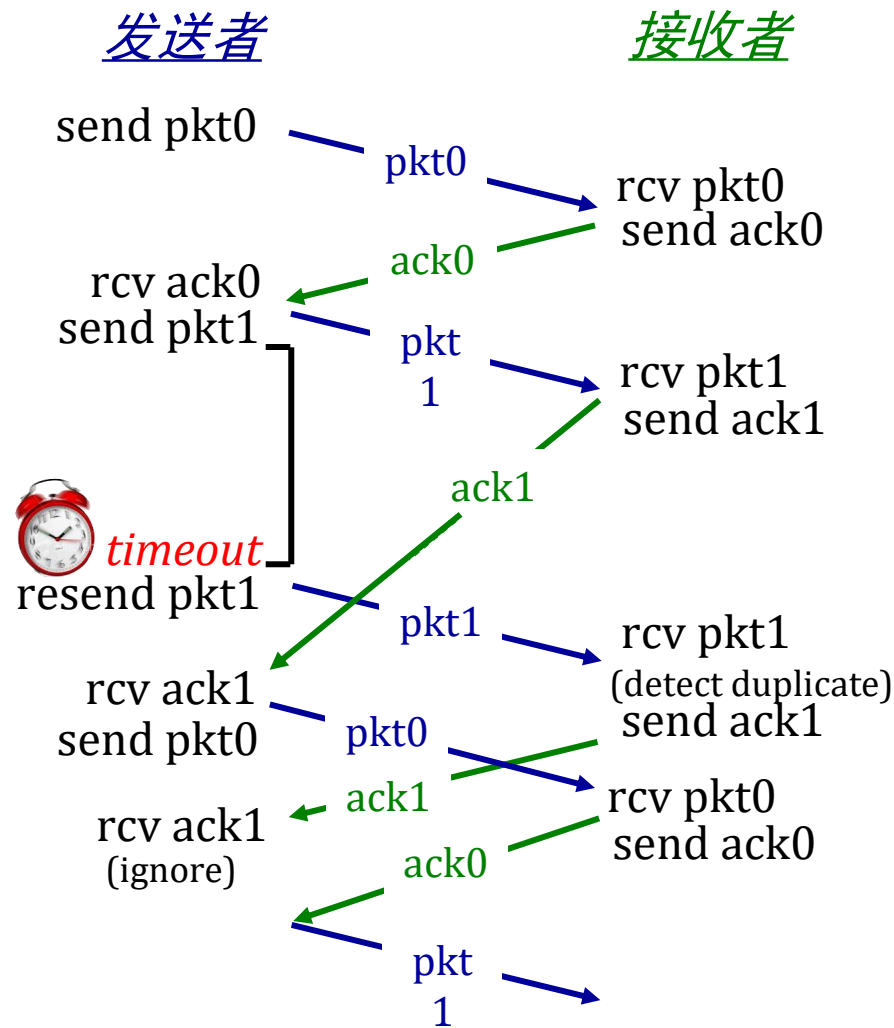
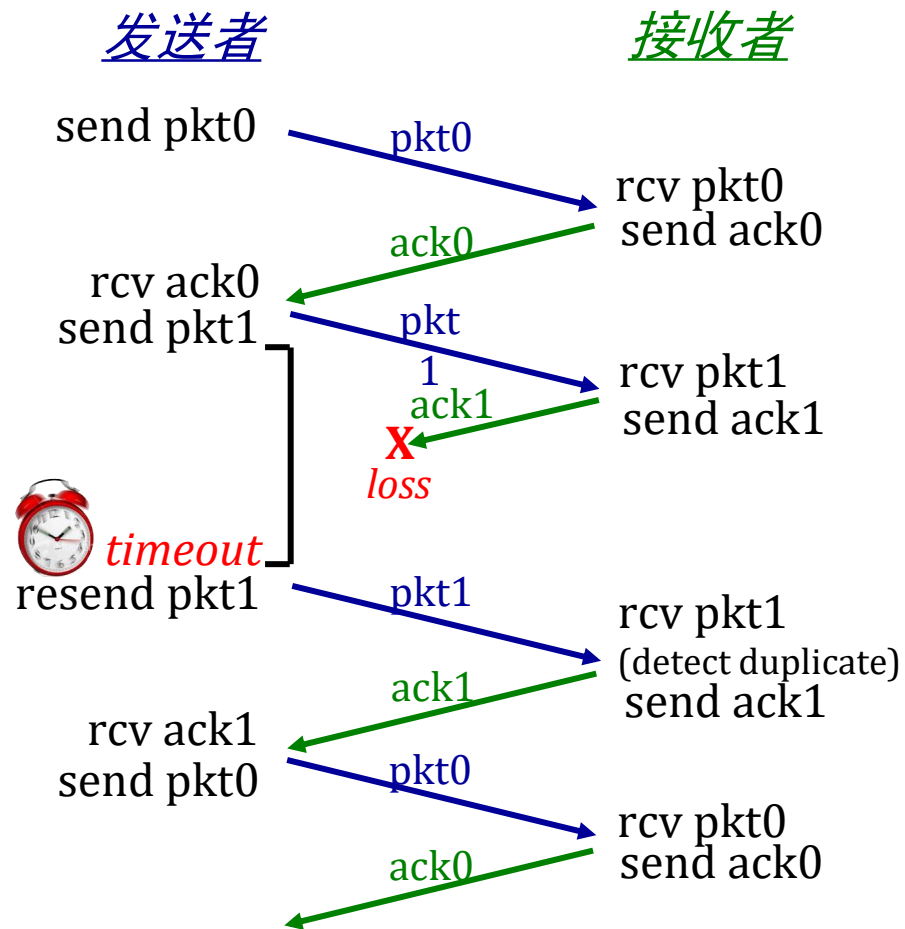


(a) 无损



(b) 丢包

RDT3.0运作过程

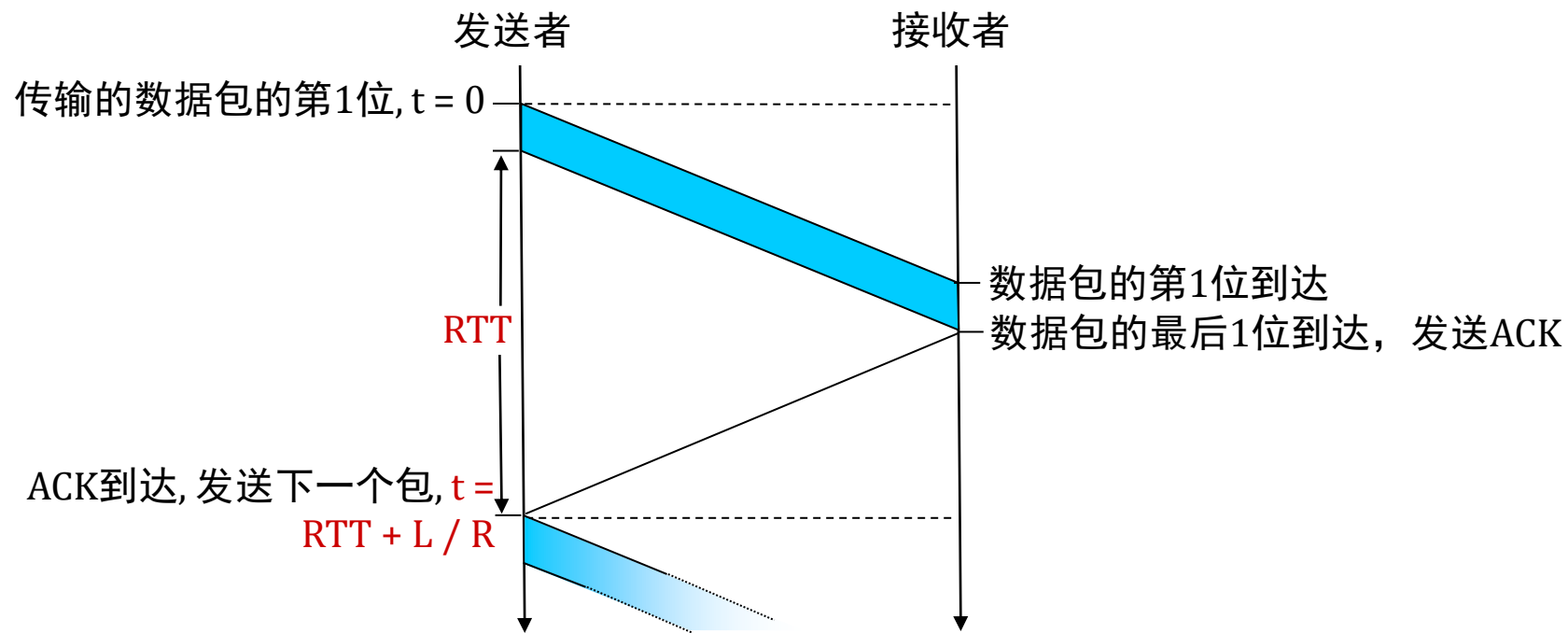


RDT3.0的性能（停止-等待）

- $U_{\text{发送者}}$: 利用率 - 发送方忙于发送的时间的百分比
- 示例: 1 Gbps链路, 约15 ms 延迟, 8000位数据包
 - 将数据包传输到信道耗时:

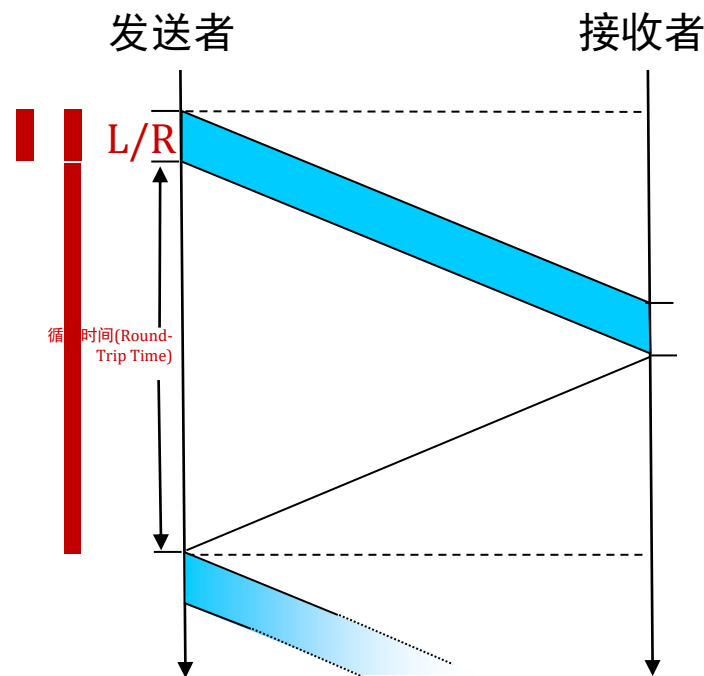
$$D_{\text{传输}} = \frac{L}{R} = \frac{8000 \text{ bits}}{10^9 \text{ bits/sec}} = 8 \text{ ms}$$

RDT3.0: 停止-等待操作



RDT3.0: 停止-等待操作

$$\begin{aligned}U_{\text{发送者}} &= \frac{L / R}{RTT + L / R} \\&= \frac{.008}{30.008} \\&= 0.00027\end{aligned}$$



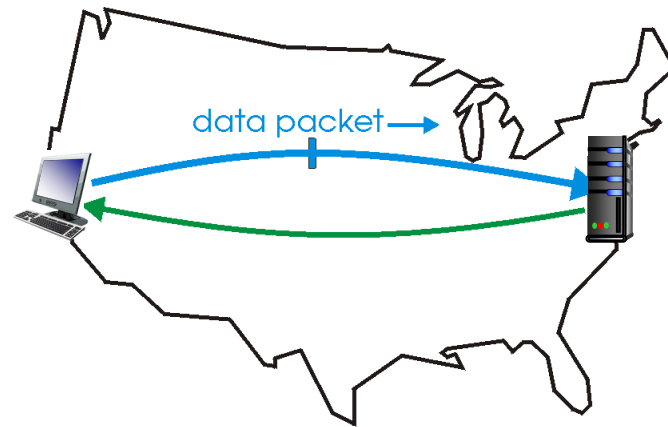
rdt 3.0 协议的性能很差！

- 协议限制了底层基础设施（信道）的性能。

RDT3.0：流水线协议操作

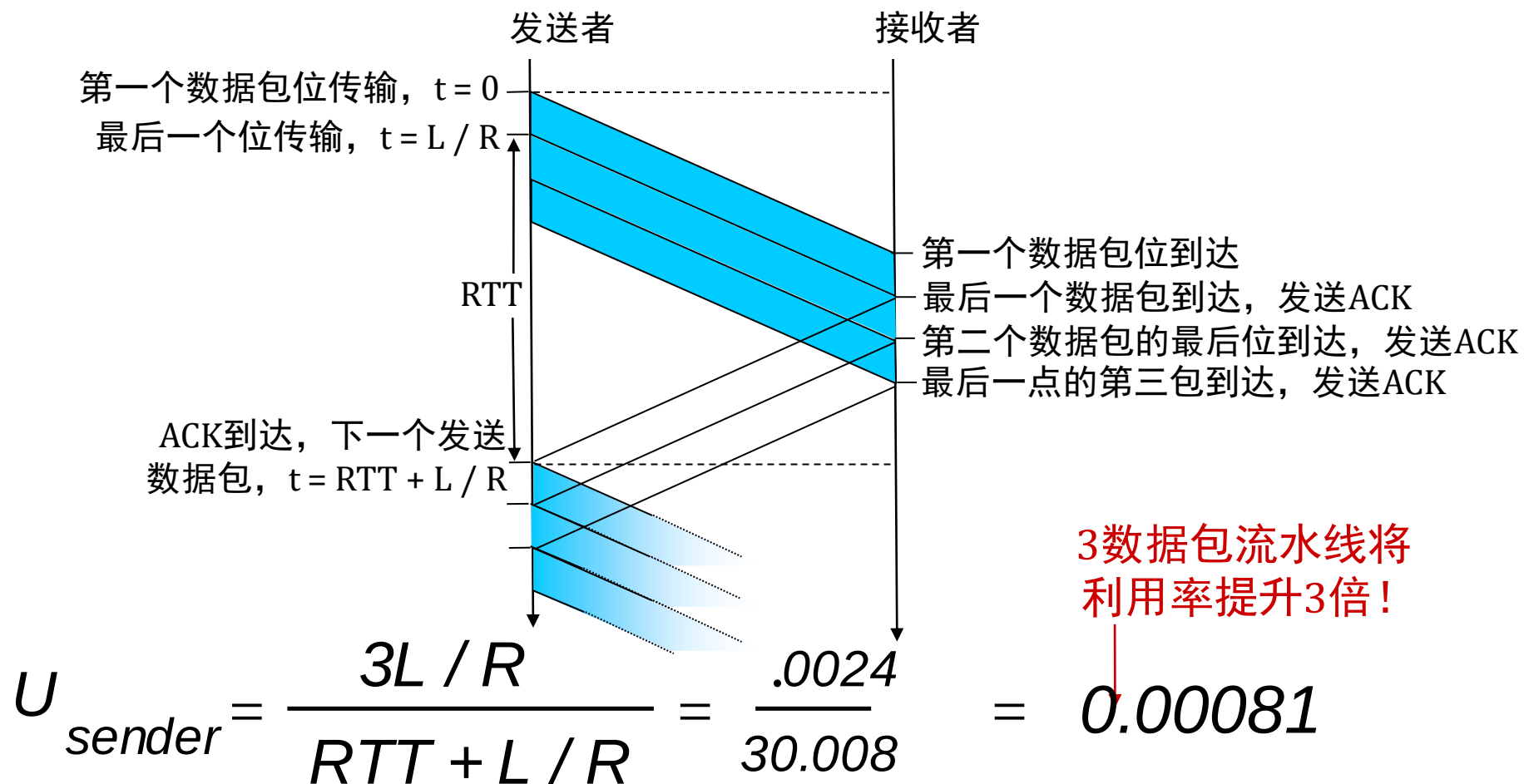
流水线技术：发送方允许多个“在途”的、尚未被确认的数据包。

- 必须增加序列号的范围。
- 在发送方和/或接收方进行缓冲。



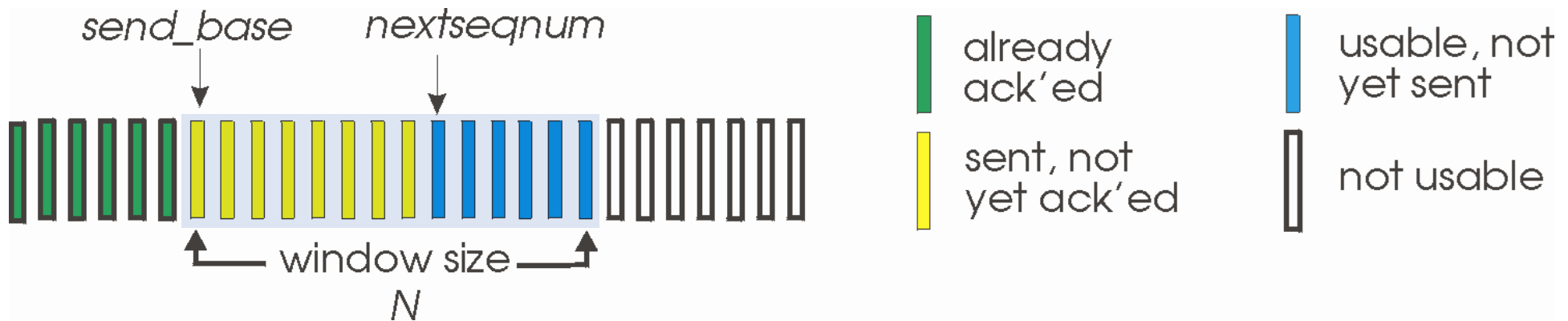
(a) a stop-and-wait protocol in operation

流水线：利用率增加



回退N帧 (GO-BACK-N) : 发送者

- 发送方：最多为 N 的“窗口”，连续传输但未被确认的数据包。
- 数据包头中有 k 位的序列号。

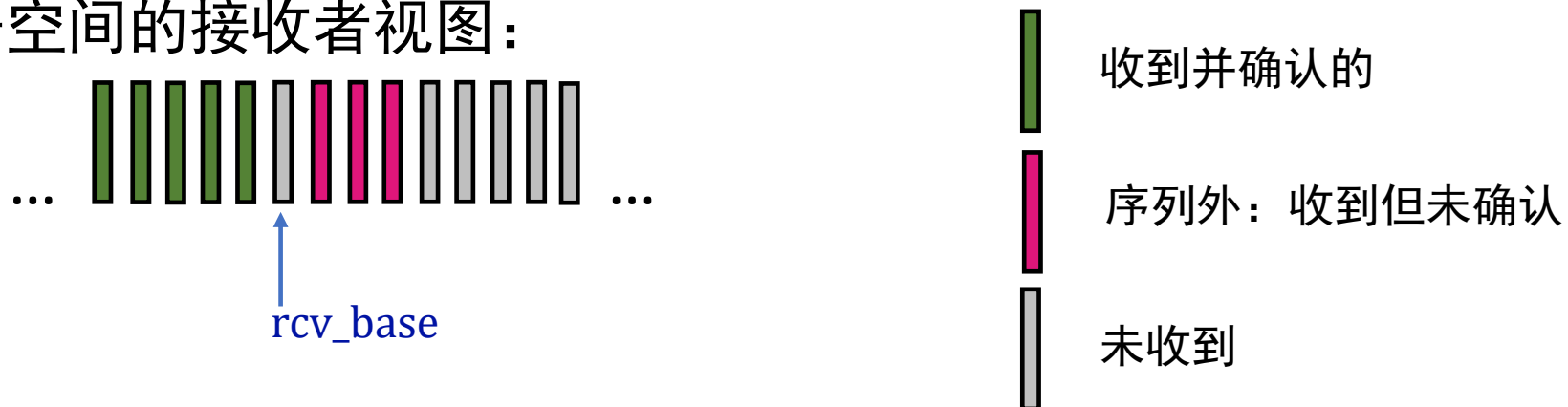


- **累积确认**：ACK(n)：确认所有直到序列号 n 的数据包（包括序列号 n ）。
- 接收到 ACK(n) 后：将窗口向前移动，起始位置为 $n+1$ 。
- 设置最旧在途数据包的计时器。
- 超时(n)：重传数据包 n 及窗口中所有序列号更高的数据包。

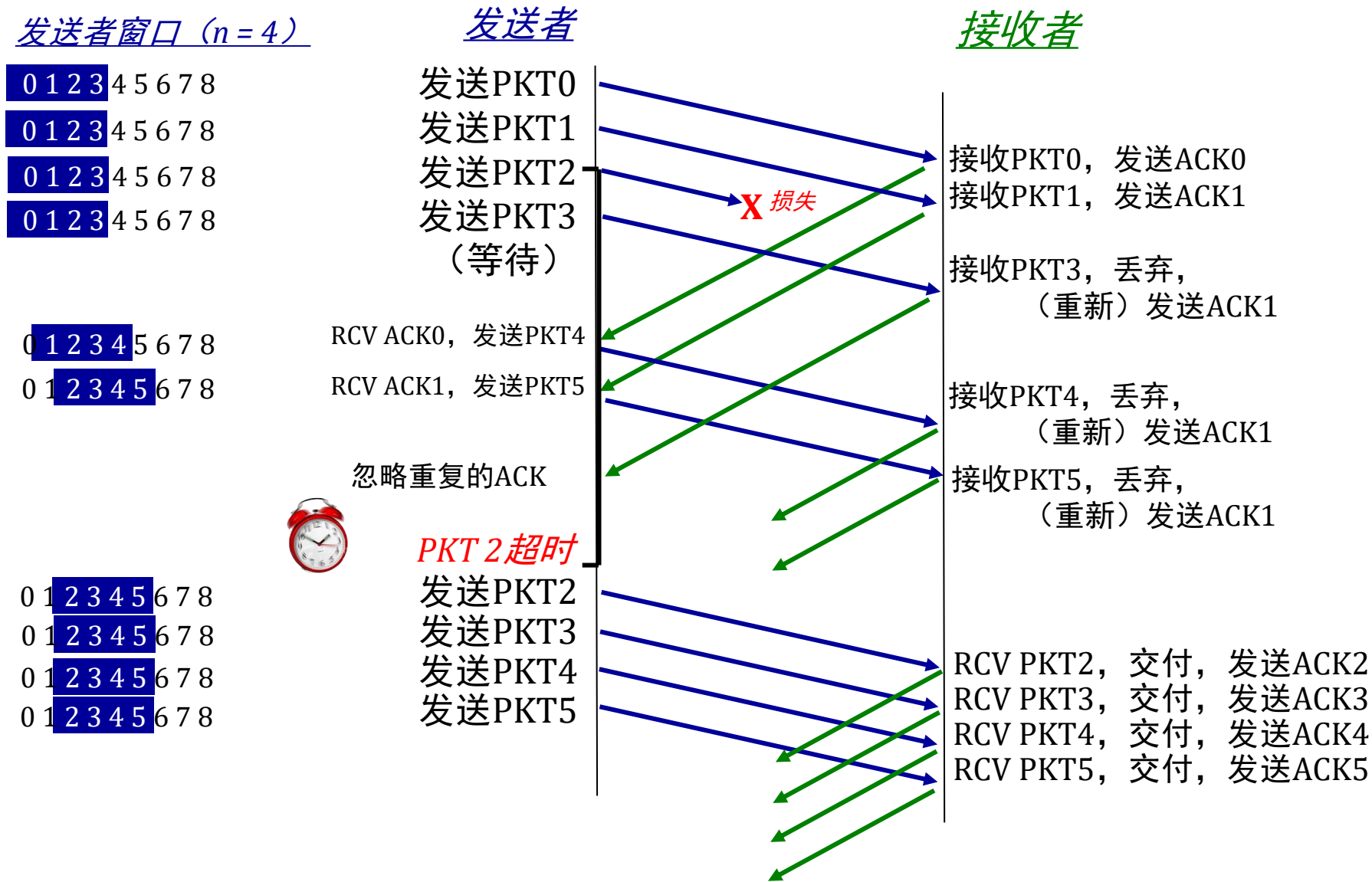
Go-Back-N: 接收者

- 仅ACK: 始终发送已正确接收的数据包的ACK, ACK的序列号为当前最高的按顺序排列的序列号。
 - 可能会生成重复的ACK。
 - 只需要记住接收基准 (rcv_base)。
- 接收到乱序数据包时:
 - 可以丢弃 (不进行缓冲) 或缓冲: 这是一个实现上的决定。
 - 对已按顺序接收的最高序列号数据包重新发送ACK。

序列号空间的接收者视图:



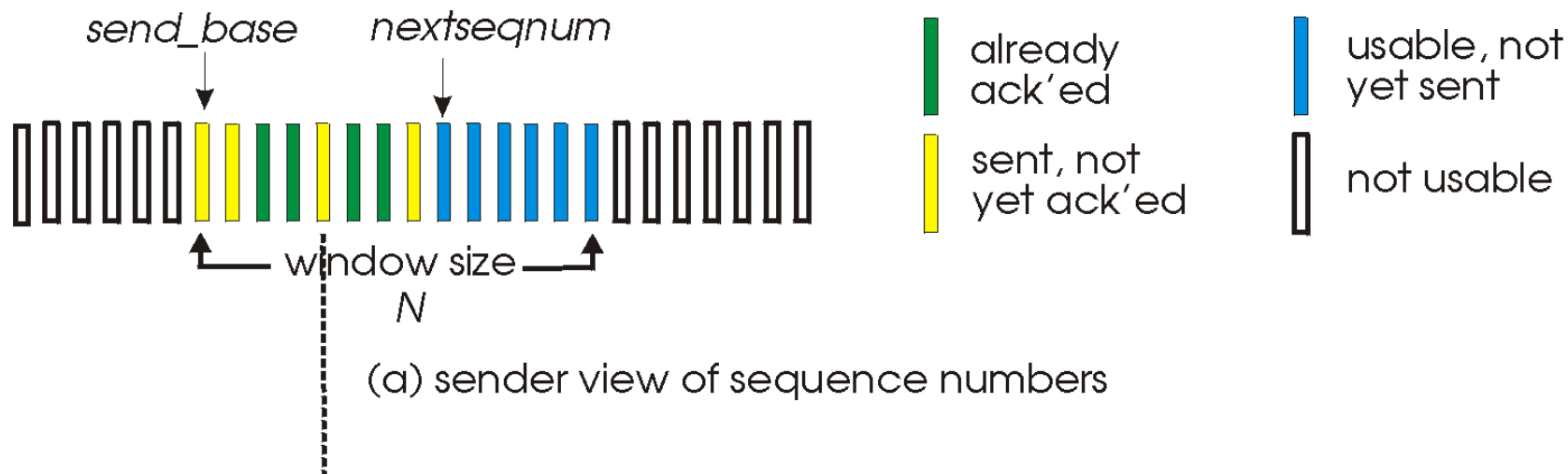
go-back-n 运作过程



选择性重传

- 接收方 **单独地** 确认所有正确接收的数据包
 - 根据需要缓存数据包，以便最终按序交付给上层
- 发送方对每个未确认的数据包单独进行超时/重传未确认的数据包
 - 发送方为每个未确认的未确认的数据包维护计时器
- 发送方窗口
 - N 个连续序列号
 - 限制已发送的、未确认的未确认的数据包的序列号

选择性重传：发送方、接收方窗口



选择性重传：发送方与接收方

sender

来自上层的数据：

- 如果窗口中的下一个可用序列号，发送数据包

超时(n):

- 重发数据包 n ，重启计时器

确认(n) 在[发送基址,发送基址+N]内：

- 标记数据包 n 为已接收
- 如果 n 是最小的未确认数据包，将窗口基址推进到下一个未确认序列号

receiver

数据包 n 在 $[rcvbase, rcvbase+N-1]$ 范围内：

- 发送 ACK(n)
- 乱序：缓冲
- 有序：交付（同时交付已缓冲的、有序的数据包），将窗口推进到下一个未接收的数据包

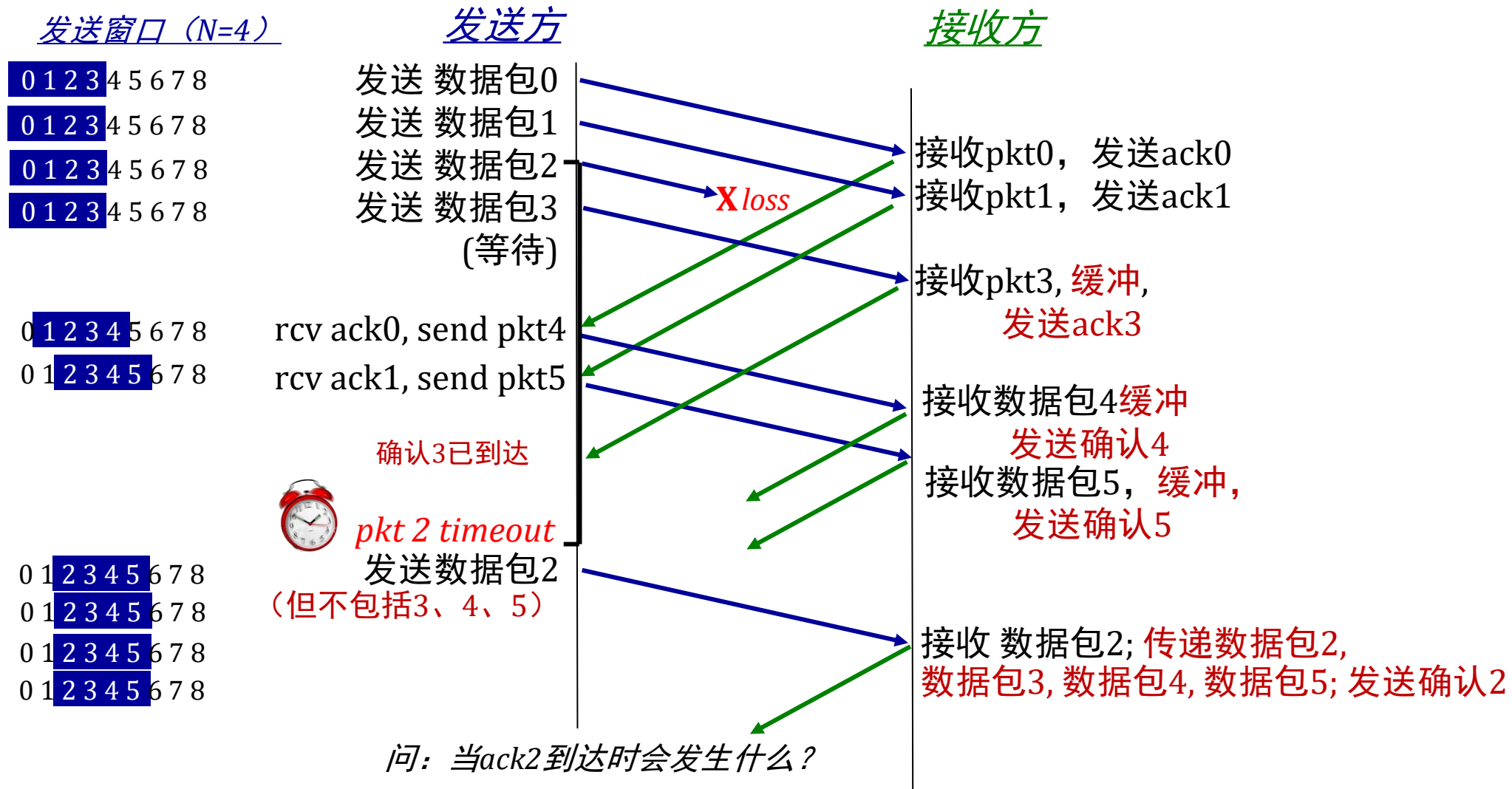
数据包 n 在 $[rcvbase-N, rcvbase-1]$ 内：

- ACK(n)

否则：

- 忽略

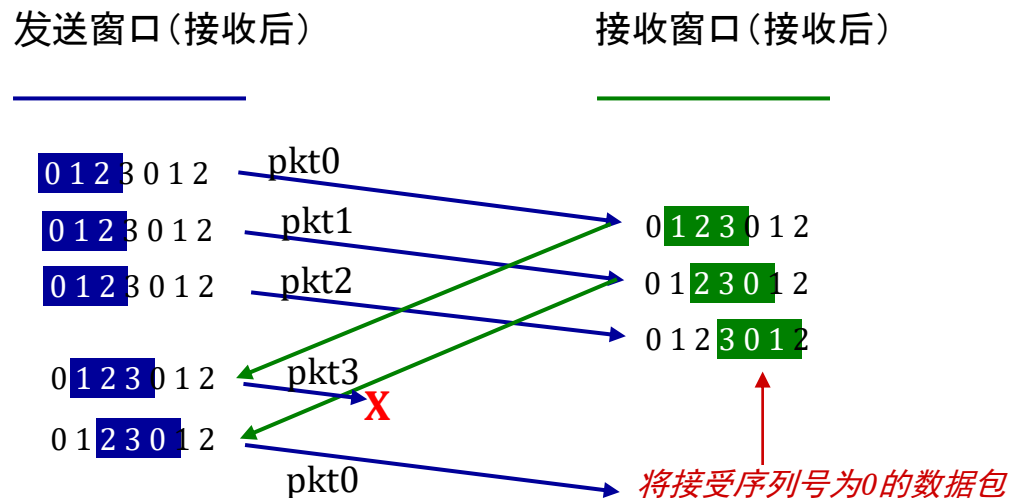
选择性重传的实际应用



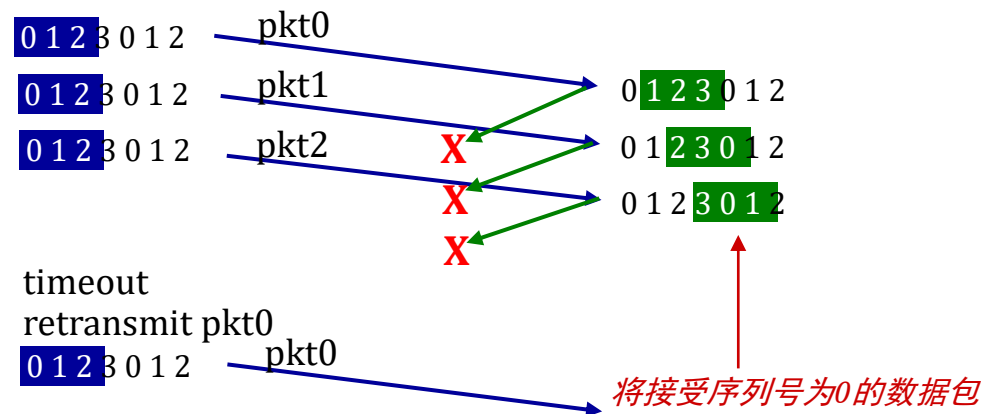
选择性重传： 一个困境！

示例：

- 序列号：0, 1, 2, 3 （四进制计数）
- 窗口大小=3



(a) 没问题



(b) 哎呀！

选择性重传： 一个困境！

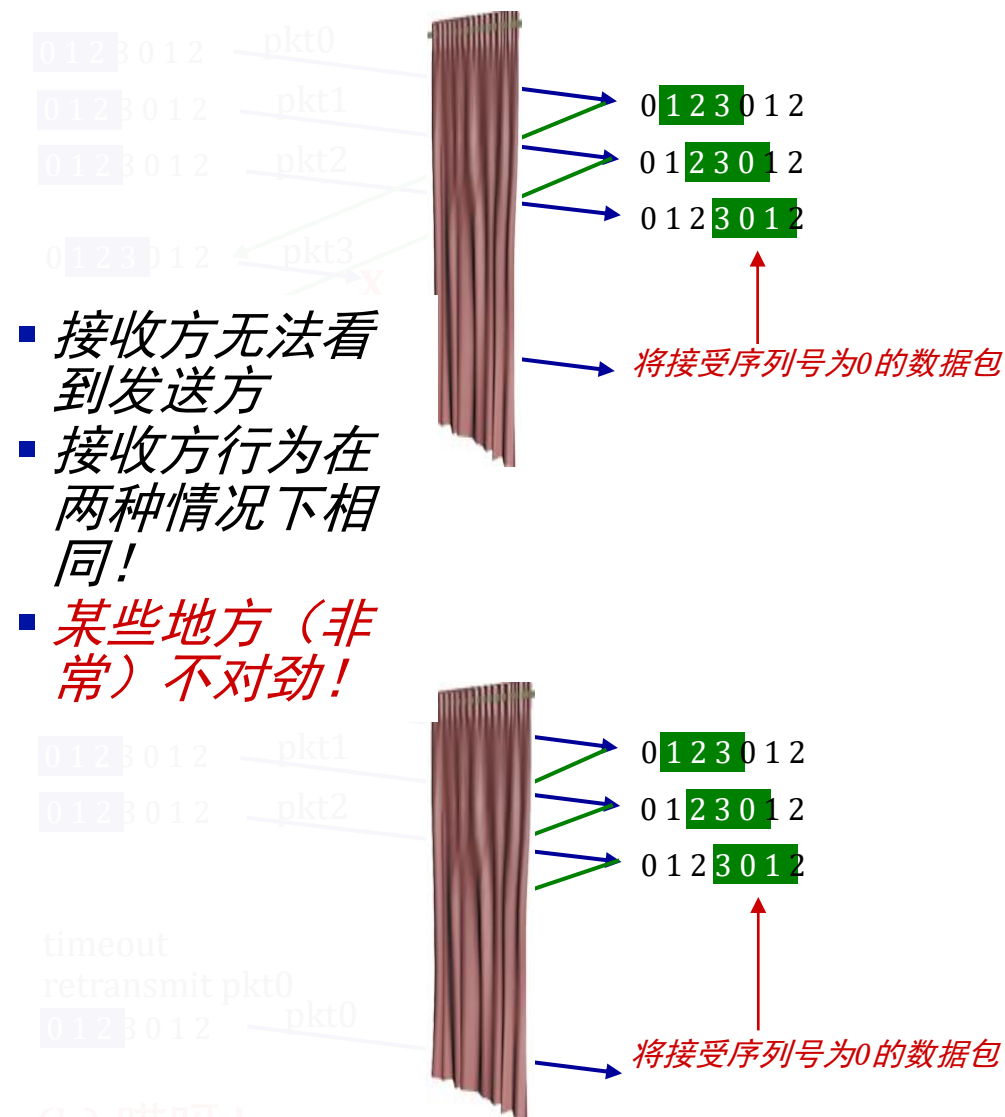
示例：

- 序列号：0, 1, 2, 3 （四进制计数）
- 窗口大小=3

问：为了避免场景（b）中的问题，序列号大小和窗口大小之间需要什么样的关系？

发送窗口（接收后）

接收窗口（接收后）



(b) 哎呀！

协议6中极端情况的分析

当 $n=3$ ，即 帧号为 0 1 2 3 4 5 6 7，且 发送窗口 $W_T =$ 接收窗口 $W_R = 7$

- 发送方当前的发送窗口为0 1 2 3 4 5 6，连续发送了7帧，帧号为0 1 2 3 4 5 6，然后等待确认
- 接收方在初始化后，接收窗口为0 1 2 3 4 5 6，在正确收到0#帧后，由于捎带确认，所以立即启动辅助定时器，并且在辅助定时器没有超时之前，发送方发送的7帧都正确地收到后，然后辅助定时器超时，接收方立即发送了ACK7，意即0 ~ 6帧全部收到并期待接收第7帧，然后取出分组交网络层、清缓冲区并调整窗口为7 0 1 2 3 4 5
- 发送方一直在等待确认，但接收方发送的ACK7由于某种原因丢失了，在重发定时器陆续超时的过程中，发送方又陆续重发了0 1 2 3 4 5 6帧，并继续等待确认

协议6中极端情况的分析（续）

- 接收方收到0 1 2 3 4 5 6帧，认为是第二批来的帧，按正常处理，发现0 1 2 3 4 5均在其接收窗口内，当然接收并存入缓冲，6丢弃，但由于期待接收的第7帧未到，所以只能仍发ACK7，意即再次确认上次收到的0 ~ 6，但由于第7帧未到，所以，已收到的0 1 2 3 4 5帧不能上交网络层
- 但在发送方来看，在收到了ACK7后才知道，重发的0 ~ 6总算收到了，于是，调整窗口为7 0 1 2 3 4 5，又从网络层取分组，并发送第二批帧
- 接收方在收到7 0 1 2 3 4 5后，发现0 1 2 3 4 5帧已在缓冲区中，是重复的，应丢弃，第7帧接收，发送ACK6，然后将7 0 1 2 3 4 5交网络层，清缓冲区、调整接收窗口
- 此时，接收方的网络层发现：数据链路层交来的第二批分组中的0 1 2 3 4 5与原来的重复，协议失败

当 $W_T=W_R=7$ 时协议6失败的原因

- 原因在于接收窗口过大，窗口中的有效顺序号在调整前和调整后有重叠
- 所以，通常：发送窗口 + 接收窗口 $\leq 2^n$
且：发送窗口 = 接收窗口
- 发送窗口 $>$ 接收窗口或者发送窗口 $<$ 接收窗口都是不合适的

第3章：路线图

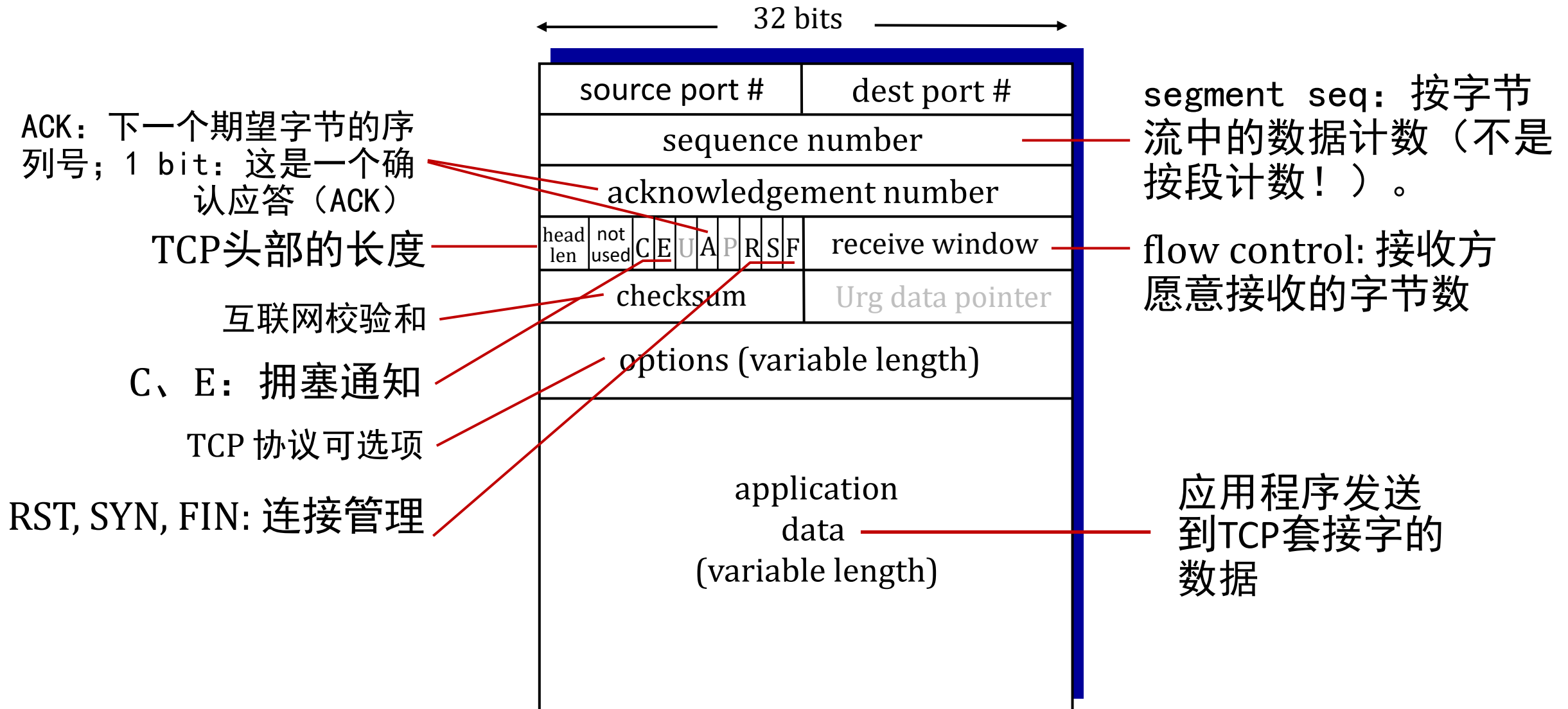
- 传输层服务
- 多路复用与多路分解
- 无连接传输：UDP
- 可靠数据传输原理
- 面向连接的传输：TCP
 - 段结构
 - 可靠数据传输
 - 流量控制
 - 连接管理
- 拥塞控制原理
- TCP拥塞控制



TCP: 概述 RFCs: 793, 1122, 2018, 5681, 7323

- 点对点:
 - 一个发送者, 一个接收者
- 可靠、有序的字节流:
 - 无“消息边界”
- 全双工数据:
 - 同一连接中的双向数据流
 - MSS: 最大段大小
- 累积确认
- 流水线技术:
 - TCP拥塞和流量控制设置窗口大小
- 面向连接的:
 - 握手（交换控制消息）在数据交换前初始化发送方和接收方状态
- 流量控制:
 - 发送方不会使接收方过载

TCP段结构



TCP序列号，确认号

序列号:

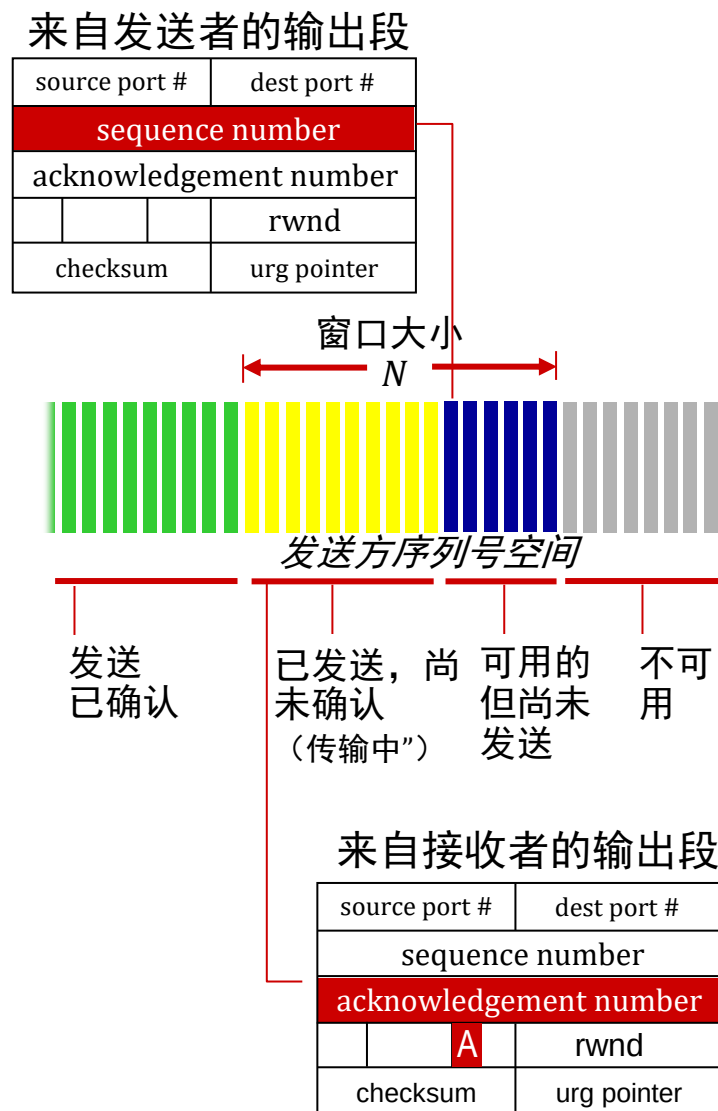
- 字节流“段数据中第一个字节的编号”

ACK:

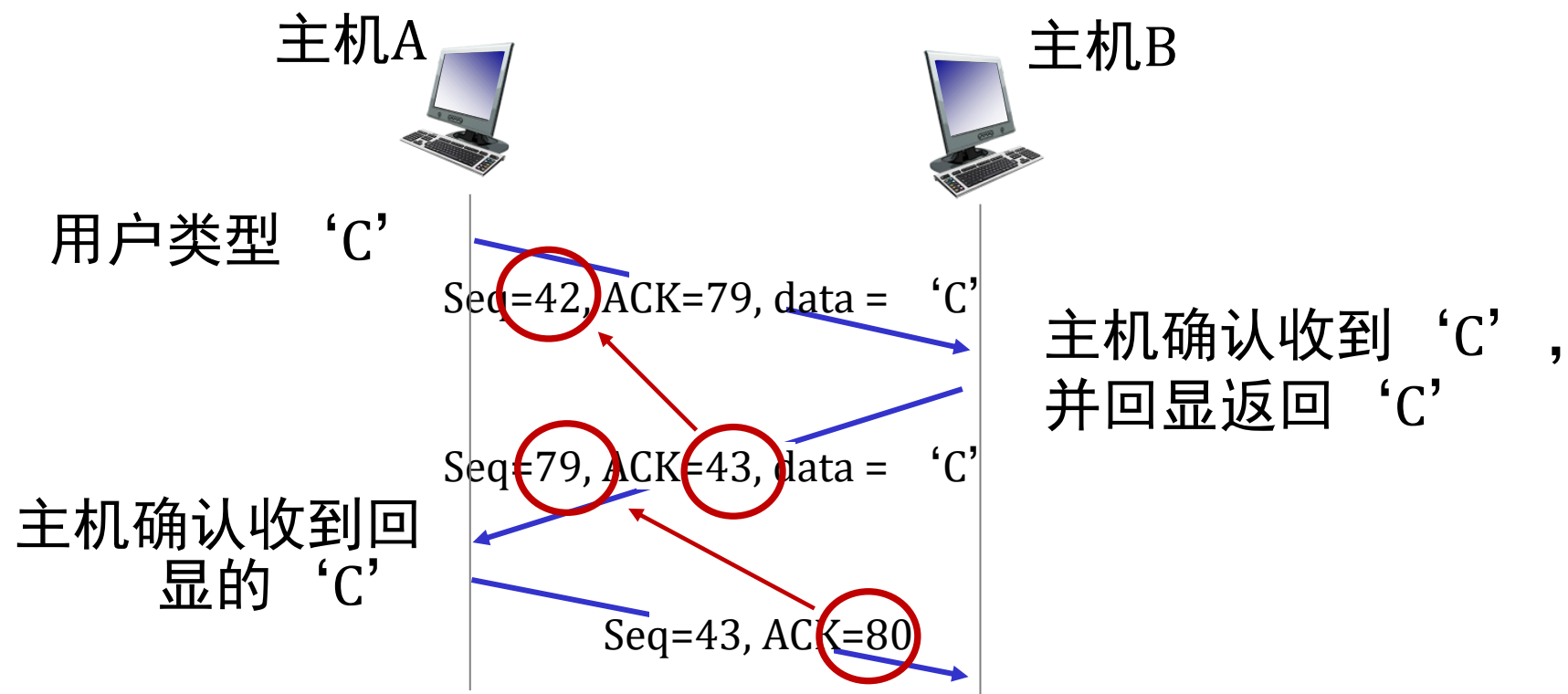
- 期望的从对方接收的下一个字节的序列号
- 累积确认

问: 接收方如何处理乱序的段

- 答: TCP规范并未明确规定，由实现者决定



TCP序列号，确认号



简单的telnet场景

TCP往返时间，超时

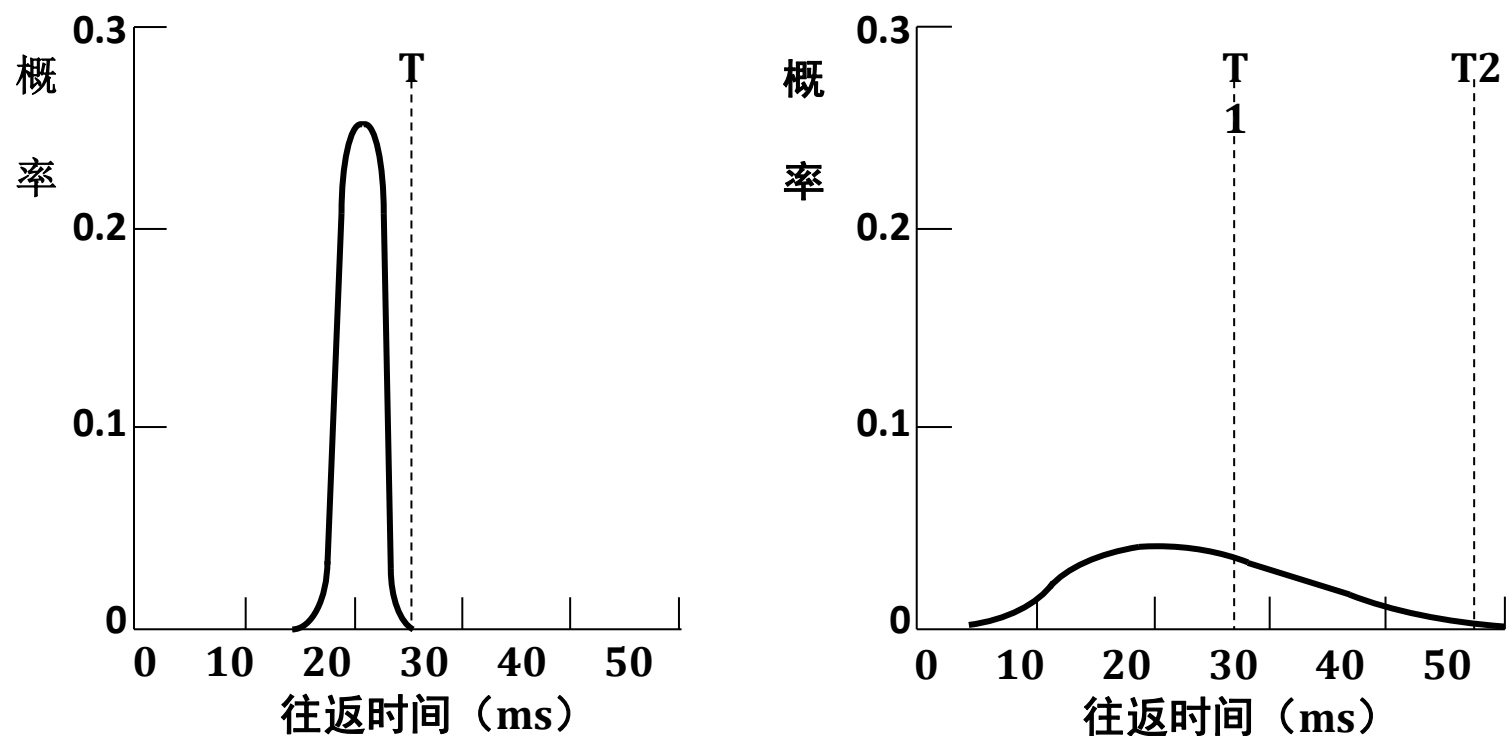
问： 如何设置TCP超时值？

- 应大于RTT，但RTT是变化的！（见下一页）
- 过短： 导致过早超时，不必要的重传
- 过长： 对数据段丢失反应迟钝

问： 如何估算RTT？

- **SampleRTT**：测量从段传输到ACK接收的时间
 - 忽略重传
- **SampleRTT** 会变化，希望估算的RTT“更平滑”
 - 对几个最近的测量值取平均，而不仅仅是当前的**SampleRTT**

链路层与TCP层延时分布对比



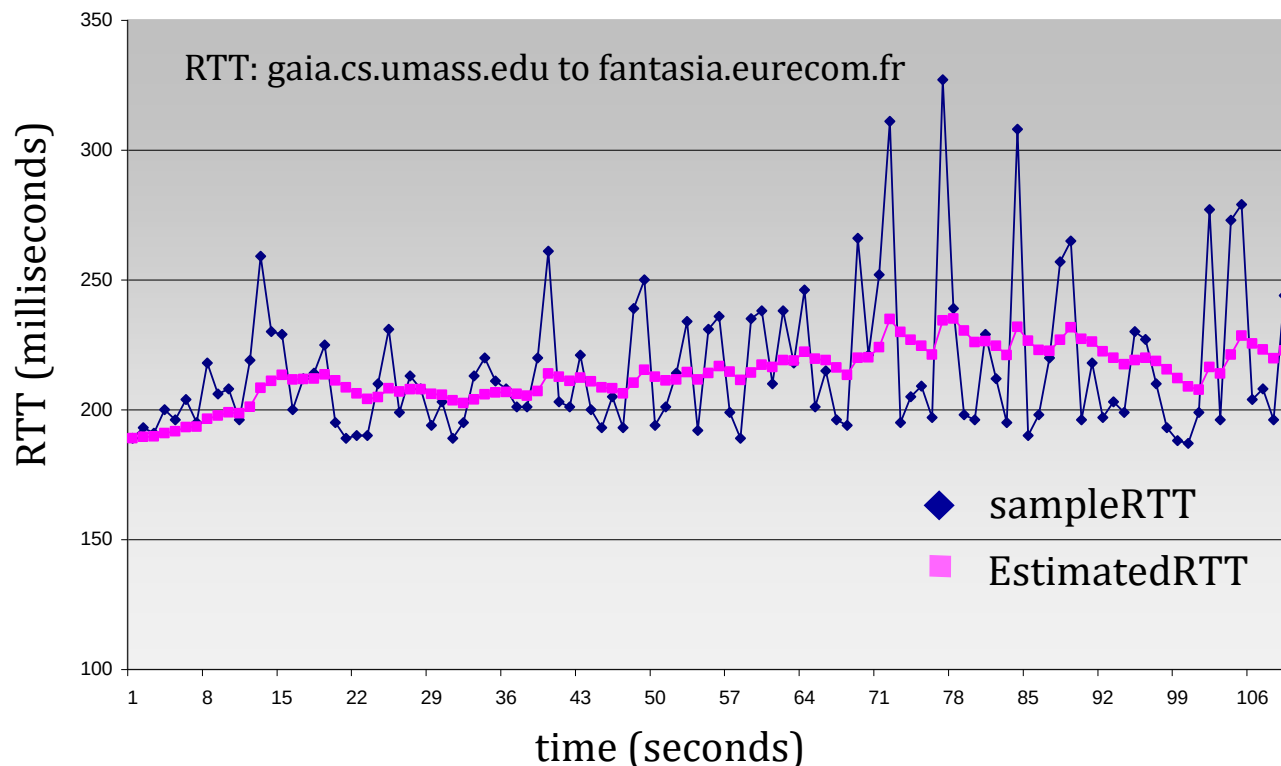
Tnbm P551 图 6-38

(a)数据链路层中确认到达时间的概率密度 (b)TCP层中确认到达时间的概率密度

TCP往返时间， 超时

$$\text{EstimatedRTT} = (1 - \alpha) * \text{EstimatedRTT} + \alpha * \text{SampleRTT}$$

- 指数加权移动平均（exponential weighted moving average, EWMA）
- 过去样本的影响指数级递减
- 典型值： $\alpha=0.125$



TCP往返时间，超时

- 超时间隔： **EstimatedRTT**加上“安全裕度（safety margin）”
 - 在 EstimatedRTT 中存在较大波动：需要更大的安全裕度

$$\text{TimeoutInterval} = \text{EstimatedRTT} + 4 * \text{DevRTT}$$



↑
estimated RTT

↑
“safety margin”

- **DevRTT**： **SampleRTT** 与 **EstimatedRTT** 偏差的指数加权移动平均（EWMA）：

$$\text{DevRTT} = (1 - \beta) * \text{DevRTT} + \beta * |\text{SampleRTT} - \text{EstimatedRTT}|$$

(typically, $\beta = 0.25$)

TCP发送方（简化版）

事件：从应用程序接收到数据

- 创建带有序列号的段
- 序列号是段中第一个数据字节的字节流编号
- 如果尚未启动，则启动计时器
 - 将计时器视为针对最旧的未确认段
 - 过期间隔：**TimeoutInterval**

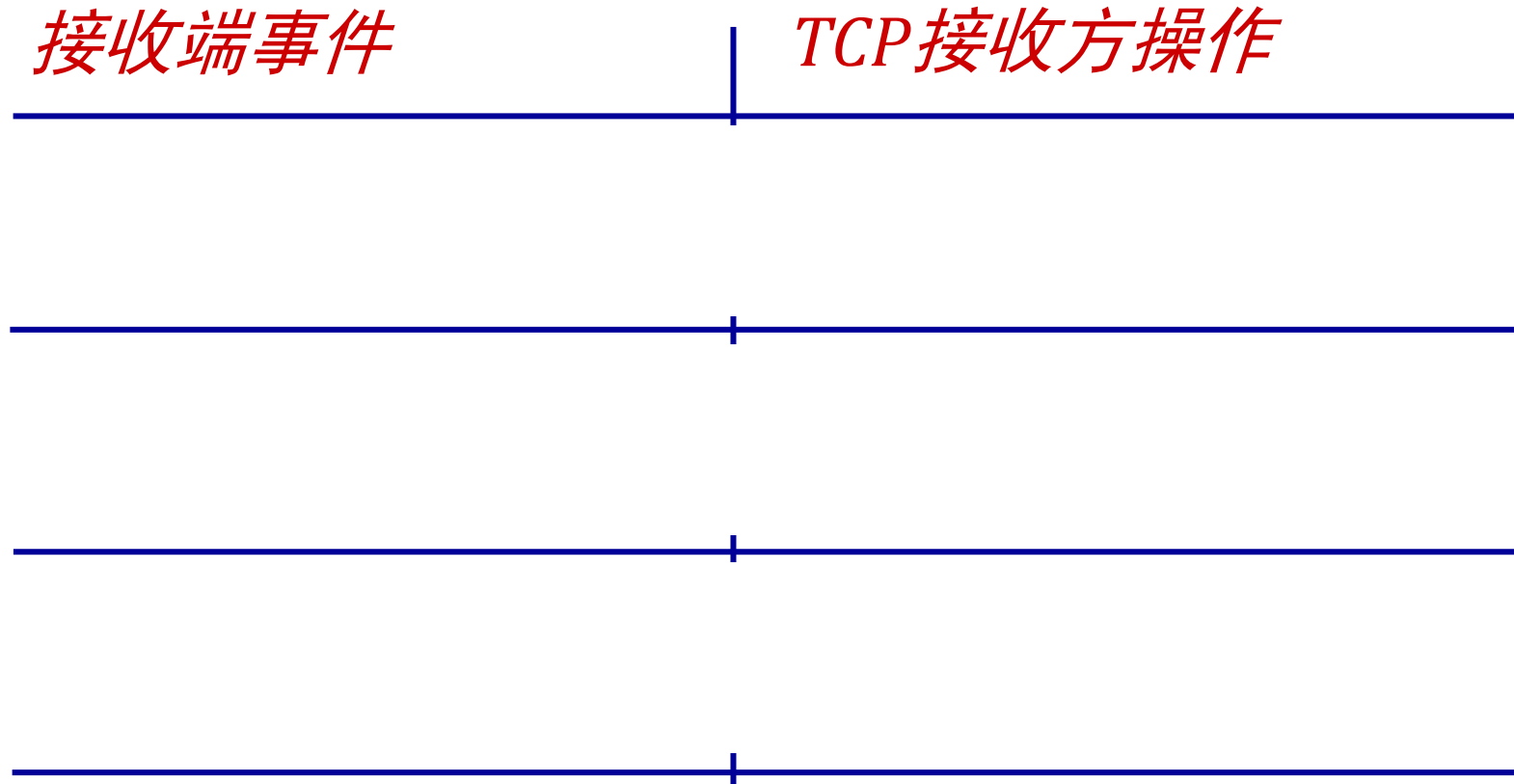
事件：超时

- 重传导致超时的段
- 重启计时器

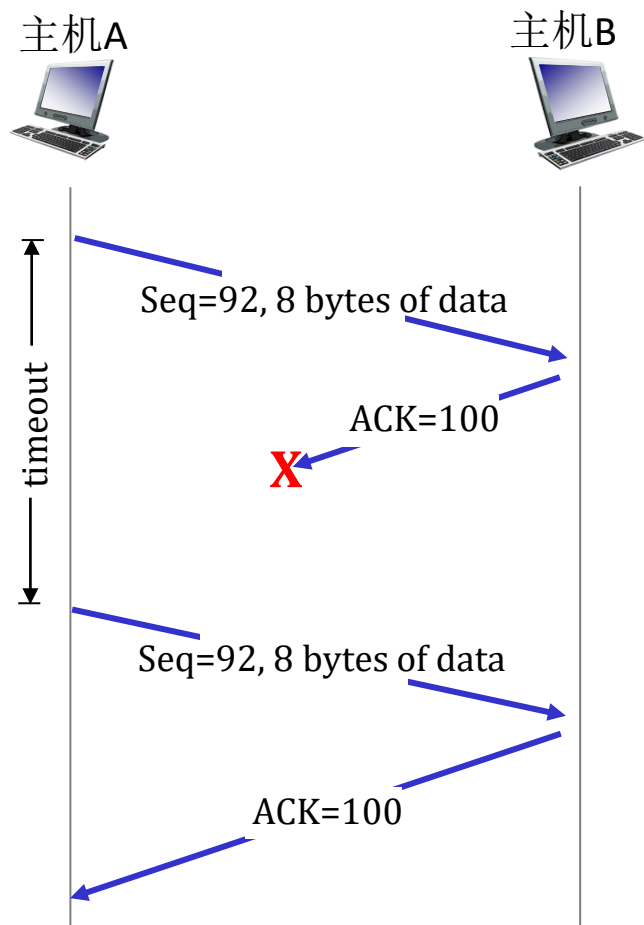
事件：收到ACK

- 如果ACK确认了之前未确认的段
 - 更新已知的已确认
 - 如果仍有未确认的段，则启动计时器

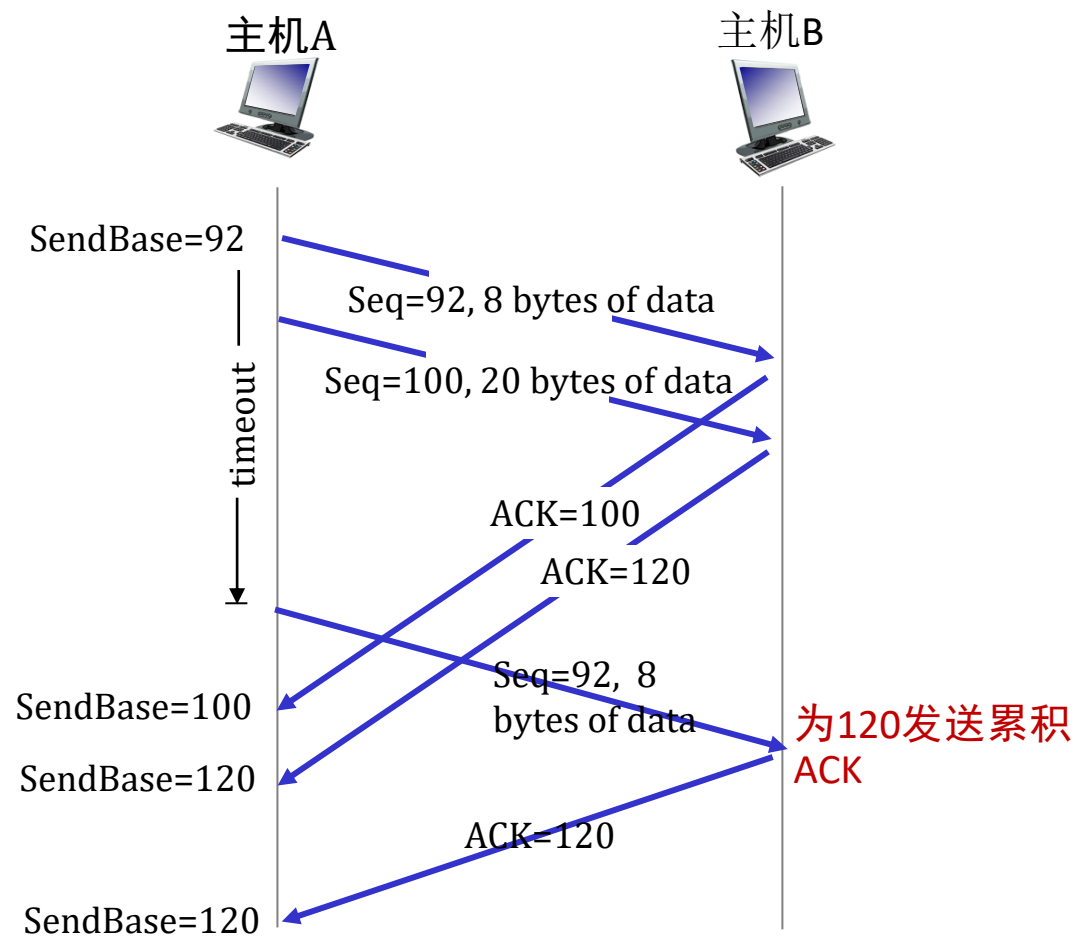
TCP接收方：ACK生成 [RFC 5681]



TCP: 重传场景

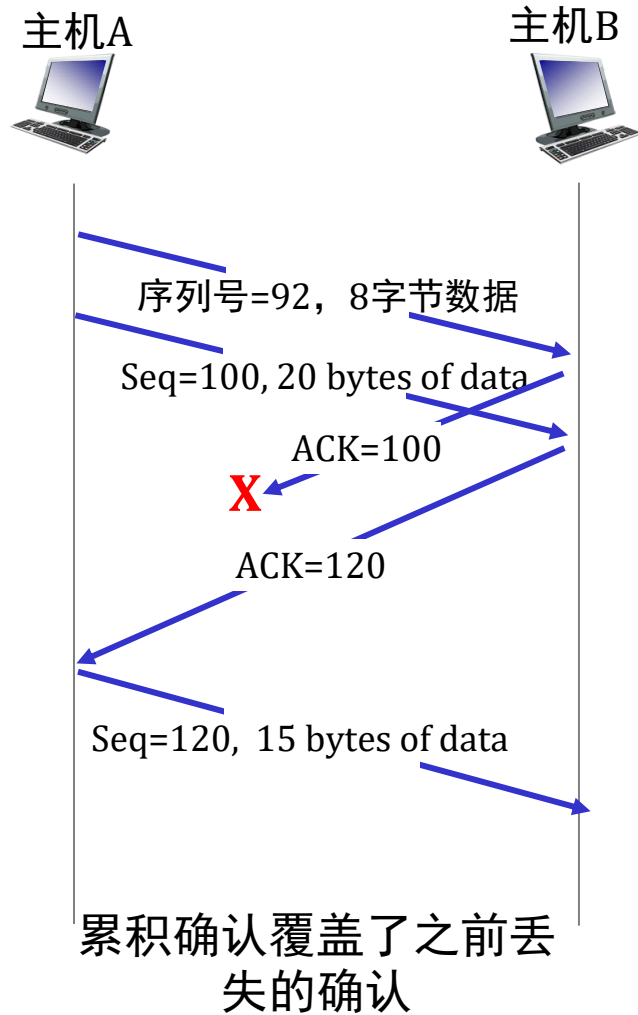


丢失ACK的场景



过早超时

TCP: 重传场景



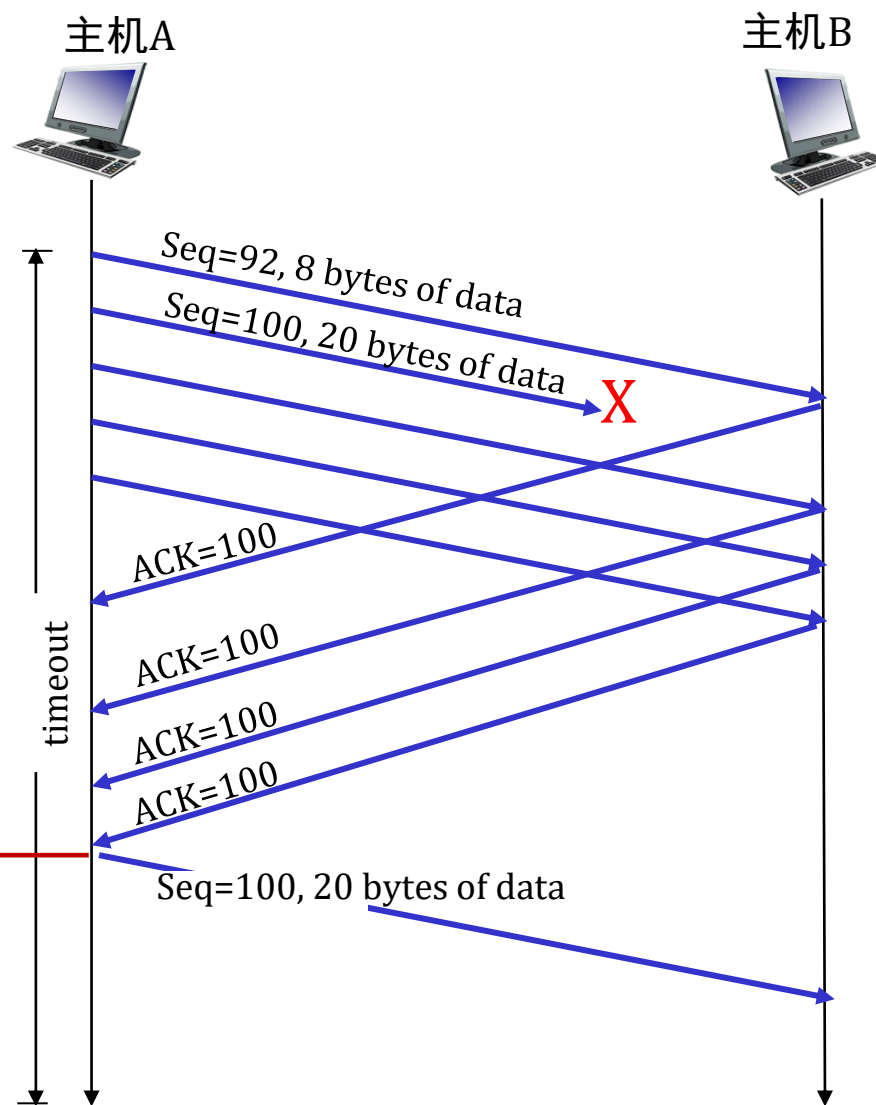
TCP快速重传（快速重传）

TCP fast retransmit

如果发送方收到相同数据的 3 个额外的确认应答（“三重重复 ACK”），则重新发送未确认的序列号最小的数据段



收到三重重复的 ACK 表示在丢失的段之后已经接收到 3 个段——丢失的段很可能已经丢失。因此，进行重传！



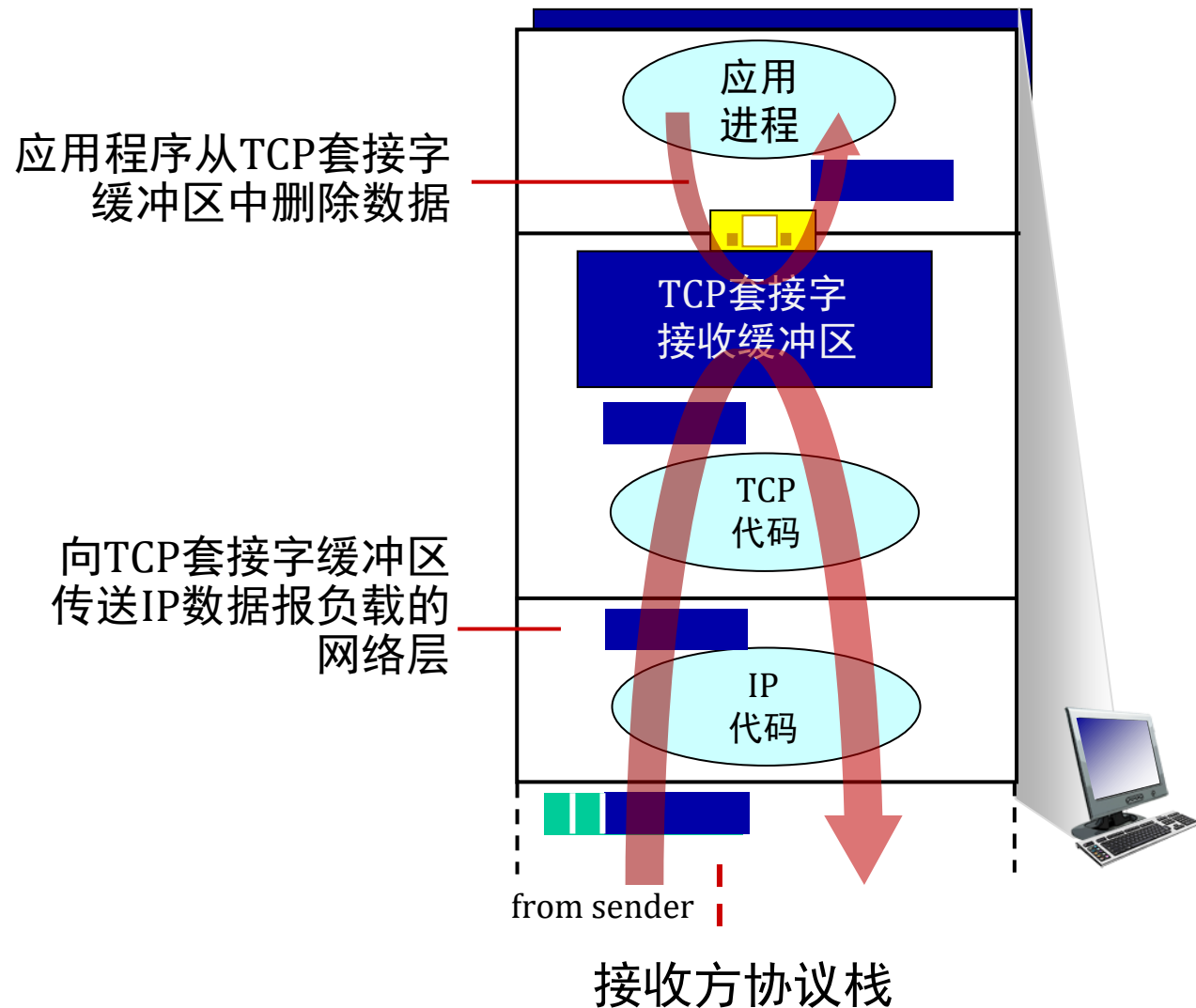
第3章：路线图

- 传输层服务
- 多路复用与多路分解
- 无连接传输：UDP
- 可靠数据传输原理
- 面向连接的传输：TCP
 - 段结构
 - 可靠数据传输
 - 流量控制
 - 连接管理
- 拥塞控制原理
- TCP拥塞控制



TCP流量控制

问: 如果网络层传递数据的速度比应用层从套接字缓冲区移除数据的速度快, 会发生什么?



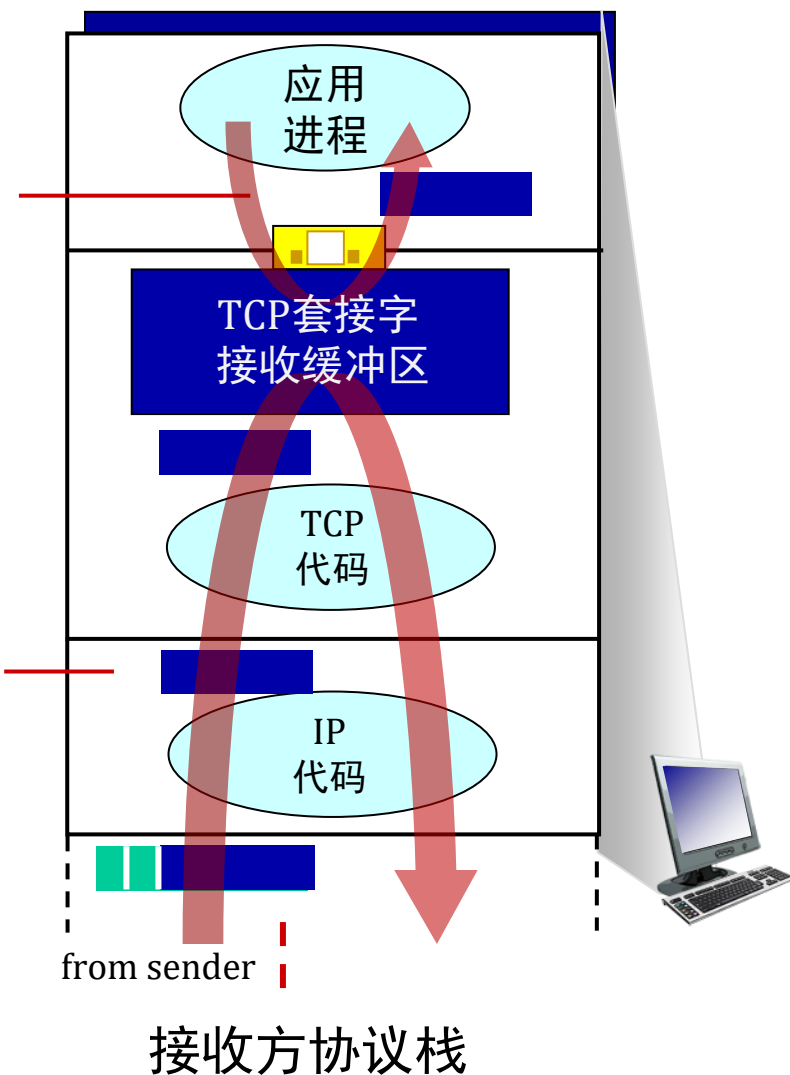
TCP流量控制

问: 如果网络层传递数据的速度快于应用层从套接字缓冲区移除数据的速度, 会发生什么?



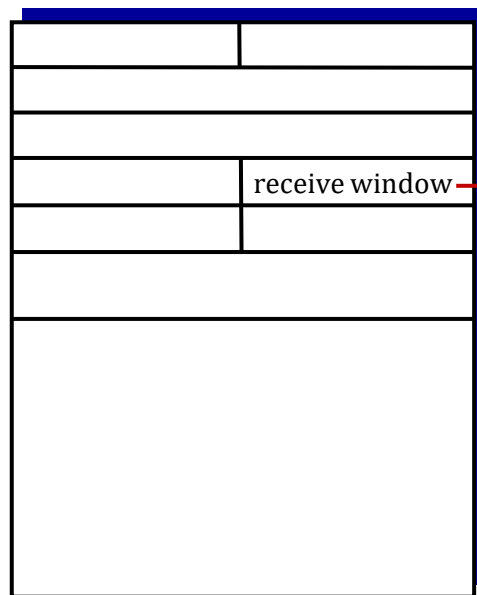
应用程序从TCP套接字缓冲区中删除数据

向TCP套接字缓冲区传送IP数据报负载的网络层



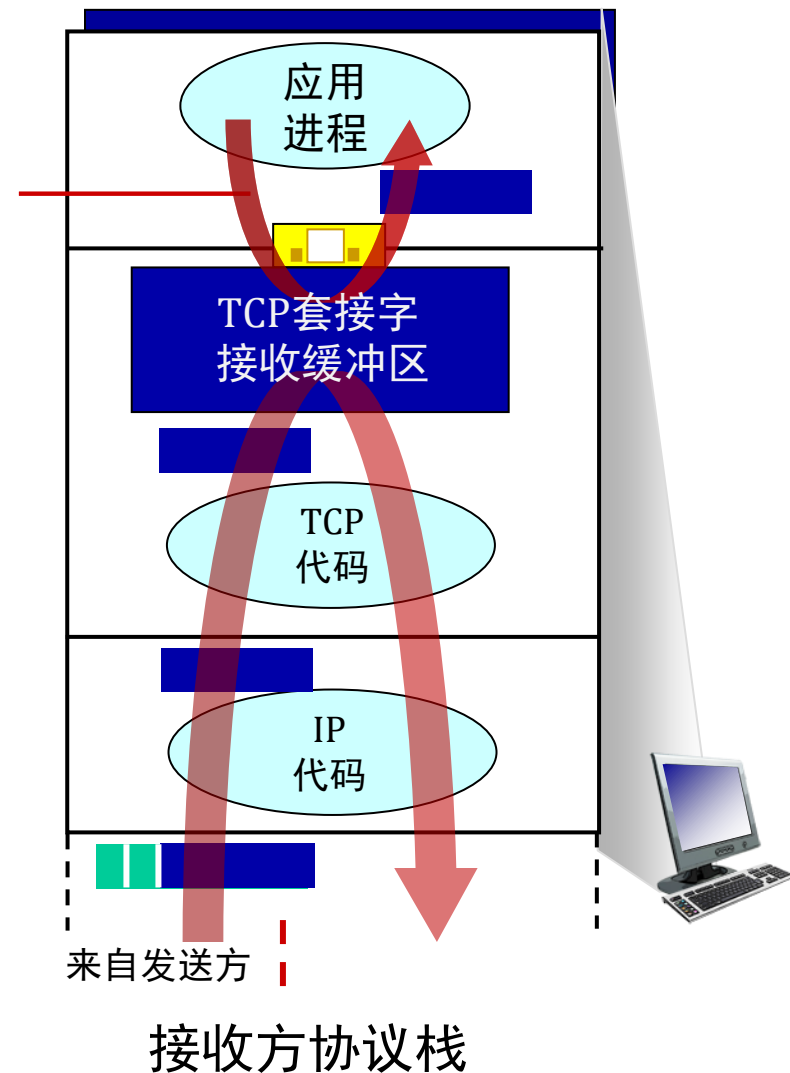
TCP流量控制

问: 如果网络层传递数据的速度比应用层从套接字缓冲区移除数据的速度快, 会发生什么?



流量控制: 接收方愿意接收的字节数

应用程序从TCP套接字缓冲区中删除数据



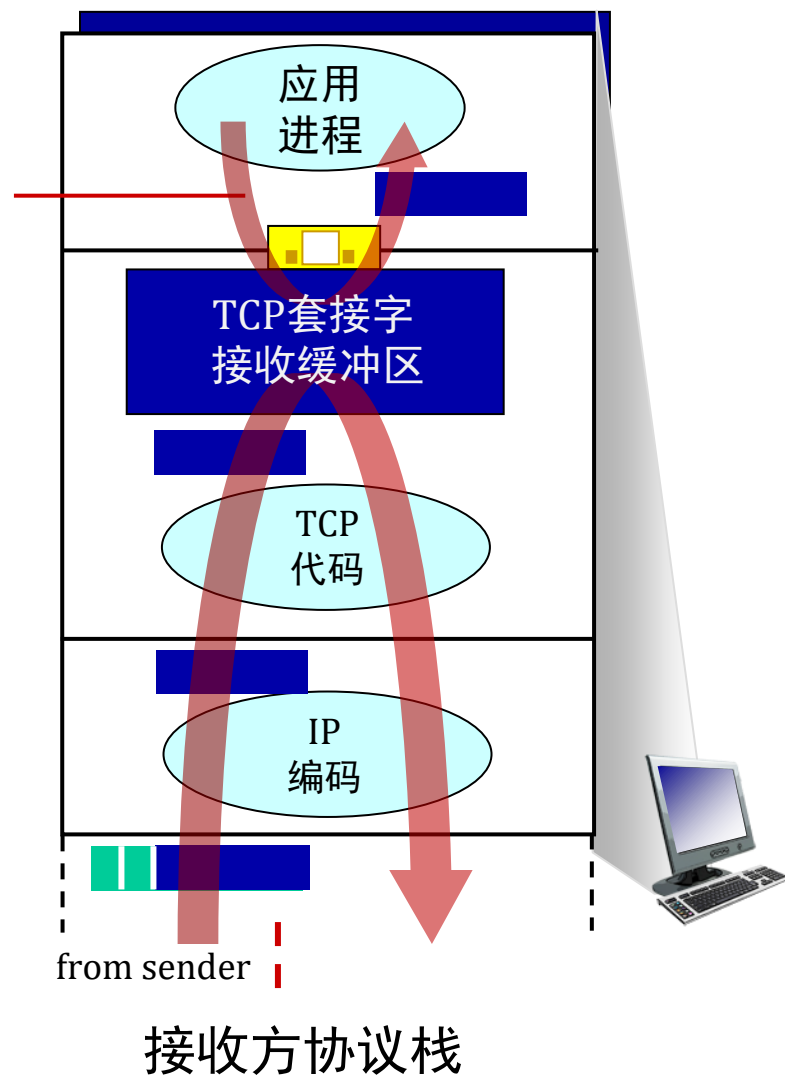
TCP流量控制

问: 如果网络层传递数据的速度比应用层从套接字缓冲区移除数据的速度快, 会发生什么?

flow control

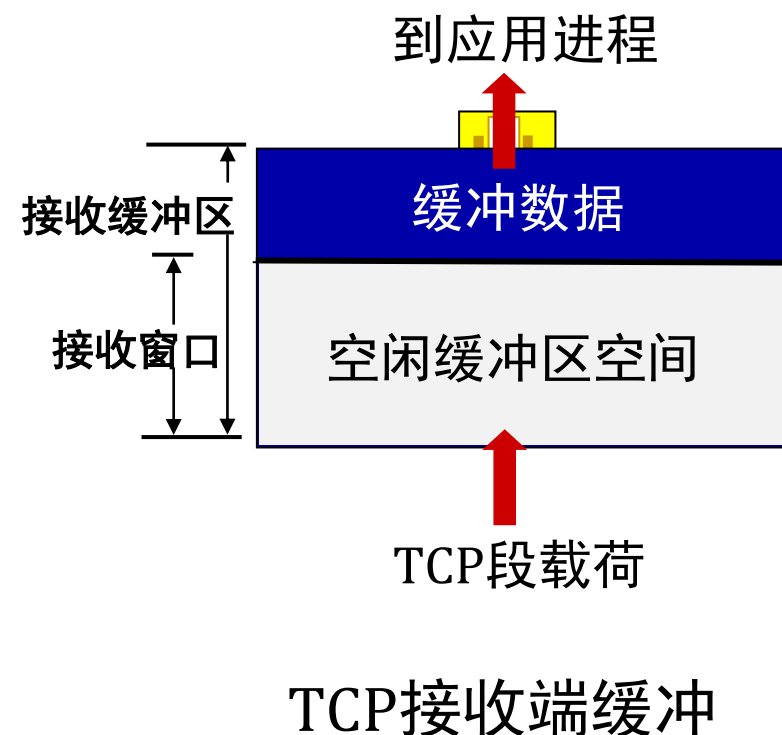
接收方控制发送方, 所以发送方不会因为发送太多太快而溢出接收方的缓冲区

应用程序从TCP套接字缓冲区中删除数据



TCP流量控制

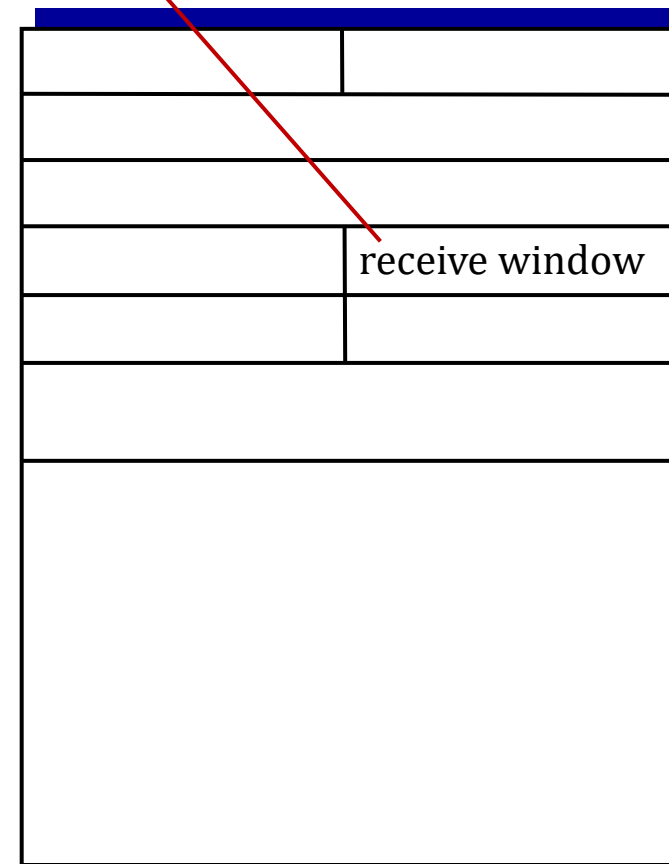
- TCP接收端“通告”空闲缓冲区空间于**rwnd**字段（位于TCP首部中）
 - **RcvBuffer** 大小通过套接字选项设置（默认通常为4096字节）
 - 许多操作系统自动调整 **RcvBuffer**
- 发送方限制未确认（“传输中”）数据量为接收端**rwnd**
- 确保接收缓冲区不会溢出



TCP流量控制

- TCP接收端“通告”空闲缓冲区空间于**rwnd**字段中
 - **RcvBuffer** 大小通过套接字选项设置（典型默认值为4096字节）
 - 许多操作系统自动调整 **RcvBuffer**
- 发送方限制未确认的（“传输中”）数据量不超过接收到的**rwnd**
- 确保接收缓冲区不会溢出

流量控制：接收方愿意接受的字节数

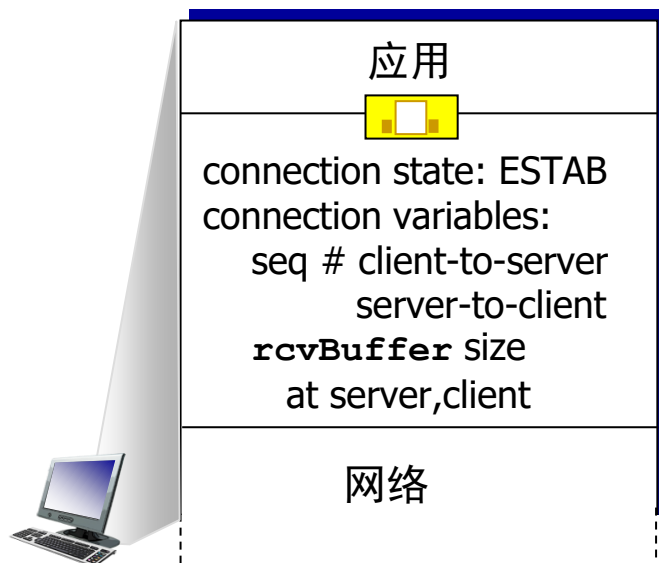


TCP段格式

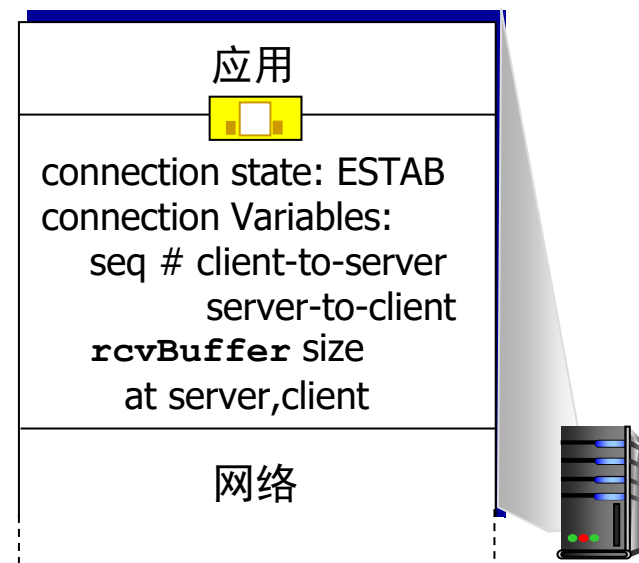
TCP连接管理

在交换数据之前，发送方/接收方“握手”：

- 同意建立连接（双方都知道对方愿意建立连接）
- 就连接参数达成一致（例如，起始序列号）



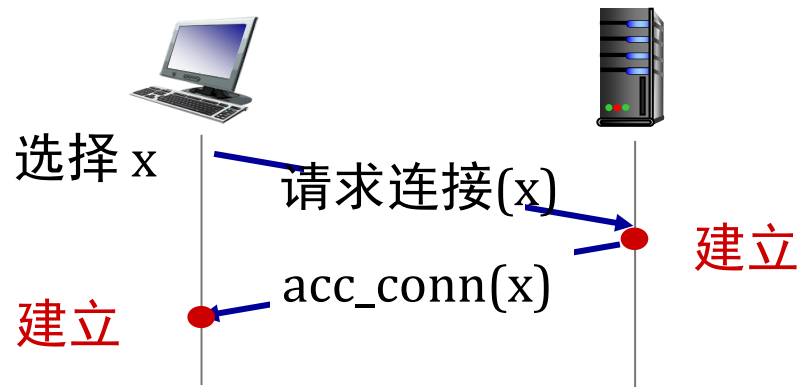
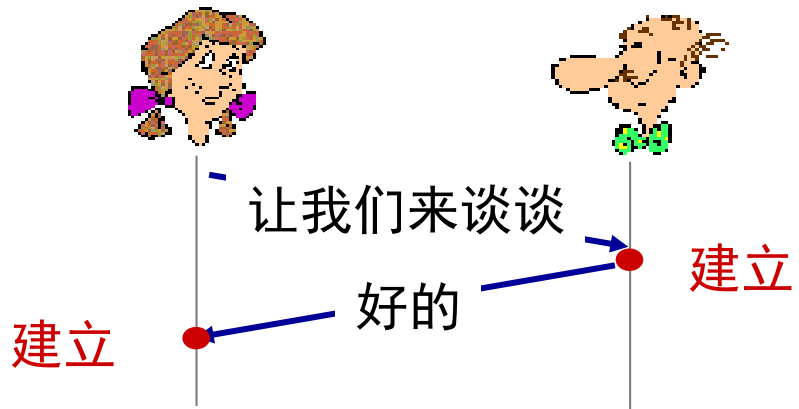
```
Socket clientSocket =  
    newSocket("hostname", "port number");
```



```
Socket connectionSocket =  
    welcomeSocket.accept();
```

同意建立连接

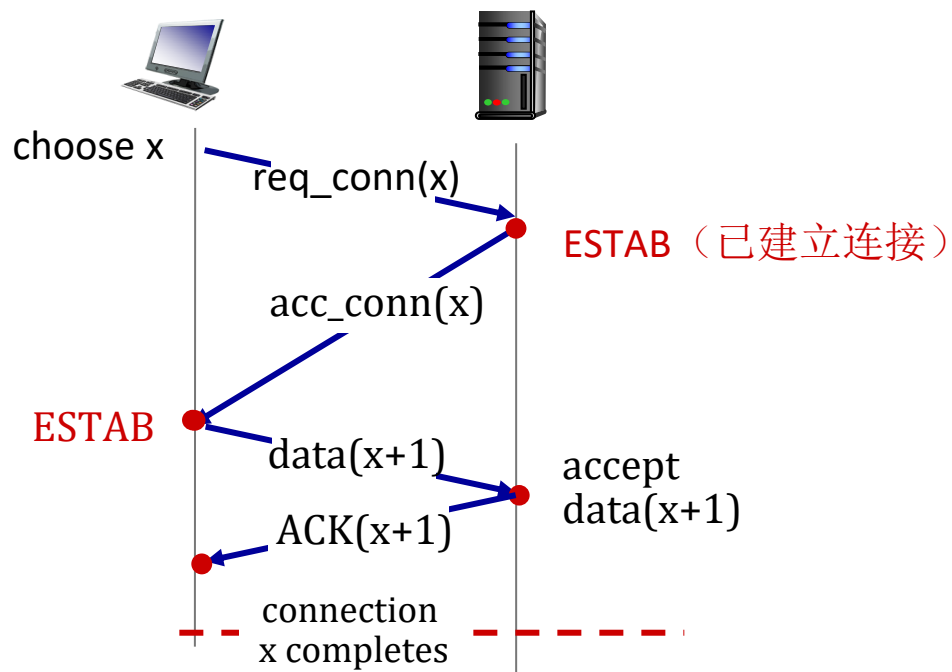
两次握手：



问：双向握手在网络中总能奏效吗？

- 可变的延迟
- 因消息丢失而重传的消息（例如 req_conn(x)）
- 消息重排序
- 无法“看到”对方

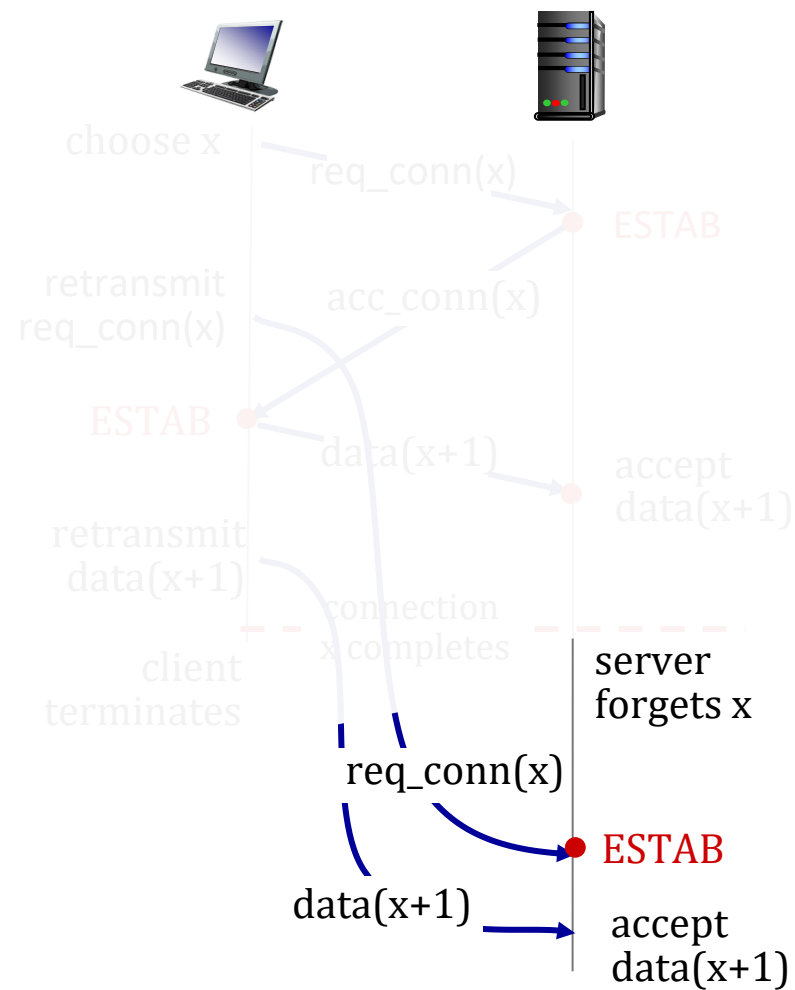
双向握手场景



No problem!



双向握手场景



Problem: 接收到重复数据!

TCP 三次握手

客户端状态

```
clientSocket = socket(AF_INET, SOCK_STREAM)
```

LISTEN

```
clientSocket.connect((serverName, serverPort))
```

SYNSENT

ESTAB 表示服务器处于活动状态;
send ACK for SYNACK;
此段可能包含客户端到服务器的数据

choose init seq num, x
send TCP SYN msg

SYNbit=1, Seq=x

SYNbit=1, Seq=y
ACKbit=1; ACKnum=x+1

ACKbit=1, ACKnum=y+1

received ACK(y)
表示客户端处于激活状态

服务器状态

```
serverSocket = socket(AF_INET, SOCK_STREAM)  
serverSocket.bind(('', serverPort))  
serverSocket.listen(1)  
connectionSocket, addr = serverSocket.accept()
```

LISTEN

SYN RCVD

ESTAB

人类的三次握手协议



关闭TCP连接

- 客户端和服务端各自关闭其连接端
 - 发送FIN位设置为1的TCP段
- 对接收到的FIN进行ACK响应
 - 在接收到FIN时，ACK可与自身的FIN合并
- 同时进行的FIN交换可以被处理

第3章：路线图

- 传输层服务
- 多路复用与解复用
- 无连接传输：UDP
- 可靠数据传输原理
- 面向连接的传输：TCP
- **拥塞控制原理**
- TCP拥塞控制
- 传输层功能的演进



拥塞控制原理

拥塞：

- 非正式定义：“过多的源以过快的速度发送过多的数据，超出**网络**的处理能力”
- 表现：
 - 长延迟（路由器缓冲区中的排队）
 - 数据包丢失（路由器缓冲区溢出）
- 不同于流量控制！
- 十大问题！



拥塞控制：
发送者太多
发送太快

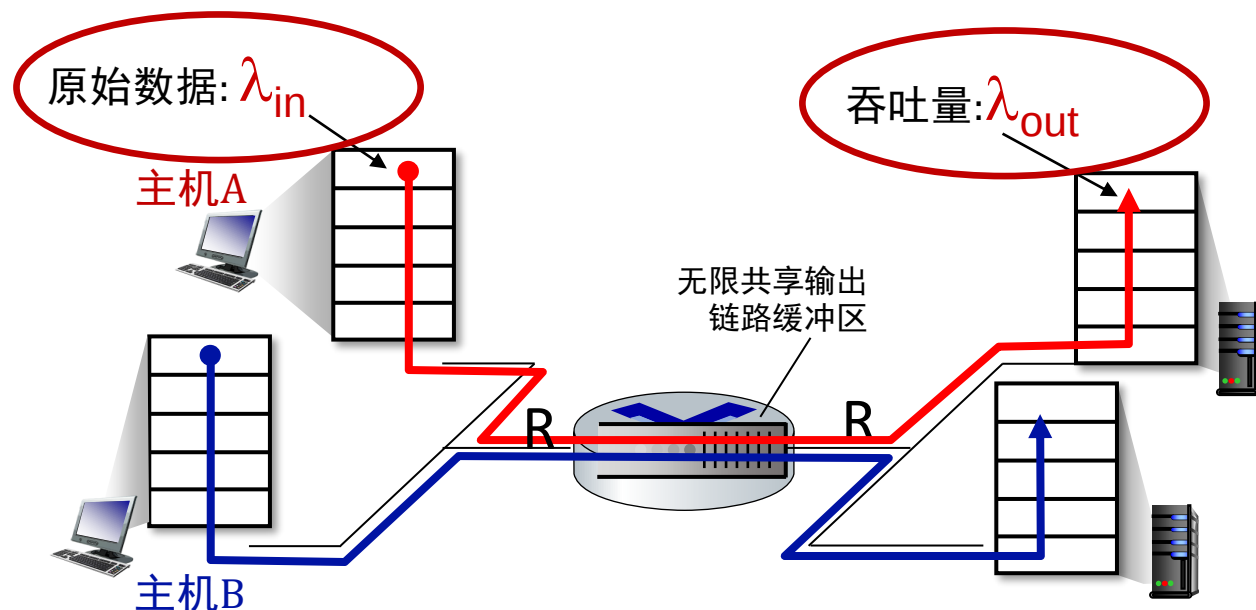


流量控制：一个发送者对于一个接收者来说太快了

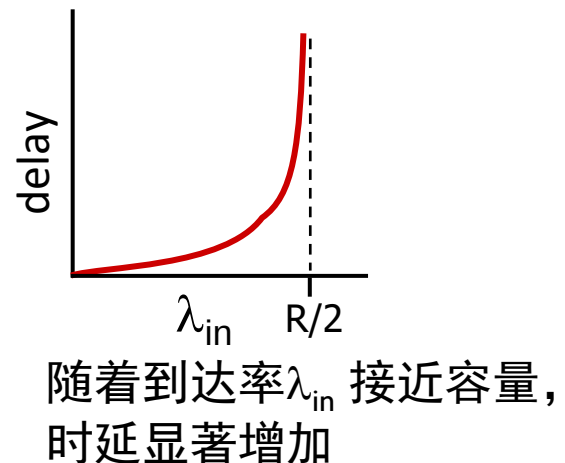
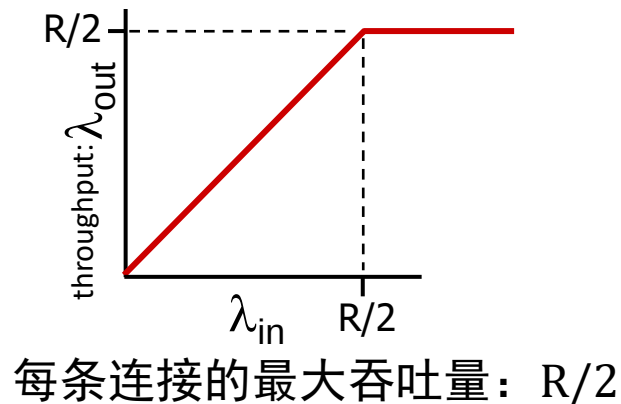
拥塞的成因/代价：场景1

最简单的场景：

- 一台路由器，无限缓冲区
- 输入，输出链路容量：R
- 两个流
- 无需重传

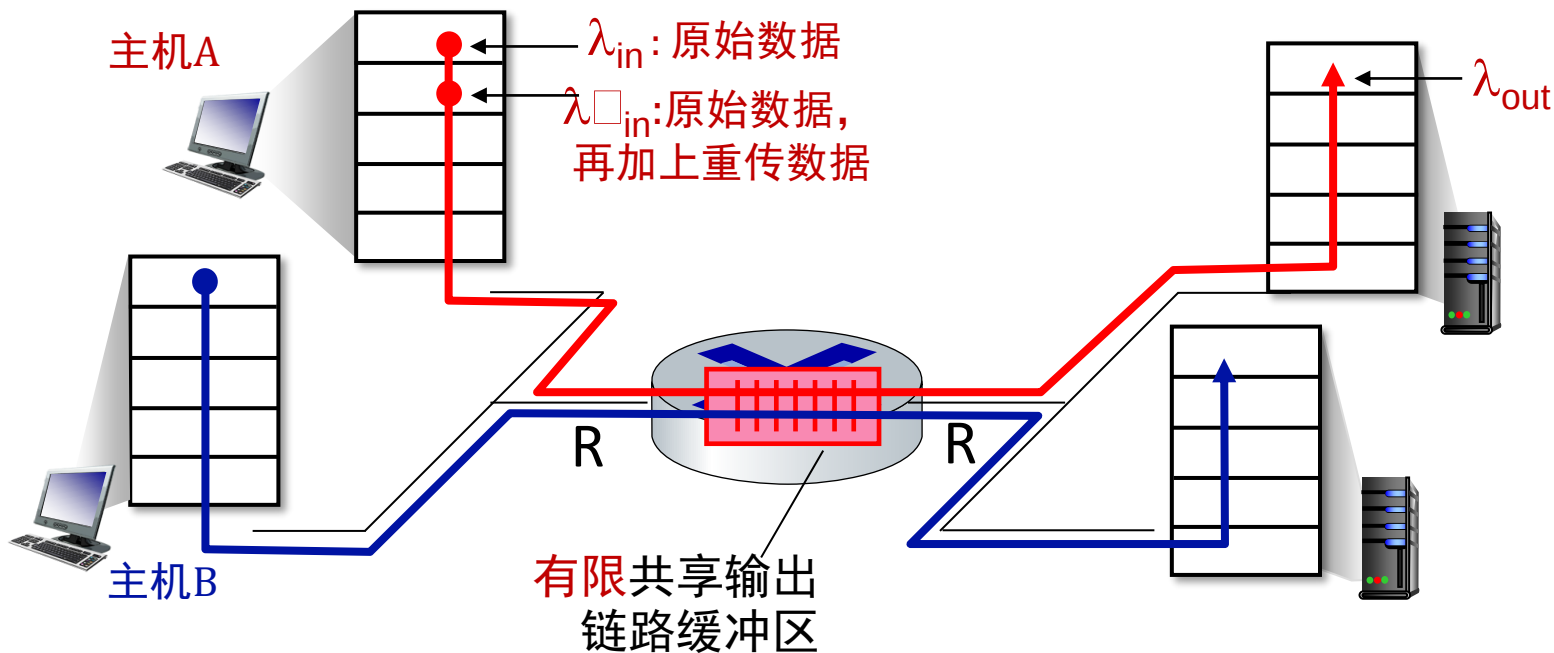


问：当到达率 λ_{in} 接近 $R/2$ 时会发生什么情况？



拥塞的成因/代价：场景2

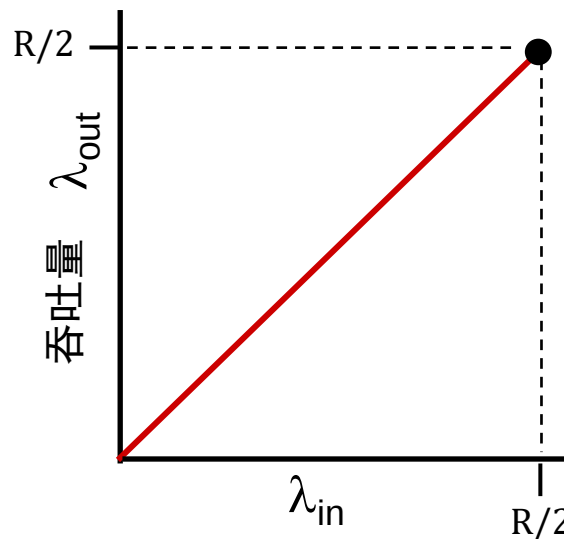
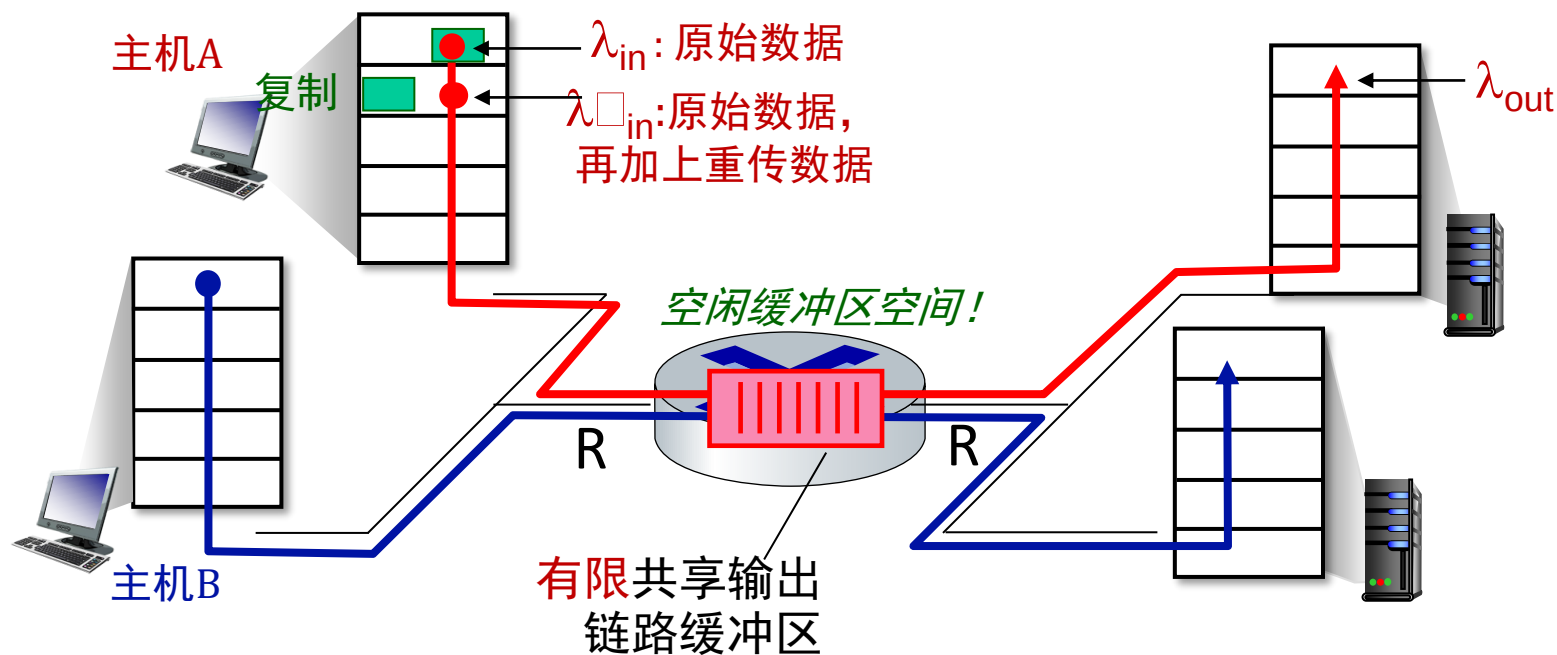
- 一台路由器，**有限的**缓冲区
- 发送方重传丢失的超时数据包
 - 应用层输入=应用层输出: $\lambda_{in} = \lambda_{out}$
 - 传输层输入包括重传: $\lambda'_{in} \geq \lambda_{in}$



拥塞的成因/代价：场景2

理想化：完美信息

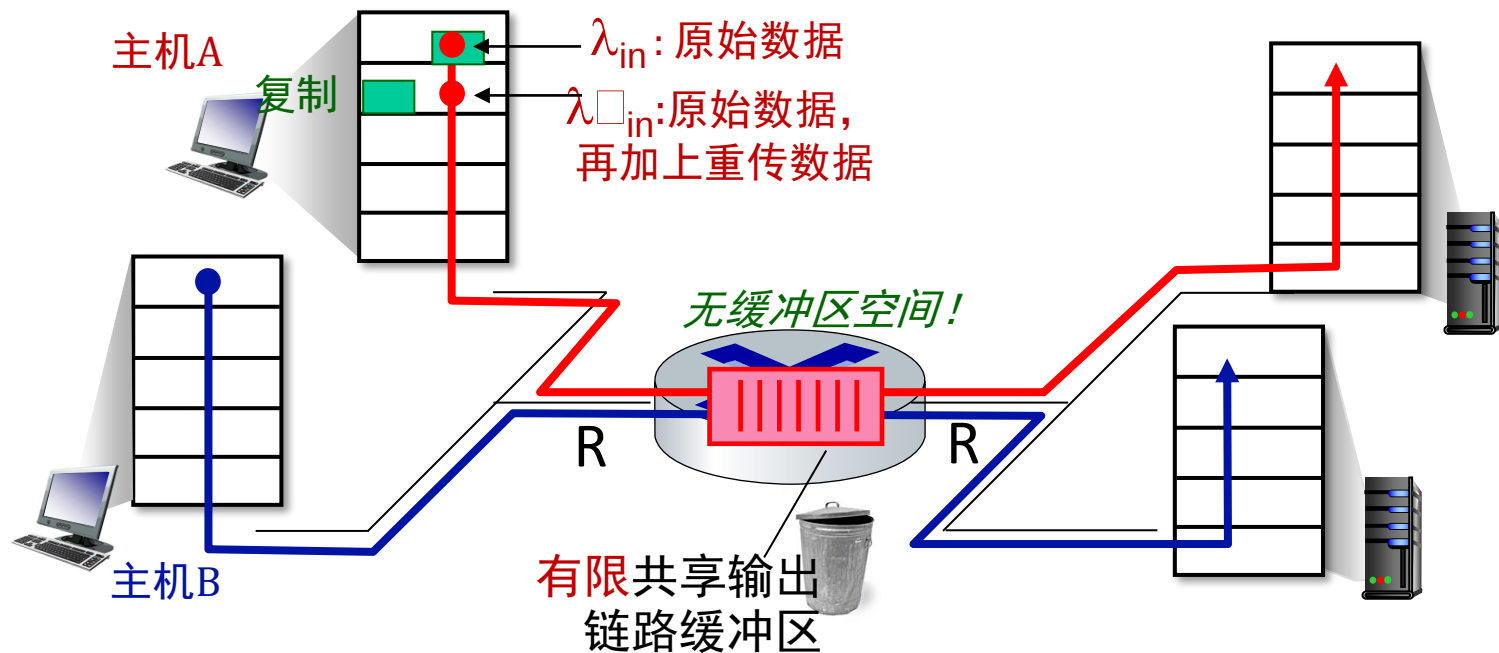
- 发送方仅在路由器缓冲区可用时发送



拥塞的成因/代价：场景2

理想化假设：**具备**完全的信息掌握（包括丢失信息）

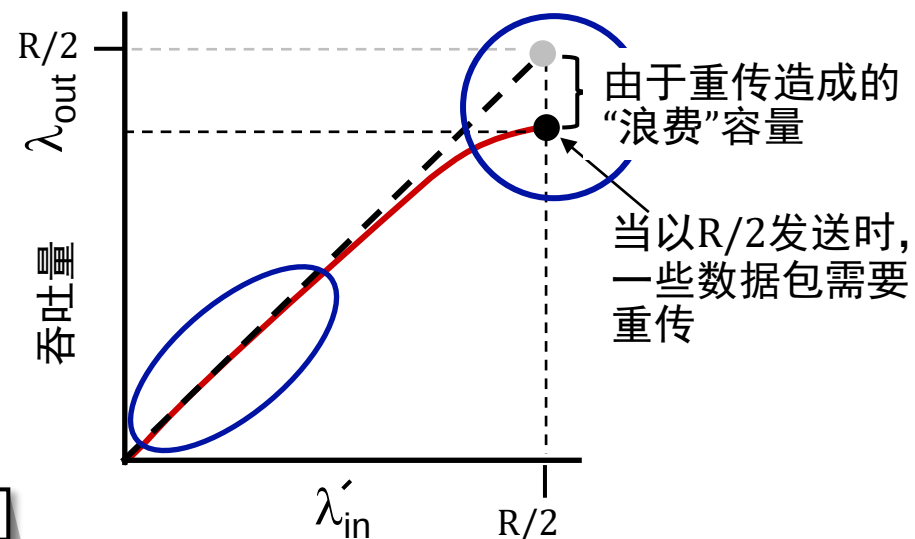
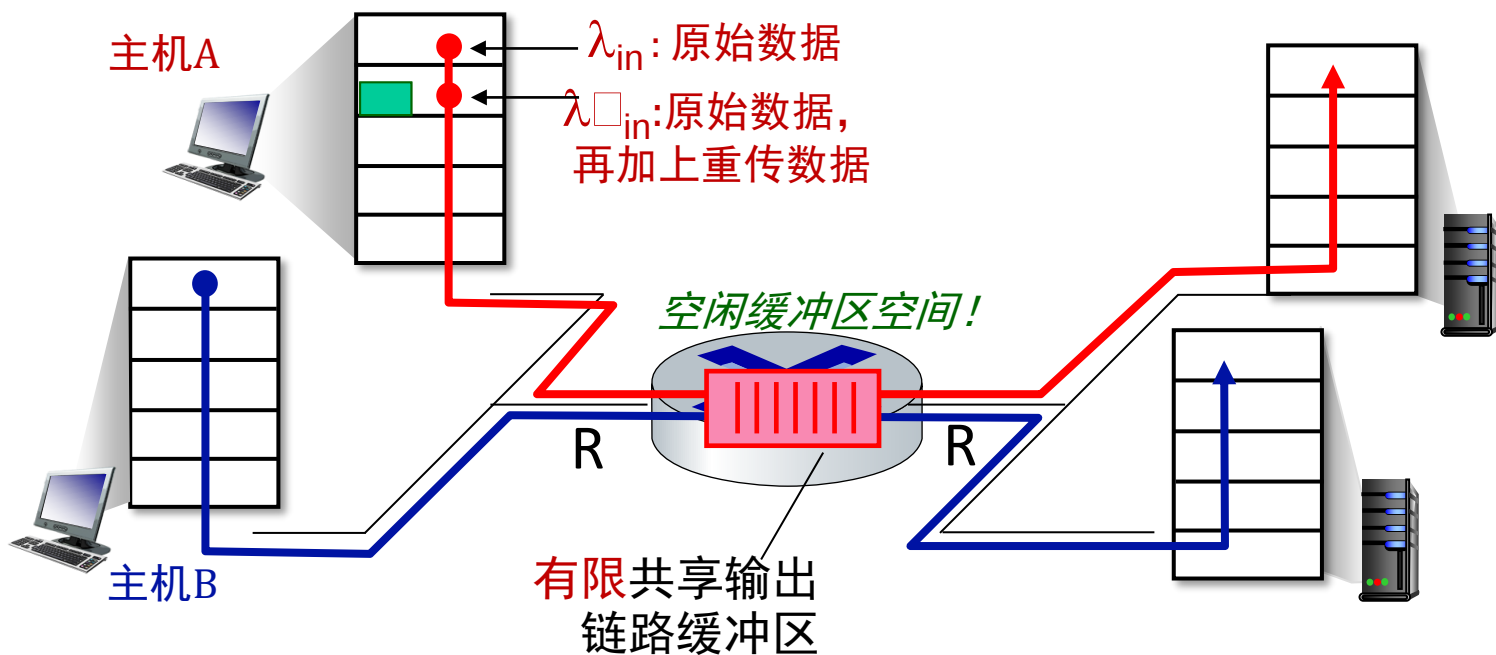
- 数据包可能因缓冲区满而在路由器处丢失
- 发送方知晓数据包何时被丢弃：仅在确认*已知*丢失时重发



拥塞的成因/代价：场景2

理想化假设：某些完美知识

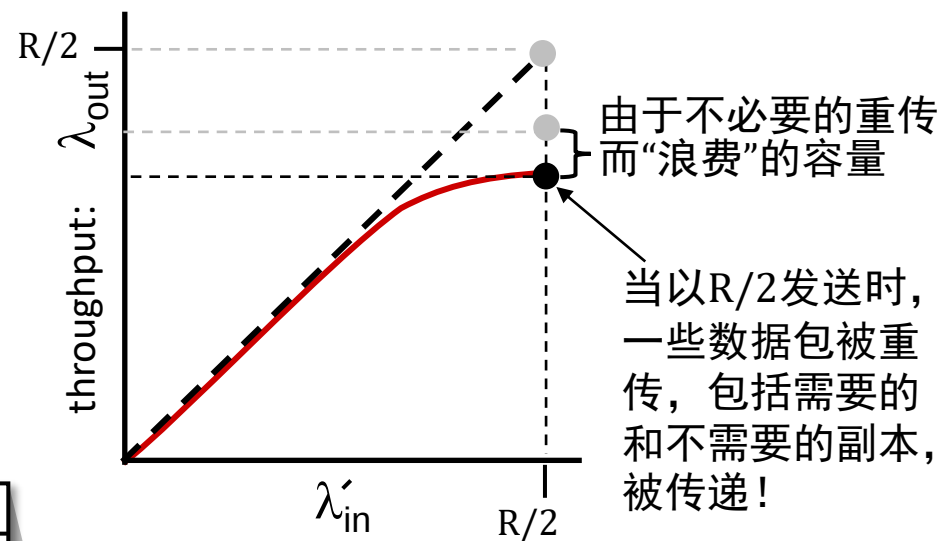
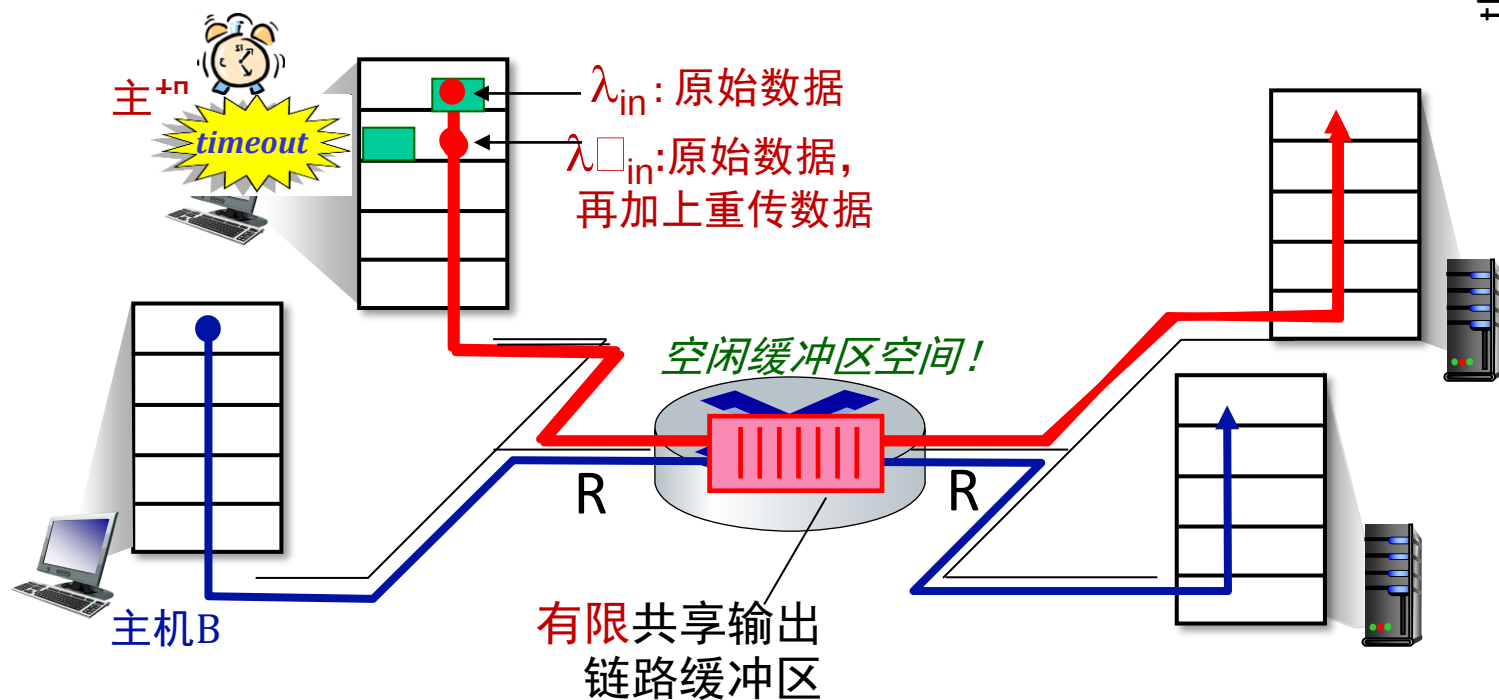
- 数据包可能因缓冲区满而在路由器处丢失
- 发送方知晓数据包何时被丢弃：仅当确认数据包已丢失时才会重发



拥塞的成因/代价：场景2

现实场景：不必要的重复

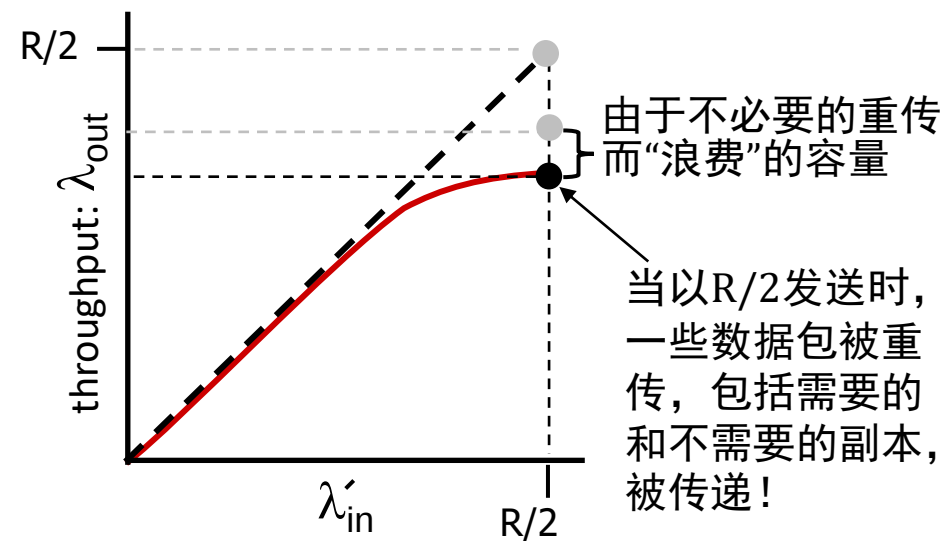
- 数据包可能会丢失，由于路由器缓冲区满而被丢弃——需要重传
- 但发送方可能会过早超时，发送两个副本，两者都被成功传递



拥塞的成因/代价：场景2

现实场景：不必要的重复

- 数据包可能会丢失，由于路由器缓冲区满而被丢弃——需要重传
- 但发送方可能会过早超时，发送两个副本，两者都被成功传递



“拥塞”的代价：（拥塞代价1）

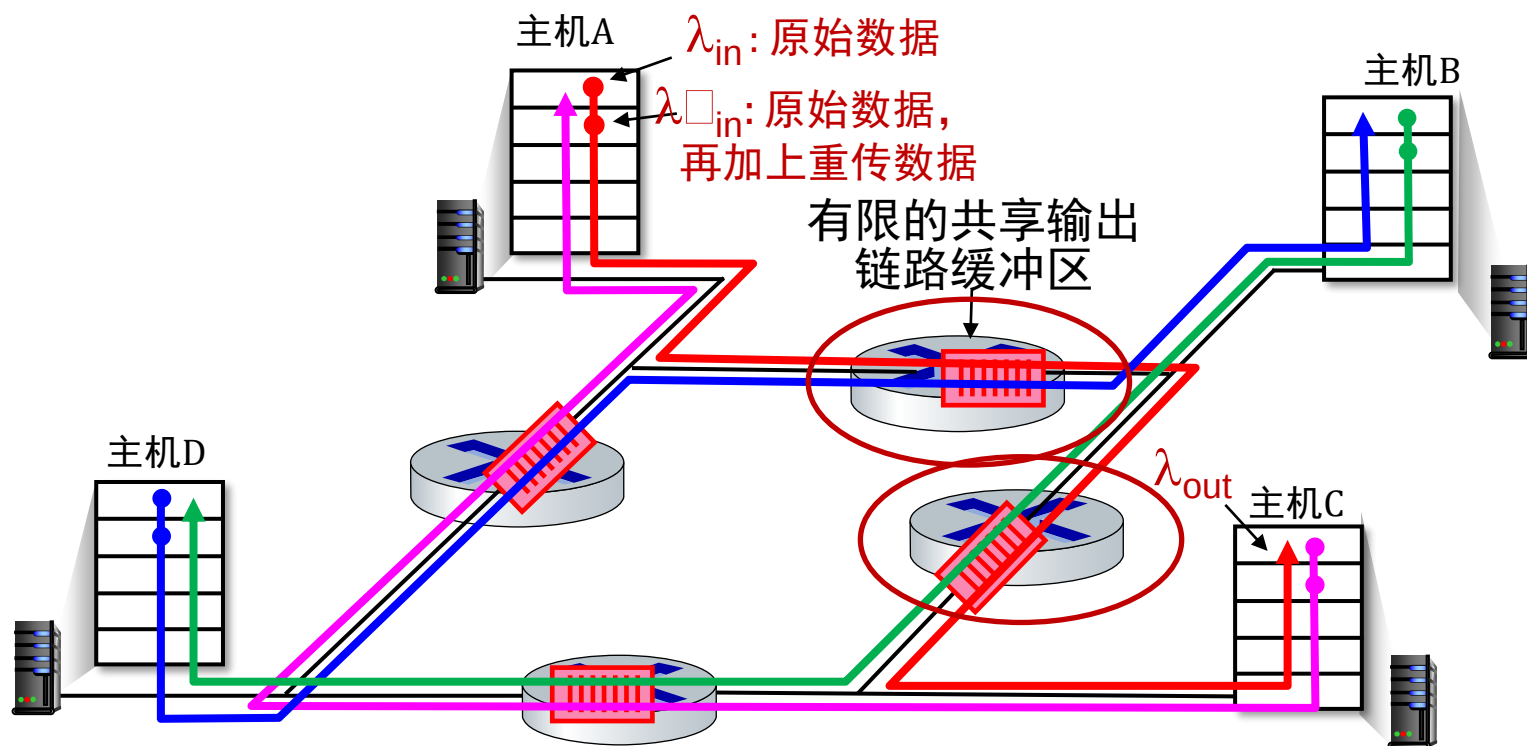
- 给定接收端的更多工作（重传）吞吐量
- 不必要的重传：链路承载了数据包的多个副本
 - 降低了可达到的最大吞吐量

拥塞的成因/代价：情景3

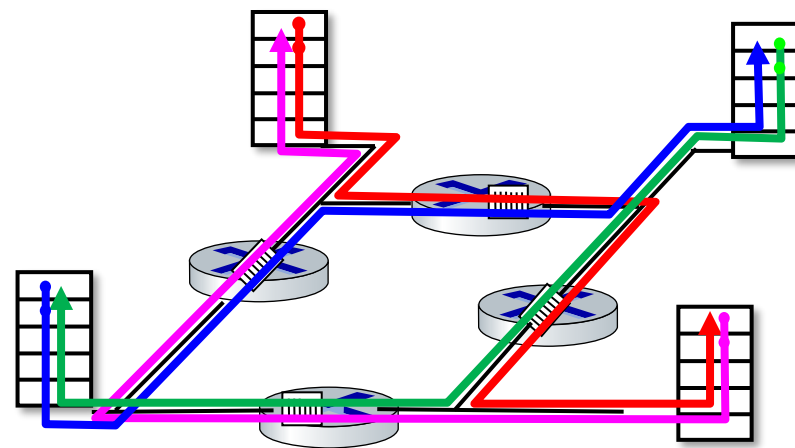
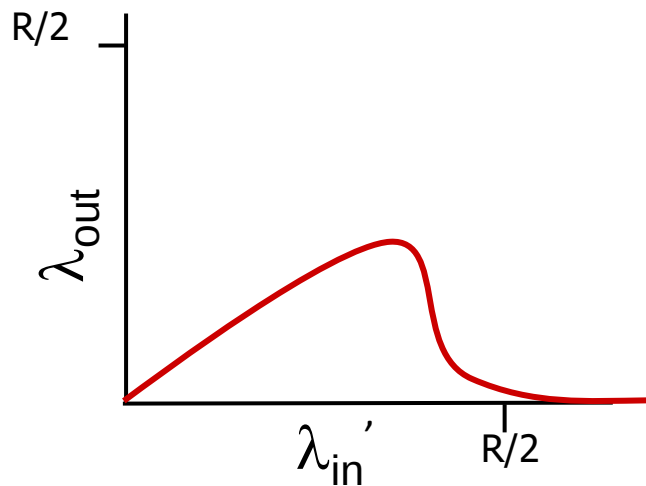
- 四个发送者
- 多跳路径
- 超时/重传

问：当 λ_{in} 和 λ_{in}' 增加时会发生什么？

A：随着红色 λ_{in}' 增加，上层队列中所有到达的蓝色数据包都被丢弃，蓝色吞吐量降为0



拥塞的成因/代价： 场景3

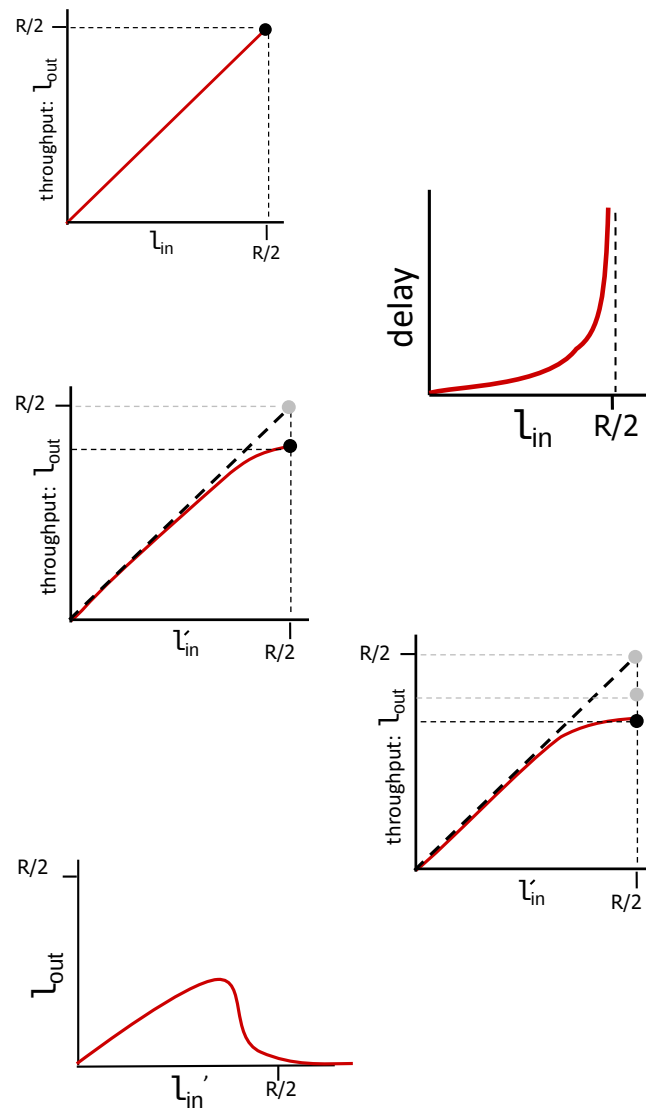


另一个“拥塞成本”：（拥塞代价2）

- 当数据包被丢弃时，任何用于该数据包的上游传输容量和缓冲都被浪费了！

拥塞的成因/代价：洞见

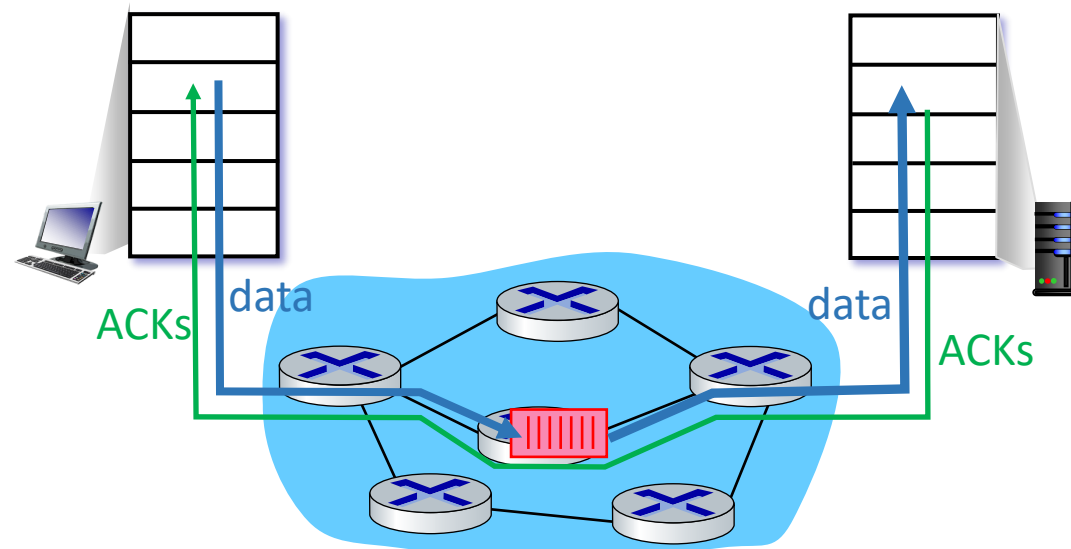
- 吞吐量永远不能超过容量
- 随着容量的接近，延迟增加
- 丢失/重传会降低有效吞吐量
- 不需要的副本进一步降低了有效吞吐量
- 上行传输容量/缓冲因下行丢失而浪费



拥塞控制方法

端到端拥塞控制：

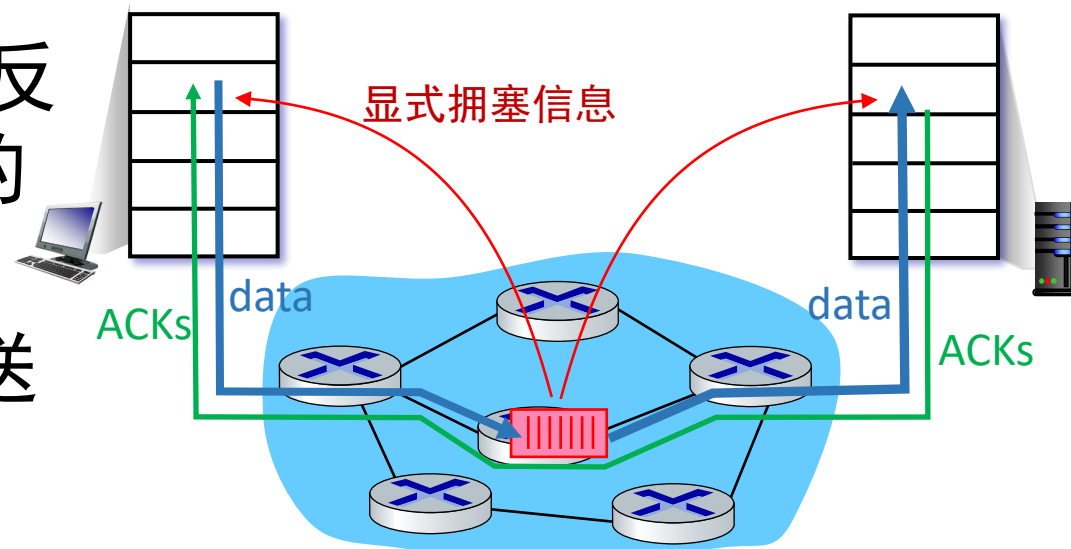
- 无来自网络的显式反馈
- 拥塞通过观察到的丢包、延迟推断
- TCP采用的方法



拥塞控制方法

网络辅助的拥塞控制：

- 路由器向发送/接收主机提供**直接**反馈，这些主机有流经拥塞路由器的流量
- 可能指示拥塞级别或明确设置发送速率
- TCP ECN、ATM、DECbit 协议



第3章：路线图

- 传输层服务
- 多路复用与多路分解
- 无连接传输：UDP
- 可靠数据传输原理
- 面向连接的传输：TCP
- 拥塞控制原理
- TCP拥塞控制
- 传输层功能的演进



复习：TCP快速重传

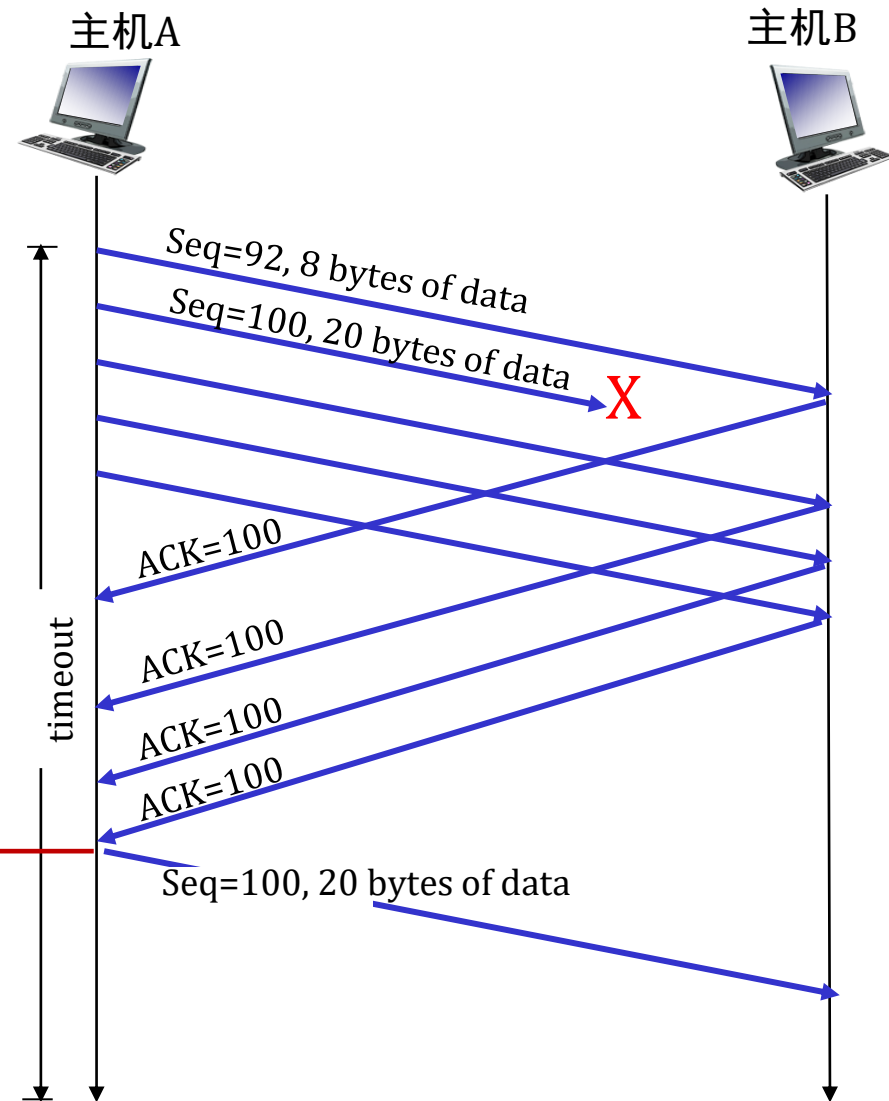
TCP fast retransmit

如果发送方收到相同数据的 3 个额外的 ACK（“三重重复 ACK”），则重新发送未确认的数据段，且该段具有最小的序列号。

- 这通常意味着未确认的数据段丢失，因此不必等待超时。



收到三个重复的ACKs表示在丢失段后收到3个段-可能丢失段。所以重新发送!



TCP拥塞控制：AIMD

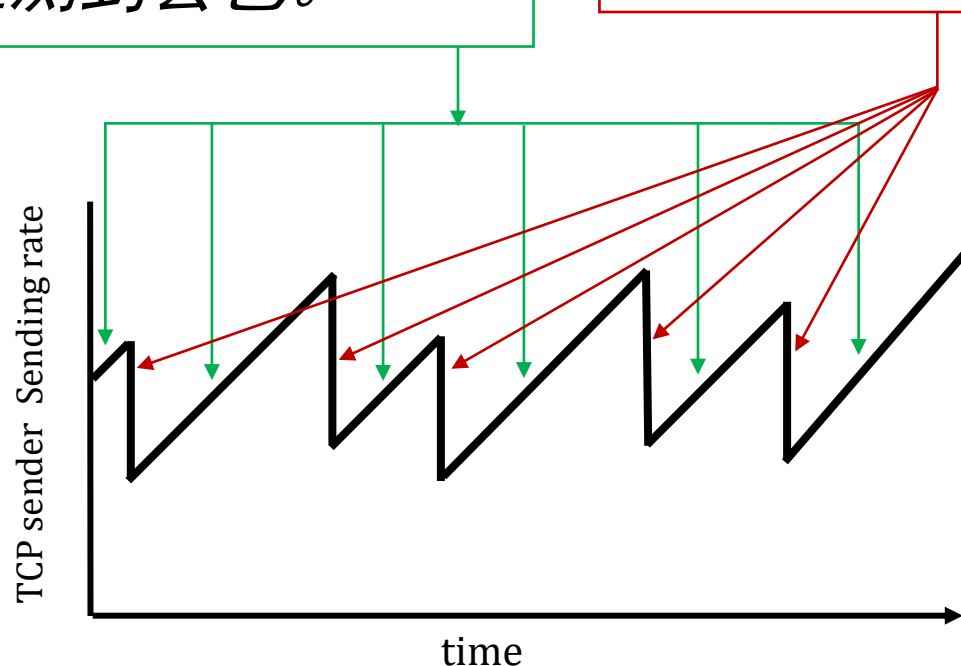
- **方法：**发送方可以逐步增加发送速率直至发生数据包丢失（拥塞），然后在丢包事件发生时降低发送速率

Additive Increase 加性增

每经过一个RTT就增加1个最大报文段大小（MSS）的发送速率，直到检测到丢包。

Multiplicative Decrease 乘性减

在每次丢失事件时将发送速率减半



AIMD锯齿状
行为：**探测**
带宽

TCP AIMD: 更多

乘法减少 细节: 发送速率

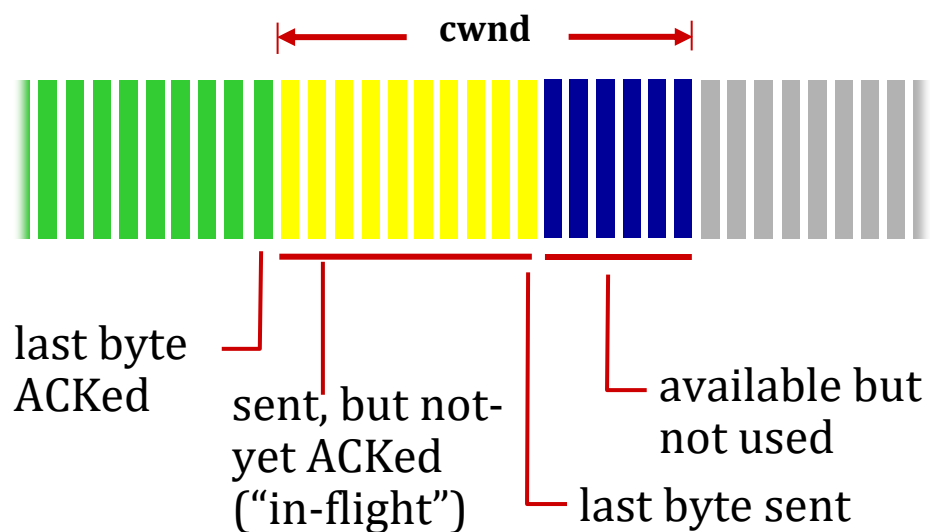
- 在检测到三次重复ACK时减半 (TCP Reno)
- 在检测到超时是降至1 MSS (最大段大小) (TCP Tahoe)

为什么 **AIMD**?

- AIMD——一种分布式、异步算法——已被证明能够:
 - 在全网范围内优化拥塞流速率!
 - 具备理想的稳定性特性

TCP拥塞控制： 细节

发送方序列号空间



TCP发送行为:

- 粗略地说：发送cwnd字节，等待RTT的ack，然后发送更多的字节

$$\text{TCP rate} \approx \frac{\text{cwnd}}{\text{RTT}} \text{ bytes/sec}$$

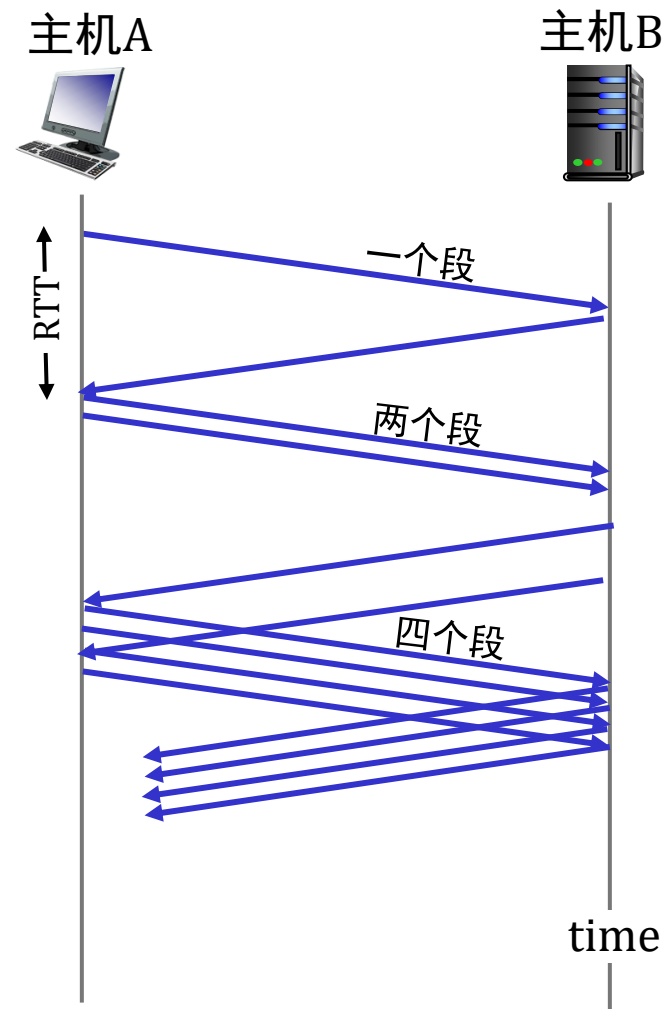
- TCP发送方限制传输：

$$\text{LastByteSent} - \text{LastByteAcked} \leq \text{cwnd}$$

- cwnd根据观察到的网络拥塞动态调整（实现TCP拥塞控制）

TCP慢启动

- 当连接开始时，速率呈指数增长，直到首次丢包事件发生：
 - 初始时 **cwnd** = 1 MSS
 - 每经过一个RTT， **cwnd** 翻倍
 - 通过每收到一个ACK， **cwnd** 递增来实现
- **总结：** 初始速率较慢，但呈指数级增长（慢启动并不慢☺）



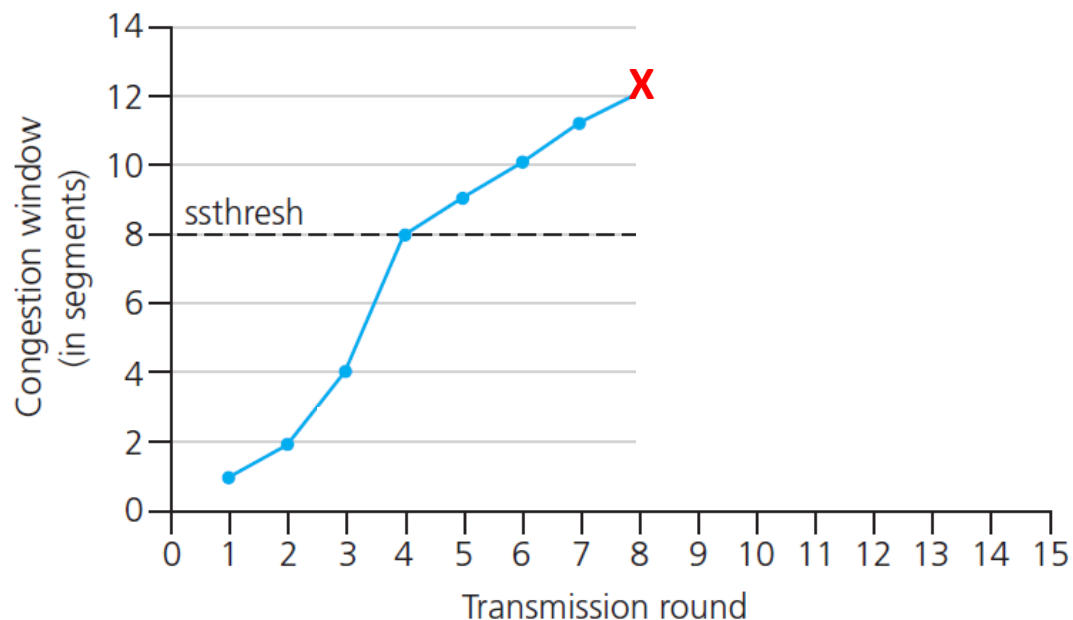
TCP: 从慢启动到拥塞避免

问: 指数增长何时应转为线性增长?

答: 当cwnd达到超时前其值的一半时。

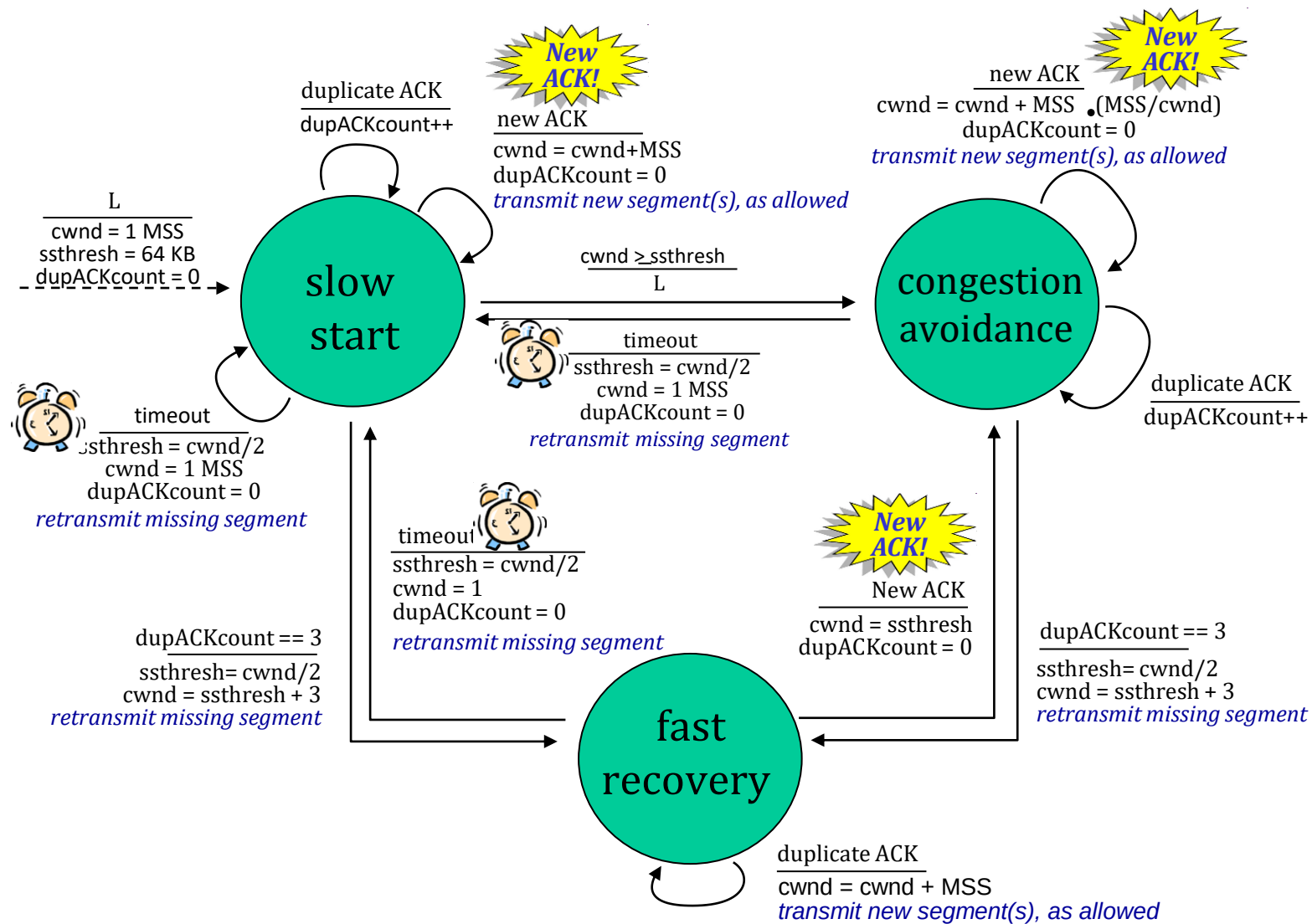
实现:

- 变量ssthresh
- 在丢包事件发生时, ssthresh被设置为cwnd 在丢包事件之前的一半



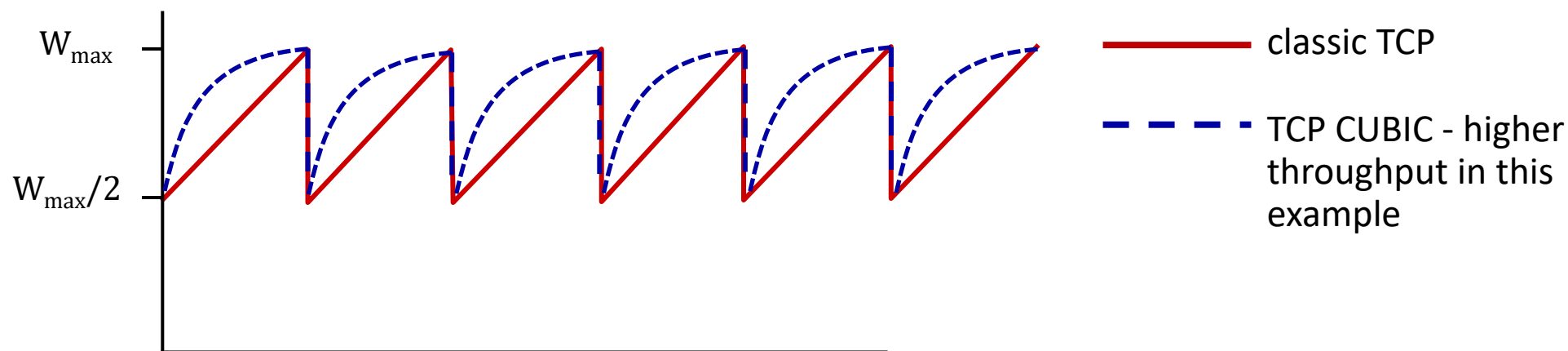
* 查看在线互动练习以获取更多示例: http://gaia.cs.umass.edu/kurose_ross/interactive/

总结：TCP拥塞控制



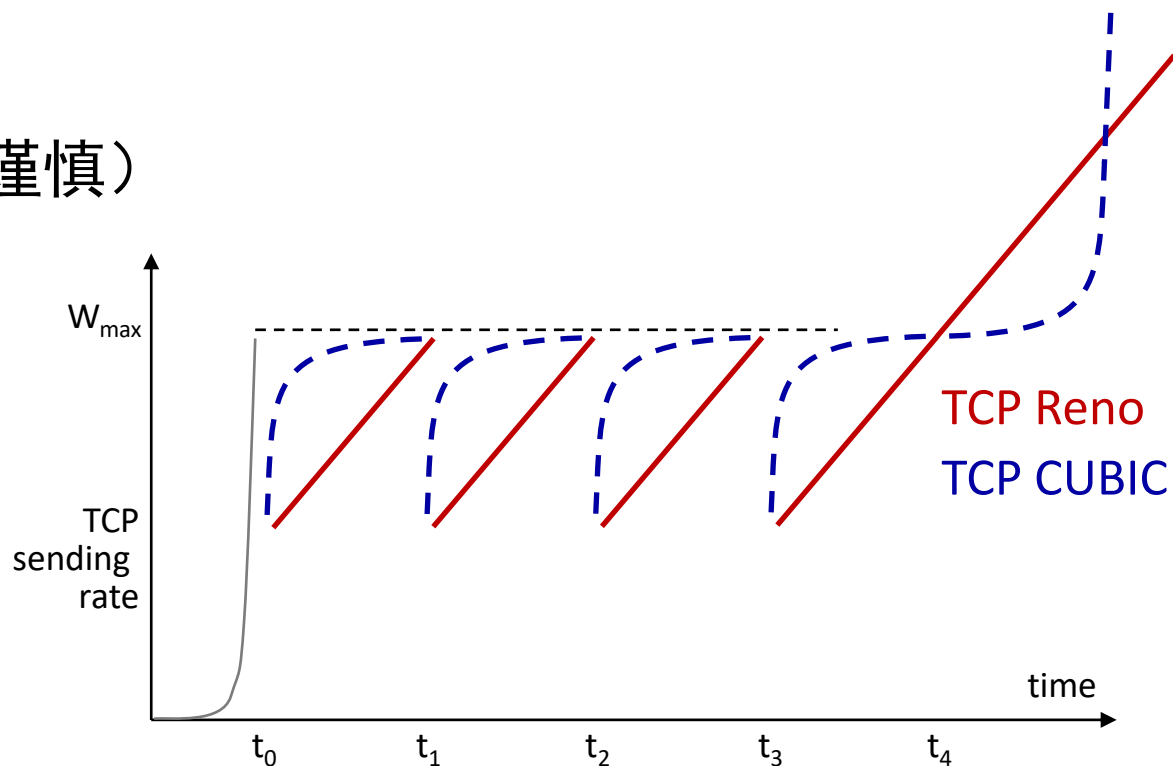
TCP CUBIC（一种TCP拥塞控制算法）

- 有没有比AIMD更好的方法来“探测”可用带宽？
- 洞察/直觉：
 - $W_{\max}$ ：检测到拥塞丢失时的发送速率
 - 瓶颈链路的拥塞状态可能（？）没有太大变化
 - 在损失后将速率/窗口减半后，最初更快地攀升至 $W_{\max}$ ，但随后更缓慢地接近 $W_{\max}$



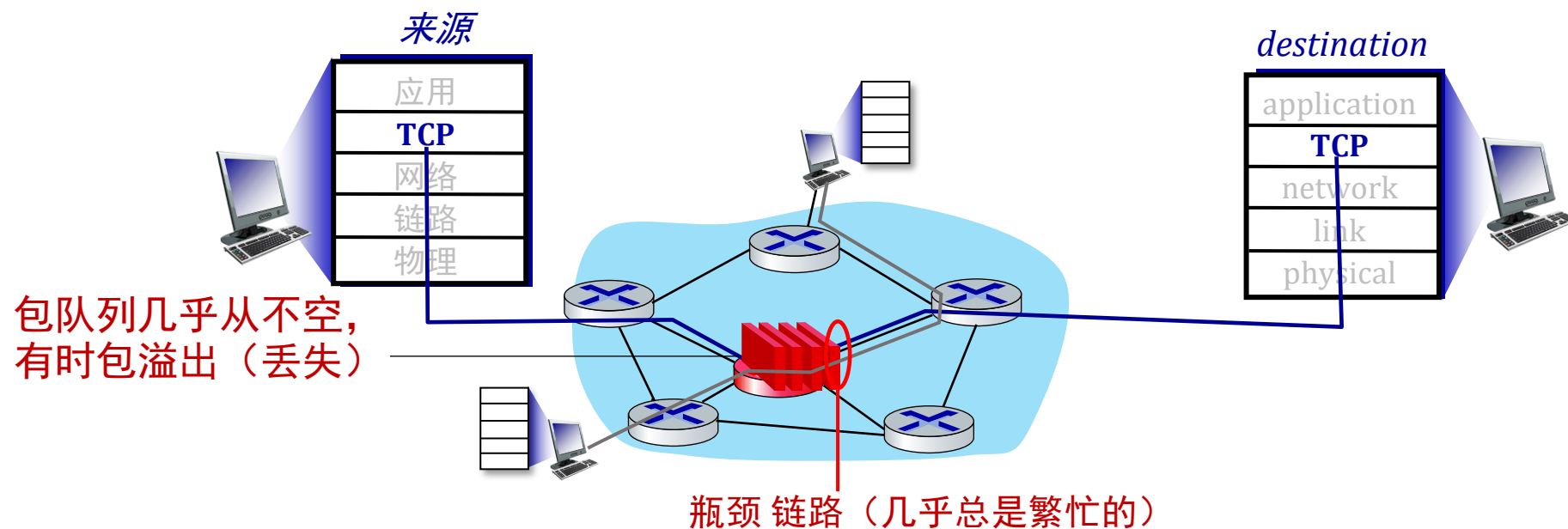
TCP CUBIC

- K: TCP窗口大小将达到 $W_{\max}$
 - K本身是 可调节的
- 将W作为当前时间与K之间距离的 **立方**函数来增加
 - 距离K越远, 增幅越大
 - 距离K越近, 增幅越小 (谨慎)
- TCP CUBIC 是Linux 中的默认协议, 也是流行Web 服务器中最常用的 TCP 协议



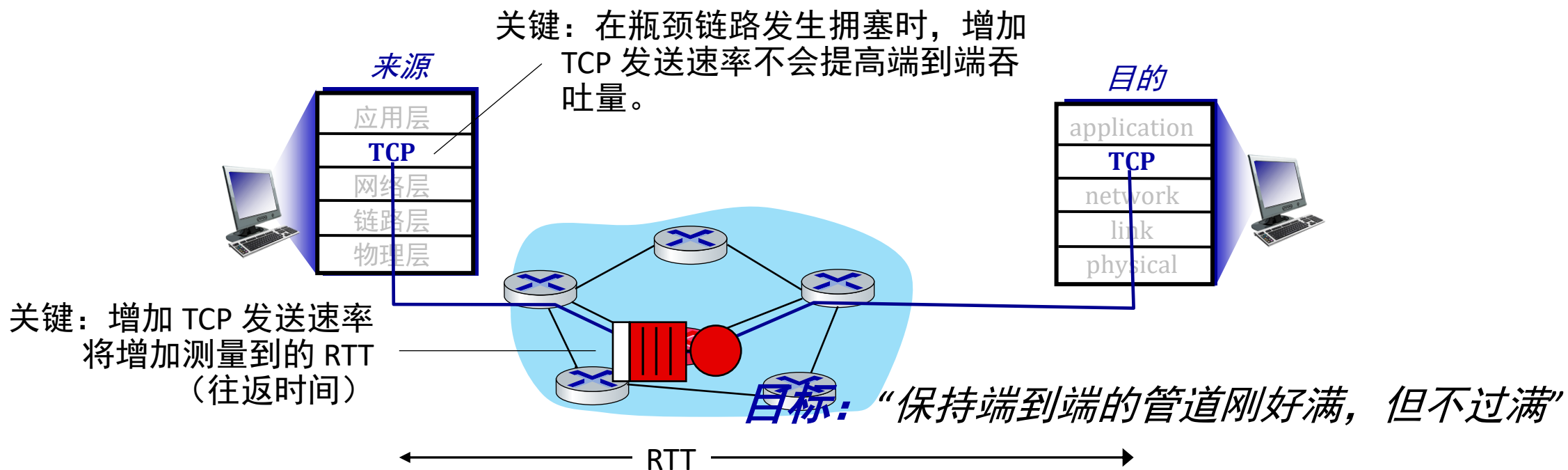
TCP与拥塞的“瓶颈链路”

- TCP（经典、CUBIC）增加TCP的发送速率，直到某个路由器的输出发生数据包丢失：**瓶颈链路**



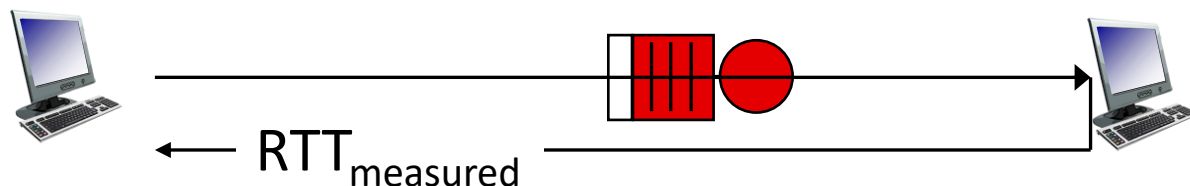
TCP与拥塞的“瓶颈链路”

- TCP（经典、CUBIC）增加TCP的发送速率，直到某个路由器的输出发生数据包丢失：**瓶颈链路**
- 理解拥塞：关注拥塞的瓶颈链路很有用



基于延迟的TCP拥塞控制

保持发送方到接收方的管道“恰到好处，但不过满”：确保瓶颈链路持续传输，同时避免高延迟/缓冲



$$\text{measured throughput} = \frac{\text{\# bytes sent in last RTT interval}}{\text{RTT}_{\text{measured}}}$$

基于延迟的方法：

- $\text{RTT}_{\min}$ - 观察到的RTT最小值（无拥塞路径）
- 无拥塞吞吐量，拥塞窗口为 cwnd 是 $\text{cwnd}/\text{RTT}_{\min}$

如果测得的吞吐量“非常接近”无拥塞吞吐量

线性增加 cwnd /* 因为路径未拥塞 */

否则如果测得的吞吐量“远低于”无拥塞吞吐量

线性减少 cwnd /* 因为路径拥塞 */

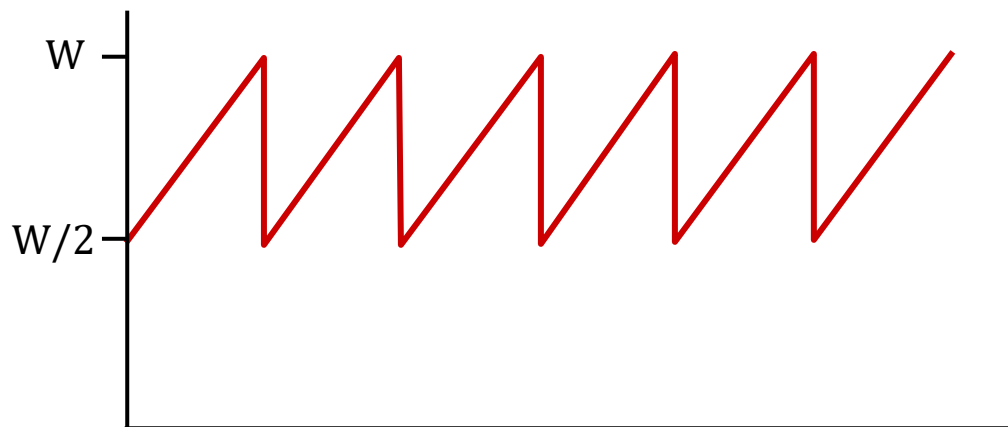
基于延迟的TCP拥塞控制

- 不引发/强制丢包的拥塞控制
- 在保持低延迟（“...但不过满”）的同时最大化吞吐量（“保持管道刚好满...”）
- 许多已部署的TCP协议采用基于延迟的方法
 - BBR在谷歌（内部）骨干网络上部署

TCP吞吐量

- 平均TCP 吞吐量 作为窗口大小、RTT的函数？
 - 忽略慢启动，假设总有数据要发送
- W ： 发生丢包的地方的窗口大小（以字节为单位）
 - 平均窗口大小（在途字节数）为 $\frac{3}{4} W$
 - 平均 吞吐量 为每个RTT的 $\frac{3}{4} W$

$$\text{avg TCP thruput} = \frac{3}{4} \frac{W}{\text{RTT}} \text{ bytes/sec}$$



TCP经过“long, fat pipes”（经高带宽路径）

- 示例：1500字节的段，100ms的RTT，期望达到10 Gbps的吞吐量
- 需要 $W = 83,333$ 个在途段
- 根据段丢失概率 L 的吞吐量[Mathis 1997]:

$$\text{TCP throughput} = \frac{1.22 \cdot \text{MSS}}{\text{RTT} \sqrt{L}}$$

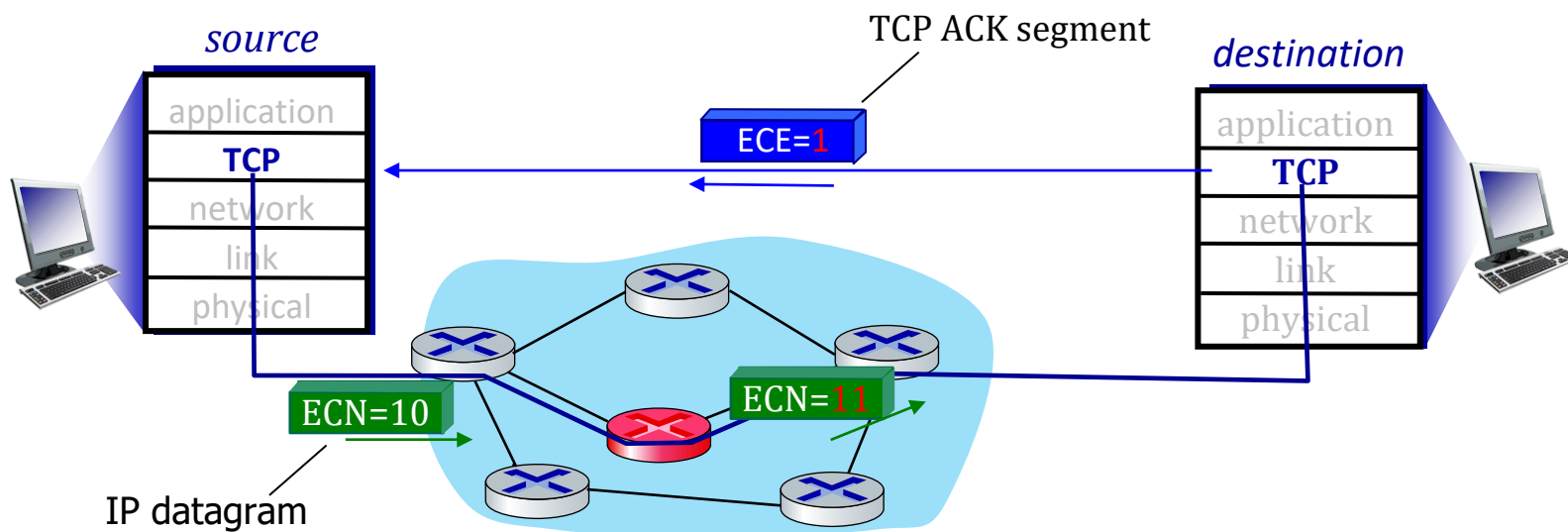
→ 要达到10 Gbps的吞吐量，需要丢失率为 $L = 2 \cdot 10^{-10}$ – 一个非常小的丢失率！

- 适用于长距离、高速场景的TCP版本

显式拥塞通知(ECN)显式拥塞通知

TCP部署通常实现**网络辅助的**拥塞控制：

- IP首部中的两个位（ToS字段）被标记**由网络路由器**以指示拥塞
 - 策略由网络运营商决定标记方式
- 拥塞指示传递到目的端
- 目的端在ACK段中设置ECE位以通知发送方拥塞情况
- 涉及IP（IP首部ECN位标记）和TCP（TCP首部C、E位标记）

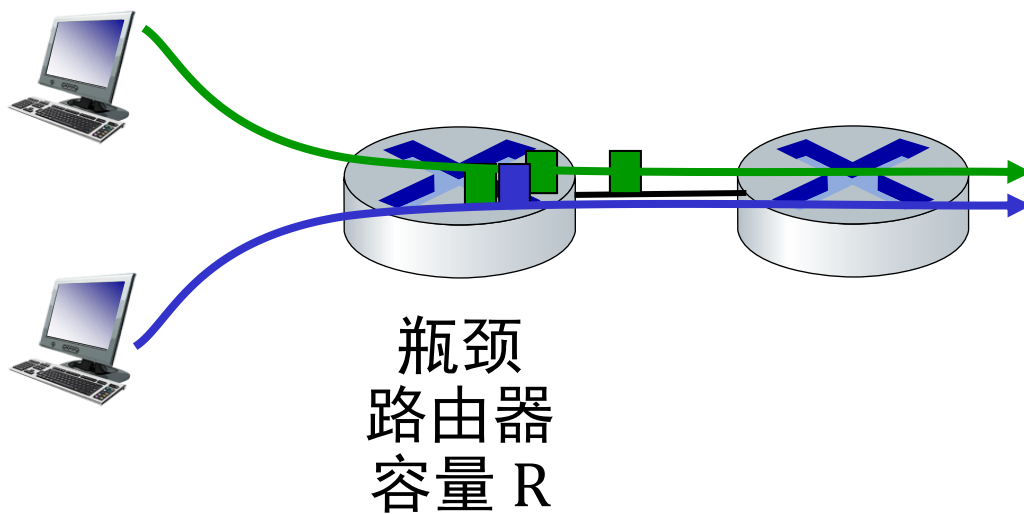


TCP 公平性

公平性目标： 如果 K 个 TCP 会话共享同一带宽为 R ，每个会话的平均速率应为 R/K

TCP连接1

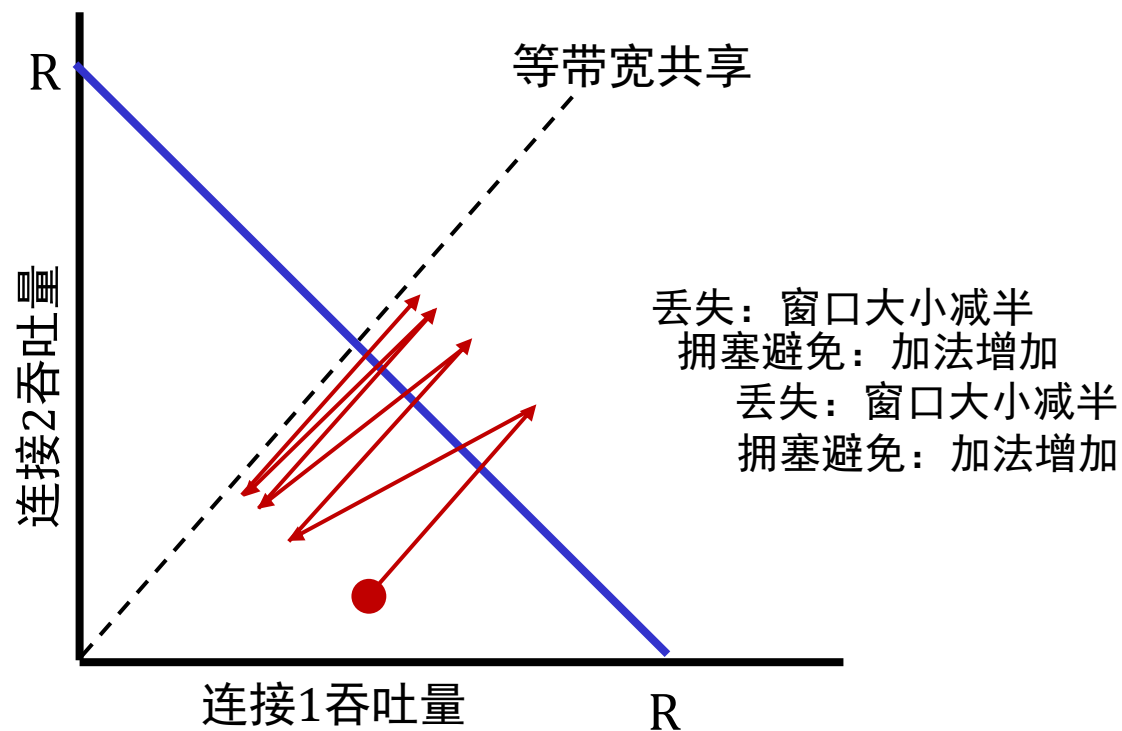
TCP连接2



问：TCP是公平的吗？

示例：两个竞争的TCP会话：

- 加法递增使斜率保持为1，随着吞吐量的增加
- 乘法递减则按比例减少吞吐量



TCP公平吗？

- A: 是的，在理想的假设下：
- 相同的RTT
 - 仅在拥塞避免阶段存在固定数量的会话

公平性：所有网络应用都必须“公平”吗？

公平性与UDP

- 多媒体应用通常不使用TCP
 - 不希望因拥塞控制而限制速率
- 转而使用UDP：
 - 以恒定速率发送音频/视频，容忍丢包
- 没有“互联网警察”监管拥塞控制的使用

公平性，并行TCP连接

- 应用程序可以打开多个两台主机之间的并行连接
- 网页浏览器就是这样做的，例如，已有9个连接时，链路速率为R：
 - 新应用请求1个TCP连接，获得速率 $R/10$
 - 新应用请求11个TCP连接，获得速率 $R/2$

传输层：路线图

- 传输层服务
- 多路复用与解复用
- 无连接传输：UDP
- 可靠数据传输原理
- 面向连接的传输：TCP
- 拥塞控制原理
- TCP拥塞控制
- 传输层功能的演进



不断演进的传输层功能

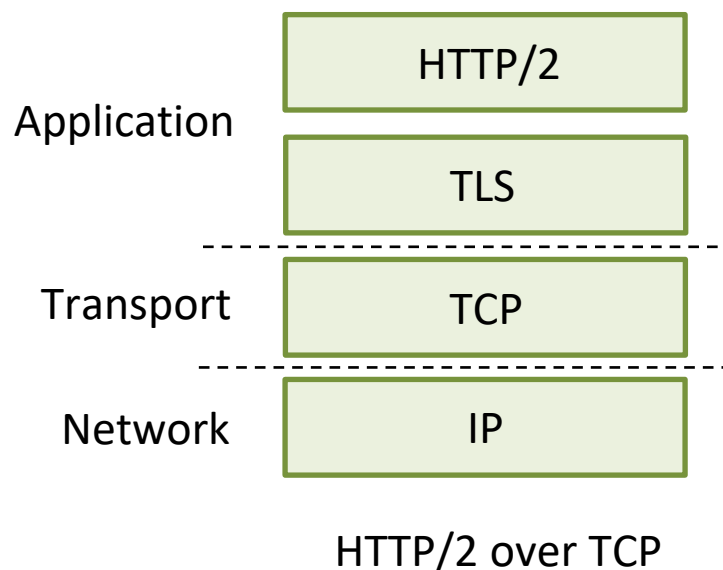
- TCP、UDP：40年来的主要传输协议
- 针对特定场景开发了不同“风味”的TCP：

Scenario	Challenges
Long, fat pipes (large data transfers)	许多数据包“在传输中”；丢包会关闭管道。
Wireless networks	由于无线链路噪声和移动性导致的丢包；TCP 将其视为拥塞丢包。
Long-delay links	非常长的RTTs
Data center networks	时延敏感
Background traffic flows	低优先级，“后台”TCP流

- 将传输层功能移至应用层，基于UDP
 - HTTP/3：QUIC快速UDP互联网连接协议

QUIC: 快速UDP互联网连接

- 应用层协议，基于UDP
 - 提升HTTP性能
 - 部署于众多Google服务器及应用程序（如Chrome、移动版YouTube应用）

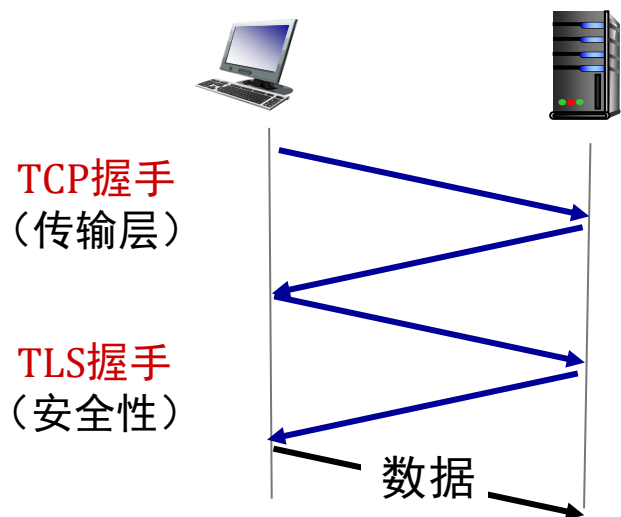


QUIC：快速UDP互联网连接

采用了我们在本章中学习的连接建立、错误控制、拥塞控制的方法

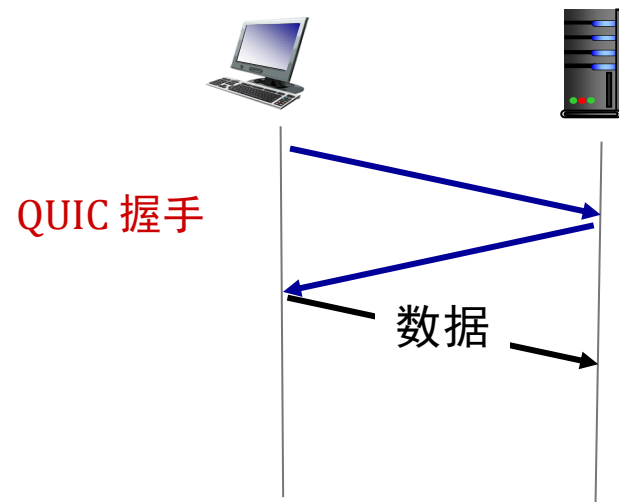
- **错误与拥塞控制：**“熟悉TCP丢失检测与拥塞控制的读者会发现，这里的算法与著名的TCP算法有很好的并行性。”[摘自QUIC规范]
 - **连接建立：**可靠性、拥塞控制、认证、加密，状态在一个RTT内建立
-
- 在单个QUIC连接上多路复用的多个应用级“流”
 - 独立的可靠数据传输、安全性
 - 通用的拥塞控制

QUIC: 连接建立



TCP (可靠性, 拥塞控制状态) +
TLS (认证, 加密状态)

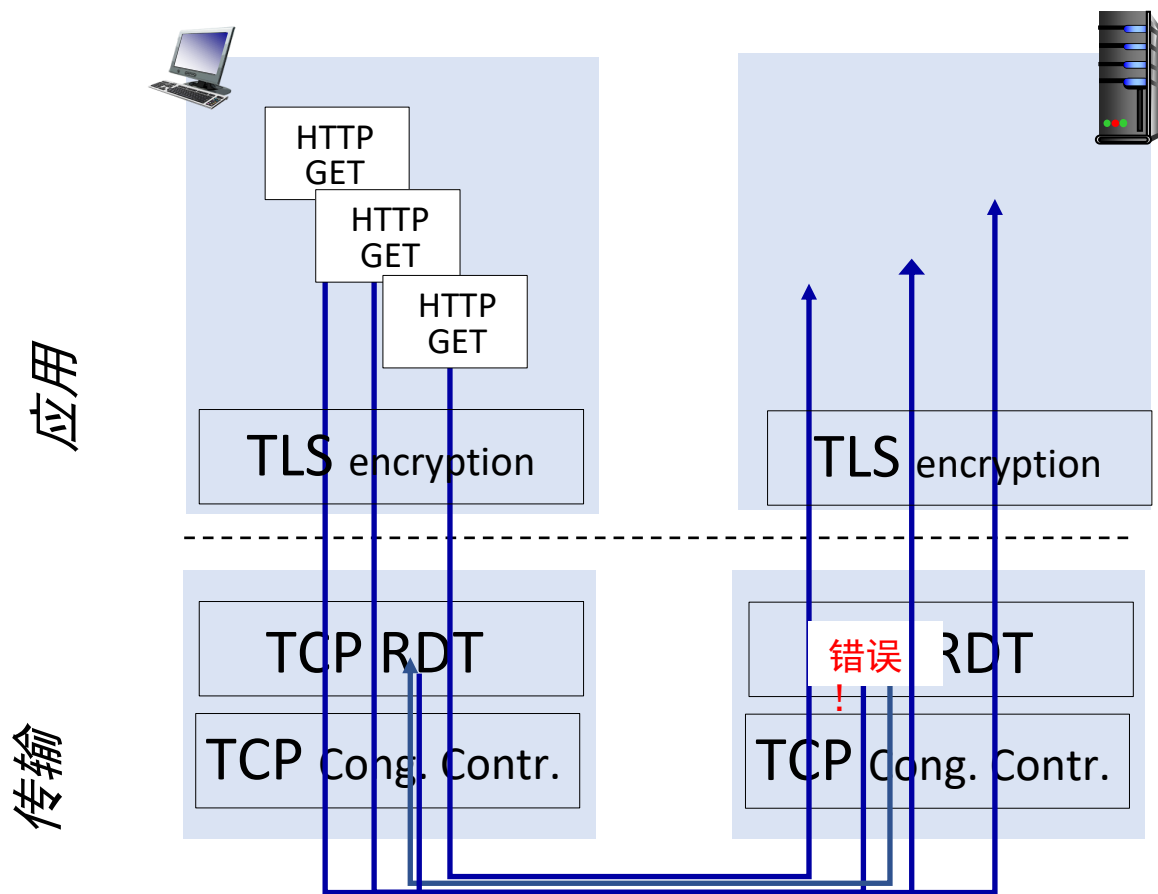
- 2次串行握手



QUIC: 可靠性, 拥塞控制, 身份验证, 加密状态

- 1次握手

QUIC: 流: 并行性, 无HOL阻塞



(a) HTTP 1.1

第3章：总结

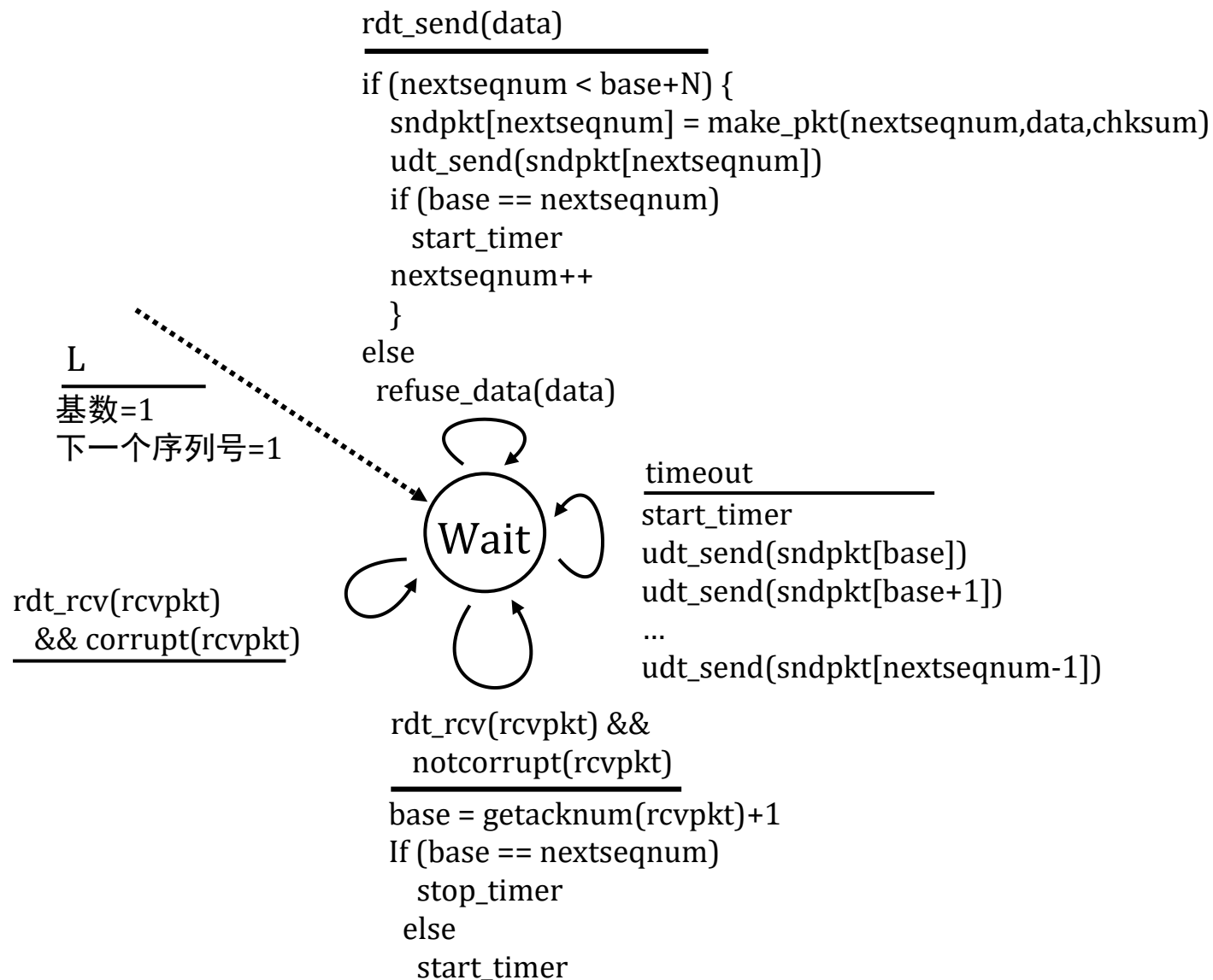
- 传输层服务背后的原理：
 - 多路复用，解复用
 - 可靠数据传输
 - 流量控制
 - 拥塞控制
- 在互联网中的实例化与实现
 - UDP
 - TCP

接下来：

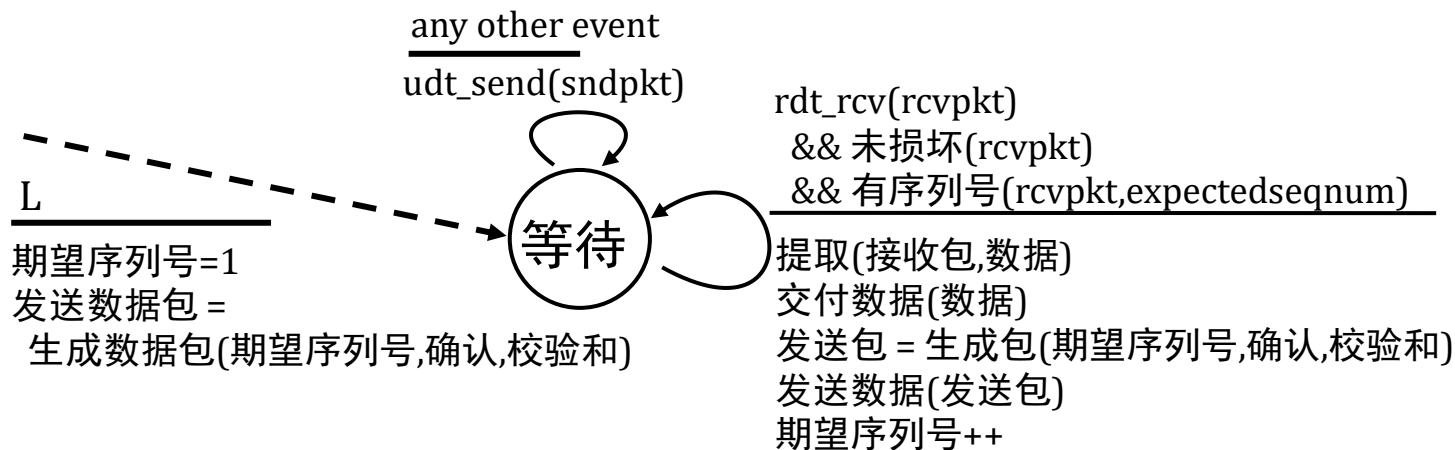
- 离开网络“边缘”（应用层、传输层）
- 进入网络“核心”
- 两个网络层章节：
 - 数据平面
 - 控制平面

附加第3章幻灯片

回退N：发送方扩展有限状态机



回退N协议：接收方扩展有限状态机



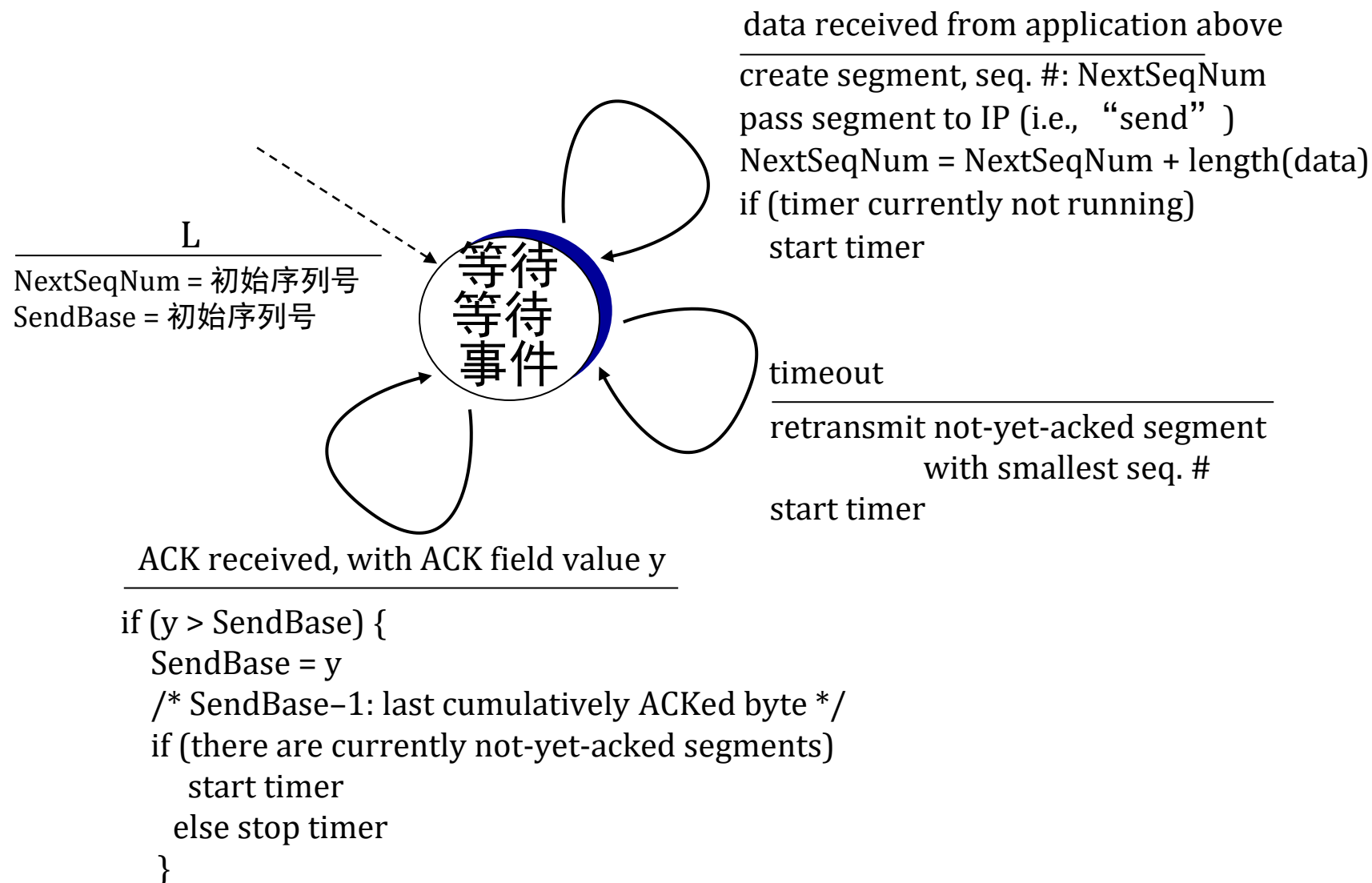
仅确认（ACK-only）：总是为正确接收的、具有最高**顺序**序列号的数据包发送确认

- 可能产生重复的确认
- 只需记住**期望的序列号**

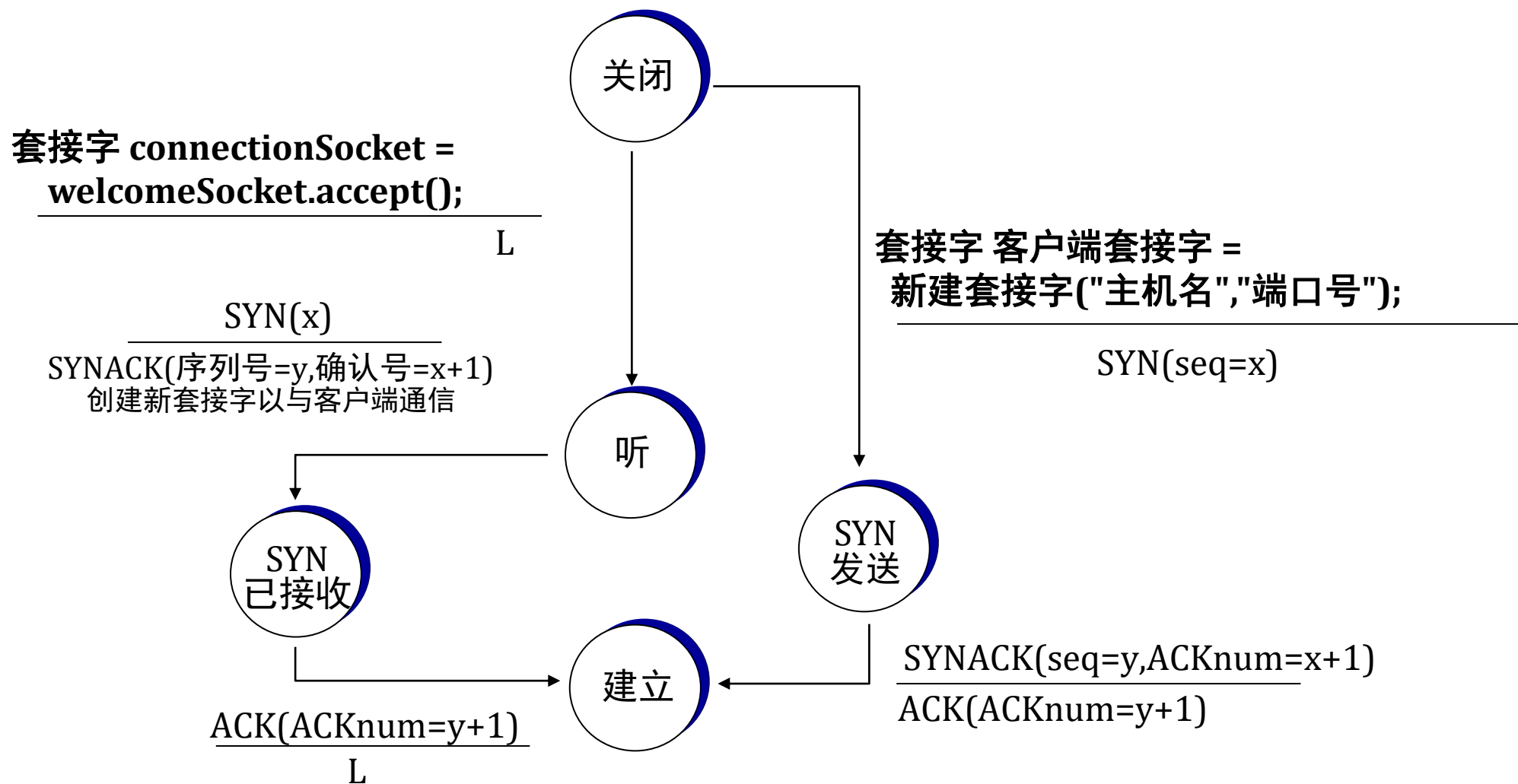
■ 乱序数据包：

- 丢弃（不缓存）：**接收端无缓冲！**
- 重新确认具有最高顺序序列号的数据包

TCP发送方（简化版）



TCP 三次握手有限状态机



关闭TCP连接

客户端状态

建立

FIN_WAIT_1

FIN_WAIT_2

TIMED_WAIT

CLOSED

clientSocket.close()

can no longer
send but can
receive data

wait for server
close

timed wait
for 2*max
segment lifetime



FINbit=1, seq=x

ACKbit=1; ACKnum=x+1

FINbit=1, seq=y

ACKbit=1; ACKnum=y+1

can still
send data

can no longer
send data

服务器状态

建立

CLOSE_WAIT

LAST_ACK

CLOSED